

Bachelorarbeit

Erweiterung der Rust-Standardbibliothek für das Betriebssystem D3OS

vorgelegt von

Mate-Trpimir Jakulj

Abteilung Betriebssysteme
Prof. Dr. Michael Schöttner
Heinrich-Heine-Universität Düsseldorf

14. September 2026

Erstgutachter:	Univ.-Prof. Dr. M. Schöttner
Zweitgutachter:	Univ.-Prof. Dr. M. Mauve
Betreuer:	Niklas Sombert

Inhaltsverzeichnis

1 Einleitung **1**
- 1.1 Motivation und Ausgangslage . . . 1
- 1.2 Ziel und Aufbau . . . 2
- 1.3 Eigenanteil und fremde Beiträge . . . 2

2 Grundlagen **4**
- 2.1 `core`, `alloc`, `std` und die Portierungsschicht . . . 4
- 2.2 Kopplung von Compiler und Betriebssystem . . . 5
- 2.3 Der übernommene Port und verwandte Ports . . . 7

3 Konzept **10**
- 3.1 Vorgehen . . . 10
  - 3.1.1 Anker-Sprungkette und Rebase statt Merge . . . 10
  - 3.1.2 Von der Vorlage zum Backend . . . 11
  - 3.1.3 Einordnung und Behandlung von Abweichungen . . . 12
- 3.2 Architektur . . . 13

4 Toolchain-Upgrade: rustc 1.90 → 1.94 **18**
- 4.1 Ausgangszustand und Impact-Analyse . . . 18
- 4.2 Etappe A: 1.90 → 1.91 . . . 19
- 4.3 Etappe B: 1.91 → 1.92 . . . 22
- 4.4 Etappe C: 1.92 → 1.94 . . . 26
- 4.5 Bilanz der drei Etappen . . . 28

5 Die Standardbibliothek erweitern **30**
- 5.1 Dynamische Speicherverwaltung . . . 30
- 5.2 Standardeingabe . . . 32
- 5.3 Datei-Ein- und -Ausgabe . . . 34
- 5.4 Threads . . . 37
- 5.5 Herkunft des Quellcodes . . . 39

6 Evaluation **41**
- 6.1 Wahl des Nachweisverfahrens . . . 41
- 6.2 Aufteilung der Testanwendung . . . 42
- 6.3 Ergebnisse und ihre Grenzen . . . 42
- 6.4 Diskussion . . . 45

7 Fazit und Ausblick **47**
- 7.1 Kritische Reflexion des Vorgehens . . . 47

7.2 Zusammenfassung und Ausblick 49

Anhang **51**
A Commit-Chronologie 51
B Code-Herkunft farblich markiert 52

Literatur **57**

Abbildungsverzeichnis **60**

Tabellenverzeichnis **61**

Kapitel 1

Einleitung

1.1 Motivation und Ausgangslage

D3OS ist ein neues verteiltes Betriebssystem für Rechenzentren, das der Lehrstuhl für Betriebssysteme der Heinrich-Heine-Universität Düsseldorf in Rust entwickelt [1]. Die Rust-Standardbibliothek gliedert sich in drei aufeinander aufbauende Crates, nämlich `core`, `alloc` und `std`, die sich darin unterscheiden, welche Anforderungen sie an das darunterliegende System stellen [2]. Anwendungen für D3OS waren bislang auf die beiden unteren Ebenen beschränkt, da `std` nicht zur Verfügung stand. Abschnitt 2.1 stellt diese Dreiteilung im Einzelnen dar.
Baranowski hat `std` nach D3OS portiert, ein eigenes Ziel im Compiler angelegt und mit `stdout` eine erste Ausgabefunktion umgesetzt [3]. Diese Vorarbeit stellt die Infrastruktur bereit, nicht jedoch die Anbindung der darüberliegenden Betriebssystemdienste: Das Anfordern von Speicher, das Lesen einer Zeile von der Tastatur, das Anlegen einer Datei und das Starten eines Threads führten zu einer Fehlermeldung oder zum Programmabbruch. Im Ausblick seiner Arbeit benennt Baranowski die weiteren Schritte, namentlich `stdin` für `stdio` sowie darauf aufbauend eigene Module wie die Dateioperationen [3].
An dieser Fortführung setzt die vorliegende Arbeit an. Die Standardbibliothek bildet die Grundlage portabler Rust-Software und steht jedem Rust-Crate standardmäßig zur Verfügung [4]. Daraus ergibt sich der Nutzen für D3OS: Eine externe Crate lässt sich nur dann übernehmen, wenn sie entweder ausdrücklich auf `std` verzichtet oder das Zielsystem `std` bereitstellt. Erfolgt die Entwicklung für D3OS ohne `std`, ist stattdessen ein direkter Zugriff auf die D3OS-eigenen Crates wie `concurrent` und `naming` erforderlich. Dieser Weg ist möglich, bindet den entstehenden Code jedoch an dieses eine Betriebssystem.
Hieraus resultiert eine zweite Aufgabenstellung. Der übernommene Port ist gegen `rustc 1.90` gebaut, während D3OS seine eigene Toolchain fortlaufend aktualisiert. Seit dem Stand von Baranowskis Arbeit hat der Hauptzweig fünf Versionswechsel durchlaufen, über `nightly-2025-10-20` und `nightly-2026-01-01` bis auf `nightly-2026-07-09`. Ohne eine Aktualisierung der Compiler-Version verliert der Port die Kompatibilität zum Ziel-Betriebssystem. Die Aktualisierung steht deshalb vor der Erweiterung, da die vier neuen Module sonst gegen eine Compiler-Version entstünden, die D3OS nicht mehr verwendet.
Aus beiden Aufgabenstellungen leiten sich die beiden Leitfragen ab, die den Rahmen dieser Arbeit bilden:

1. Woran scheitert die Aktualisierung einer betriebssystemspezifischen Standardbibliothek auf eine neuere Compiler-Version?

2. In welchem Umfang lassen sich bestehende Plattform-Backends der Rust-Standardbibliothek als Vorlage übernehmen, und an welchen Stellen ist eine eigenständige Implementierung erforderlich?

1.2 Ziel und Aufbau

Aus den beiden Leitfragen ergibt sich die Zweiteilung des Hauptteils. Der erste Teil aktualisiert den Port von `rustc 1.90` auf `1.94`, der zweite erweitert die Standardbibliothek um vier Module. Beide Teile stehen in einem Abhängigkeitsverhältnis, da die Erweiterung auf derjenigen Compiler-Version aufsetzt, die die Aktualisierung erst bereitstellt.

- Kapitel 4: Aktualisierung des Ports von `rustc 1.90` auf `1.94`
- Abschnitt 5.1: dynamische Speicherverwaltung
- Abschnitt 5.2: Standardeingabe
- Abschnitt 5.3: Datei-Ein- und -Ausgabe
- Abschnitt 5.4: Threads

Die resultierende Gliederung ordnet sich um diesen Kern wie folgt: Kapitel 2 legt die für das Verständnis des Hauptteils erforderlichen Grundlagen dar. Kapitel 3 dokumentiert das Vorgehen sowie die Architektur, gegen die beide Teile arbeiten. Kapitel 6 überprüft das Ergebnis anhand von vier Anwendungen im laufenden System. Kapitel 7 fasst die Ergebnisse zusammen, gibt einen Ausblick und beantwortet die beiden Leitfragen abschließend.

1.3 Eigenanteil und fremde Beiträge

Die vorliegende Arbeit baut auf Baranowskis Port auf. Unverändert übernommen sind seine Anbindung der threadlokalen Daten in `sys/thread_local/key/d3os.rs` samt dem zugehörigen Feld im Thread Environment auf der D3OS-Seite sowie `os.rs` und `common.rs` seiner Portierungsschicht [3]. Als Vorlage dienten daneben die Backends anderer Plattformen unter `library/std/src/sys/`. Abschnitt 5.5 dokumentiert am Beispiel von `std::fs` im Einzelnen, welche Teile daraus übernommen wurden.
Aus der Betreuung stammen vier Eingriffe. Der zunächst vorgelegte Lösungsansatz für das fehlende Zeilenende wurde verworfen und stattdessen auf eine Korrektur an der Quelle verwiesen. Den Umlaut-Defekt im Terminal hat die Betreuung aus dem Aufgabenbereich genommen. Die Anker der Sprungkette in Abschnitt 3.1.1 gehen auf ihren Vorschlag zurück. Die Idee, `dlmalloc` statt `linked_list_allocator` zu nehmen, wurde durch die Betreuung bestätigt, die zugleich `xous` als Vorlage vorgeschlagen hat.
Fremder Code ist an einer Stelle in das Ergebnis eingegangen: Die Funktion `open` des Naming Service stammt aus der parallelen Weiterentwicklung von Tom Gehrmann und wurde übernommen, anstatt eine eigene Implementierung zu erstellen. Die daneben vorgenommene Anpassung der Öffnungs-Flags ist Eigenanteil.

KI-Werkzeuge kamen in drei Funktionen zum Einsatz: am Code Claude Code in den Modellen Opus 4.7, 4.8 und 5, am Text DeepL Write sowie an den Abbildungen Google Gemini in Verbindung mit Claude Code.
Claude Code wurde zum Verstehen und Analysieren fremden Codes, zur Recherche, zur Einordnung von Fehlermeldungen, für Textentwürfe und für die Formatierung in LaTeX eingesetzt. Dabei sind auch Code-Entwürfe entstanden. Das Werkzeug war durchgehend so konfiguriert, dass es keine Änderungen selbstständig schreibt. Sämtliche Änderungen an D3OS und am Fork habe ich manuell selbst vorgenommen, teils nach einem solchen Entwurf, teils gegen ihn. Der Code ist damit in Zusammenarbeit entstanden. Jede Zeile wurde vor dem Commit gelesen und am laufenden System überprüft. Diese Überprüfung ist methodisch erforderlich, da mehrere Empfehlungen ihr nicht standgehalten haben. Abschnitt 7.1 führt die betreffenden Fälle im Einzelnen auf.

Kapitel 2

Grundlagen

2.1 `core`, `alloc`, `std` und die Portierungsschicht

Die Dreiteilung der Rust-Standardbibliothek in `core`, `alloc` und `std` [2] wird im Folgenden nicht abstrakt eingeführt, sondern zunächst an einem Beispiel aus dem D3OS-Repository entwickelt und im Anschluss verallgemeinert. Die Trennung der drei Ebenen ist dort unmittelbar an der Baukonfiguration zweier Anwendungen ablesbar. Die Anwendungen `hello` und `stdlibtest` verfügen jeweils über eine eigene `Makefile.toml`. Diese unterscheiden sich an zwei Stellen und maßgeblich ist hier die zweite: der Aufruf, mit dem übersetzt wird. `hello` gibt als Ziel die Datei `d3os_application.json` an und setzt zusätzlich das Flag `-Z build-std=core,alloc`. Damit wird Cargo angewiesen, die beiden unteren Ebenen der Standardbibliothek bei jedem Durchlauf aus den Quellen mitzuübersetzen [5]. Erforderlich ist dies, weil für dieses Ziel keine vorgebauten Artefakte bereitliegen. `stdlibtest` gibt als Ziel `x86_64-unknown-d3os` an und kommt ohne das Flag aus. Es setzt voraus, dass in der Toolchain bereits eine fertige Standardbibliothek für D3OS vorliegt, einschließlich `std`. Diese Trennung zieht sich durch das gesamte System. Der Kernel und alle übrigen Anwendungen folgen dem ersten Weg, während `stdlibtest` als einzige Anwendung den zweiten Weg beschreitet. Der Unterschied zwischen diesen beiden Aufrufen bezeichnet den Gegenstand der vorliegenden Arbeit. Abbildung 2.1 zeigt die drei Ebenen und die beiden Bauwege im Zusammenhang.

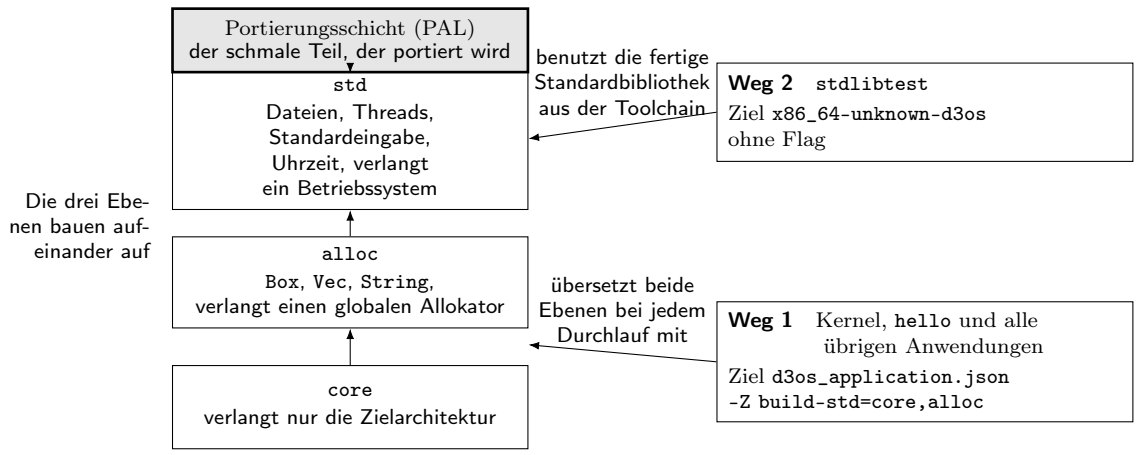


Abbildung 2.1: Die drei Ebenen der Rust-Standardbibliothek und die beiden Bauwege in D3OS.

Die drei Ebenen bauen aufeinander auf und unterscheiden sich darin, welche Anforderungen sie an ihre Umgebung stellen. `core` setzt nichts außer der Zielarchitektur voraus. `alloc` kommt hinzu, sobald `Box`, `Vec` oder `String` benötigt werden, und verlangt einen globalen Allokator. `std` baut auf beiden auf und ist die erste Ebene, die ein Betriebssystem voraussetzt, nämlich für Dateien, Threads, Standardeingabe und Uhrzeit [2]. Für die Portierung folgt daraus, dass `core` und `alloc` für D3OS bereits funktionsfähig waren, sobald der Compiler das Ziel kannte. Erst `std` verlangt Funktionen, die das Betriebssystem bereitstellen muss. Auch dort ist vieles plattformunabhängig, etwa die Fehlerbehandlung über `Option` und `Result`, die generischen I/O-Traits und die Formatierung über `format!` [4]. Zu portieren ist lediglich die schmale Schicht, die diese Funktionen an das jeweilige System weiterreicht. Sie wird als Portierungsschicht bezeichnet, kurz PAL für Platform Abstraction Layer. Ihren Aufbau und die Schnittstellen, die D3OS ihr anbietet, beschreibt Abschnitt 3.2.

2.2 Kopplung von Compiler und Betriebssystem

Ein naheliegender Ansatz besteht darin, den Aufwand eines eigenen Compiler-Builds zu vermeiden, indem ein Nightly von rust-lang installiert, die Zielbeschreibung als JSON danebengelegt und mit `-Z build-std` gebaut wird, analog zum Bauvorgang des Kernels. Für `core` und `alloc` trägt dieser Ansatz. Für `std` versagt er, weil die JSON-Datei ein Ziel beschreibt, ohne eine Implementierung zu liefern. Der plattformabhängige Teil aus dem vorigen Abschnitt muss in derjenigen Standardbibliothek enthalten sein, die der Compiler mitbringt. Ein Nightly von rust-lang bringt für D3OS keine solche mit. Der Compiler muss folglich selbst gebaut werden. Die übernommene Toolchain `dev-x86_64-unknown-d3os` ist daher ein lokal gebauter Fork, erkennbar am Versionssuffix `-dev`, und die Arbeit verteilt sich auf zwei Repositories, den Rust-Fork und D3OS selbst.
Eine zweite Abgrenzung betrifft `rustup`. `rustup` ist kein Compiler, sondern ein Multiplexer. Es verwaltet mehrere Toolchains nebeneinander und entscheidet lediglich, welche beim Aufruf zum Einsatz kommt. Welche das ist, legt eine feste Reihenfolge von Regeln fest [6]. Im D3OS-Verzeichnis greift die `rust-toolchain.toml`, die dort mit `channel = "dev-x86_64-unknown-d3os"` und `components = ["rust-src"]` gesetzt ist. Der zweite Eintrag ist nicht optional, denn `-Z build-std` benötigt die Quellen der Standardbibliothek und findet sie über diese Komponente [5]. Eine Toolchain wiederum ist ein Verzeichnis mit `bin/` und `lib/rustlib/`. In `bin/` befinden sich die Programme, in `lib/rustlib/<target>/lib/` die vorgebauten `.rlib`-Dateien von `core`, `alloc` und `std` für genau dieses Ziel.
Damit Code für `x86_64-unknown-d3os` entsteht, müssen drei Komponenten zusammenpassen, die an drei verschiedenen Stellen liegen. Erstens die Target-Spezifikation in `compiler/rustc_target/src/spec/targets/x86_64_unknown_d3os.rs` und `spec/base/d3os.rs`. Sie setzt `os: Os::Other("d3os".into())`, wählt `rust-lld` als Linker, schaltet mit `has_thread_local: false` threadlokale Daten ab und benutzt `x86_64-unknown-linux-gnu` als LLVM-Ziel. Beide Dateien nennen in ihrer ersten Zeile die OSDev-Anleitung, an der sie sich orientieren [7]. Wie die Tier-3-Targets des Upstreams [8] ist die Spezifikation fest in `rustc` einkompiliert und lässt sich, anders als eine JSON-Zielbeschreibung, nicht nachrüsten. Anders als jene liegt sie allerdings nur im hier verwendeten Fork und nicht im offiziellen Quellbaum. Zweitens die `std`-Implementierung, also die im vorigen Abschnitt beschriebene Portierungsschicht für D3OS. Drittens ein Linker für D3OS-ELF-Dateien. Die-

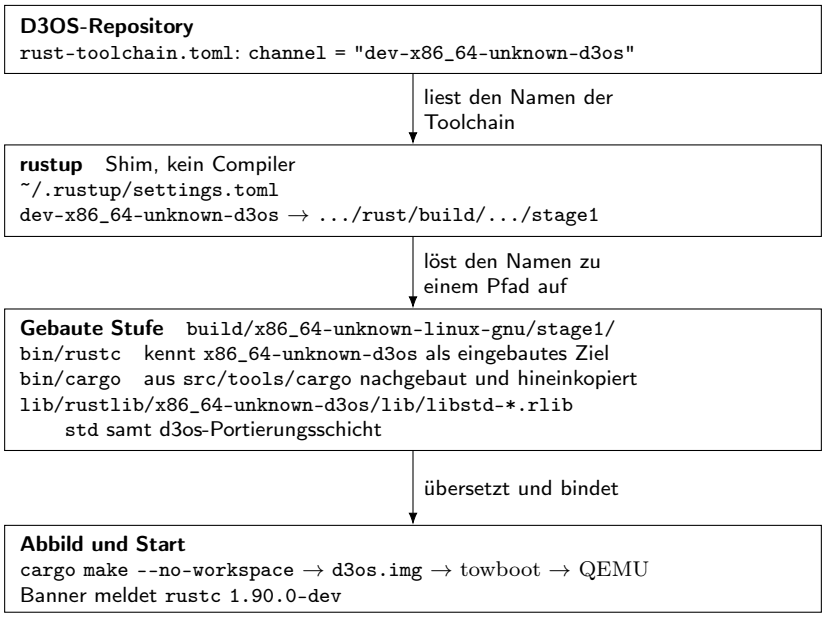


Abbildung 2.2: Die Kopplung vom Eintrag in `rust-toolchain.toml` im D3OS-Repository bis zum bootenden Abbild. Gezeigt ist der Stand des übernommenen Ports mit `rustc 1.90`.

se Rolle übernimmt das mit der Toolchain ausgelieferte `rust-lld`, das die oben genannte Target-Spezifikation auswählt. Daraus erklärt sich auch, warum zwei Sorten von Zielbeschreibungen existieren. Die beiden inhaltlich gleichen JSON-Dateien `d3os_kernel.json` und `d3os_application.json` gelten für alles, was ohne `std` gebaut wird, das eingebaute Target für alles Übrige. Der Bau dieses Compilers setzt seinerseits einen Compiler voraus. Rust löst diese zirkuläre Abhängigkeit über Stufen [9]. Stage 0 ist ein heruntergeladener Beta-Compiler von rust-lang. Mit ihm wird der neue Compiler übersetzt, woraus Stage 1 hervorgeht. Ein weiterer Durchlauf ergäbe Stage 2, was nur für offizielle Releases erforderlich ist. `./x build` baut deshalb standardmäßig bis Stage 1 und übersetzt anschließend mit dem neu gebauten `rustc` die Bibliotheken für jedes Ziel, das in der `bootstrap.toml` eingetragen ist. Im vorliegenden Fall sind dies zwei Ziele [3], das Host-Ziel und `x86_64-unknown-d3os`. Ein vollständiger Durchlauf beansprucht beim ersten Mal ein bis zwei Stunden.
Nicht mitgebaut wird dabei Cargo, da das Build-Profil `compiler` es ausschließt [3]. Sichtbar wird diese Lücke erst beim Aufruf, denn `rustup toolchain link` legt weder eine Kopie noch einen Symlink an, sondern lediglich einen Pfadeintrag in `~/.rustup/settings.toml`, ohne zu prüfen, ob unter diesem Pfad ein Artefakt vorliegt. Die Fehlermeldung lautet dann:

```
error: toolchain 'dev-x86_64-unknown-d3os' does not contain component 'cargo'
```

Das Verlinken des Cargo einer Stable-Toolchain scheidet als Lösung aus, obwohl Cargo lediglich `rustc` aufruft und weitgehend zielunabhängig ist. Kernel und Anwendungen werden mit `-Z build-std` übersetzt, und `-Z`-Flags nimmt ausschließlich ein Nightly-Cargo an [5]. Cargo wird deshalb mit `./x build cargo` aus demselben Fork gebaut und anschließend aus `stage1-tools-bin/` nach `stage1/bin/` kopiert, da der Pfadeintrag genau

dorthin verweist. Eine Suche nach `cargo` im Build-Verzeichnis liefert fünf Treffer, von denen nur einer zum neuen Compiler gehört. Zwei stammen aus Stage 0 und würden den Versionswechsel unterlaufen, ohne dass eine Fehlermeldung erfolgt. Dass der übernommene Port ohne ein selbst gebautes Cargo auskommt, hat einen eigenen Grund, den Abschnitt 2.3 erläutert.

Zwei Eigenschaften der in Abbildung 2.2 dargestellten Kette prägen den weiteren Verlauf der Arbeit. Erstens erfolgt die Kopplung über den Namen der Toolchain und nicht über eine Version. Bei jedem Compiler-Wechsel wird dasselbe Verzeichnis neu gebaut und unter demselben Namen erneut verlinkt. Die `rust-toolchain.toml` in D3OS bleibt dabei unverändert. Zweitens ist die Kette lang und nicht abkürzbar. Jede Änderung an der Standardbibliothek verlangt einen vollständigen `./x build`, bevor D3OS neu gebaut werden kann. Baranowski vermerkt zusätzlich, dass anschließend ein `cargo make clean` erforderlich ist, da der Build die neue `std` andernfalls nicht bis in die Anwendung durchreicht und unbemerkt der vorherige Stand geprüft wird [3]. Aus diesem Grund geht in den folgenden Kapiteln an mehreren Stellen eine Prüfung dem Bauvorgang voraus.

2.3 Der übernommene Port und verwandte Ports

Der übernommene Port ist funktionsfähig. Baranowskis Stand auf `rustc 1.90` baut, bootet und gibt Text aus. `stdlibtest` schreibt unmittelbar über `std::io` auf die Konsole, und `print!` sowie `println!` sind lauffähig [3]. Implementiert ist dieser Pfad in `sys/stdio/d3os.rs` und endet im Syscall `TerminalWriteOutput`. Voraussetzung dafür sind threadlokale Daten, die Baranowski eigens angebunden hat. Ihre Verwaltung im Thread Environment beschreibt Abschnitt 3.2. Damit war die Grundlage für die weiteren Arbeiten gegeben.

Dem steht eine Reihe von Schnittstellen gegenüber, die lediglich formal vorhanden sind. Die Portierungsschicht des Ports ist eine Kopie der `unsupported`-Variante mit drei eigenen Dateien. Alle übrigen Bestandteile tragen die Signatur der jeweiligen Schnittstelle und im Rumpf einen Fehler oder einen Abbruch. Am deutlichsten zeigt sich dies bei der Standardeingabe. `Stdin::read` gibt bedingungslos `Ok(0)` zurück, wobei `Ok(0)` für die Standardbibliothek nicht „nichts gelesen“, sondern „Ende der Daten“ bedeutet [10]. Jedes Rust-Programm auf D3OS interpretierte die Eingabe folglich als einen unmittelbar erschöpften Datenstrom. `time.rs` bricht die Ausführung ab, sobald die Uhrzeit abgefragt wird. Ein Dateisystem-Backend existierte nicht. `sys/fs/mod.rs` fiel für D3OS in den `_`-Zweig und jede Dateioperation lieferte einen Fehler.

Der Allokator veranschaulicht diesen Befund am deutlichsten und bildet zugleich den Ausgangspunkt für Abschnitt 5.1. Die Datei `sys/pal/d3os/alloc.rs` enthielt einen Platzhalter, der laut Quellkommentar aus dem OSDev-Wiki stammt [7] und die drei Etappen unverändert überstanden hat. Er erfüllt die Signatur von `GlobalAlloc` vollständig, nicht jedoch deren Semantik. Der folgende Auszug gibt ihn ohne die `use`-Deklarationen wieder:

sys/pal/d3os/alloc.rs Rust

```rust
#[repr(align(4096))]
struct HeapData([u8; 65536]);

static mut HEAP_DATA: HeapData = HeapData([0; 65536]);
static HEAP_USED: AtomicUsize = AtomicUsize::new(0);

#[stable(feature = "alloc_system_type", since = "1.28.0")]
unsafe impl GlobalAlloc for System {
        #[inline]
        unsafe fn alloc(&self, layout: Layout) -> *mut u8 {
                if layout.align() > 8 {
                        return ptr::null_mut();
                }
                let num_blocks = if layout.size() % 8 == 0 {
                        layout.size() / 8
                } else {
                        (layout.size() / 8) + 1
                };
                HEAP_USED.fetch_add(num_blocks, Ordering::Relaxed);
                let ptr = unsafe {
                        ptr::addr_of_mut!(HEAP_DATA.0[
                        HEAP_USED.load(Ordering::Relaxed) - num_blocks
                        ]) as *mut u8
                };
                ptr
        }

        #[inline]
        unsafe fn dealloc(&self, _ptr: *mut u8, _layout: Layout) {}
}
```

Vier Defekte sind darin unmittelbar erkennbar. Der Heap ist ein statischer Puffer von 64 KiB. Wird dieser überschritten, liefert der Allokator Zeiger hinter dessen Ende. `dealloc` ist eine Leeroperation und gibt niemals Speicher zurück. Jede Anforderung mit `layout.align() > 8` wird mit `null_mut()` abgelehnt, was in `std` unmittelbar zum Abbruch führt. Schließlich wird der Zähler mit `fetch_add` erhöht und anschließend separat gelesen. Zwischen beiden Schritten kann ein anderer Thread den Zähler verändern. Für ein Programm, das lediglich eine Zeile ausgibt, bleiben diese Defekte folgenlos.

Offen geblieben ist der in Abschnitt 2.2 angekündigte Grund dafür, dass der übernommene Port ohne ein selbst gebautes Cargo auskommt. Baranowski hat Cargo nicht aus dem Fork gebaut. Findet `rustup` in einer verlinkten Toolchain kein eigenes Cargo, greift es auf dasjenige einer Release-Toolchain zurück und bevorzugt dabei `nightly` vor `beta` und `stable` [6]. Da auf seinem System keine Toolchain den Namen `nightly` trug, sondern `nightly-2025-10-20`, verwies der Rückfall auf ein Cargo, das die `-Z`-Flags des D3OS-Builds nicht annimmt. Sein Build brach entsprechend ab. Behoben hat er dies mit `rustup toolchain add nightly` und diesen Schritt als Vorbereitung in seinem Anhang A.1 dokumentiert [3]. Sein Cargo stammte damit aus einer anderen Quelle als sein `rustc`. Der in dieser Arbeit gewählte Weg vermeidet diese Aufspaltung, indem beide Werkzeuge aus demselben Fork gebaut werden.

Für die eigentliche Umsetzung ist Baranowskis Port von geringerer Bedeutung als die Ports anderer Plattformen. Upstream pflegt in `library/std/src/sys/` für jedes Sub-

system ein Dutzend fremder Backends. Jedes davon ist eine mögliche Vorlage, also ein bereits portiertes System, dessen Datei als Muster dienen kann. Die Auswahl richtet sich nicht nach der Ähnlichkeit des jeweiligen Betriebssystems zu D3OS, sondern danach, welches Backend dasselbe Ressourcenmodell aufweist und dessen Unterbau auf vergleichbarer Ebene ansetzt. Tabelle 2.1 führt die je Modul gewählten Backends auf.

Modul	Vorlage	Ausschlaggebend
`alloc`	`xous`	Mikrokernel, Speicher per Syscall
`fs`	`solid`	Deskriptor als Zahl, „negativ ist ein Fehler“
`thread`	`hermit`	dünnster Wrapper über ein Runtime-Crate
`stdin`	keine	Backend vorhanden, nur der Lesepfad fehlte

Tabelle 2.1: Gewählte Vorlagen je Modul und das jeweils ausschlaggebende Kriterium.

Aufschlussreich sind auch die verworfenen Ansätze. `motor` reicht alle Dateioperationen an eine fertige Dateisystem-Bibliothek des Zielsystems weiter. Das D3OS-Gegenstück dazu wäre `naming`, von dem die Standardbibliothek gerade nicht abhängen darf. `vexos` arbeitet über `C-FILE*` statt über rohe Deskriptoren und damit über das falsche Ressourcenmodell. Eine Einzelheit zur Einbindung der nicht implementierten Funktionen wurde dennoch von dort übernommen. Abschnitt 3.1.2 beschreibt sie. `unix` scheidet aufgrund seines Umfangs aus. `wasi` existiert nicht mehr als eigenständiges Backend, da WASI inzwischen in den `unix`-Zweig fällt. Die Unterschiede im Umfang sind an den Dateigrößen im Verzeichnis `sys/thread/` ablesbar. `hermit` misst dort 2479 Byte, `solid` 14582 Byte und `unix` 33232 Byte. Die Wahl einer ungeeigneten Vorlage erhöht daher nicht nur den Schreibaufwand, sondern führt zur Übernahme von Annahmen, die nachträglich erkannt und entfernt werden müssen.

Kapitel 3

Konzept

3.1 Vorgehen

3.1.1 Anker-Sprungkette und Rebase statt Merge

Der Sprung von `rustc 1.90` auf `1.94` ist in drei Etappen unterteilt, von denen jede einen festen Zielpunkt besitzt, der im Folgenden als Anker bezeichnet wird. Die Anker sind nicht selbst gewählt, sondern von der Betreuung vorgegeben. Maßgeblich dafür war die Abfolge, in der D3OS seine eigene Toolchain im Hauptzweig nachgezogen hat. Der Port folgt ihr.
Ein Anker ist zunächst ein Datum, Git adressiert jedoch Commits und keine Daten. Diese Übersetzung muss exakt sein, da ein falscher Anker die gesamte Kette verschiebt. Sie erfolgt nicht durch Schätzung, sondern über die Release-Manifeste des Rust-Projekts. Für jeden Tag wird ein eigenes Manifest angelegt, das die Pakete des betreffenden Nightly beschreibt. Ablageort und Aufbau sind in der Rust Forge dokumentiert [11]. Der Pfad ergibt sich unmittelbar aus dem Datum. Für `nightly-2025-08-18` lautet er `https://static.rust-lang.org/dist/2025-08-18/channel-rust-nightly.toml`. Dort steht der Eintrag `git_commit_hash` mit dem Wert `425a9c0a0e365c0b8c6cfd00c2ded83a73bed9a0`.
Der Commit liegt dabei einen Tag vor dem Manifest, da das Nightly vom 18. August aus dem Stand vom 17. August gebaut ist. Tabelle 3.1 nennt die vier Anker, die sich daraus ergeben.

Nightly	**Upstream-Commit**	**Version**	**Verwendung**
2025-06-25	`28f1c807`	1.90.0-nightly	Ausgangsstand
2025-08-18	`425a9c0a`	1.91.0-nightly	Etappe A
2025-10-20	`f04e3dfc`	1.92.0-nightly	Etappe B
2026-01-01	`8d670b93`	1.94.0-nightly	Etappe C

Tabelle 3.1: Die vier Anker der Sprungkette, bestimmt aus den Release-Manifesten

Für jede Etappe wird ein eigener Zweig angelegt und veröffentlicht. Das inkrementelle Vorgehen erfüllt damit drei Zwecke, von denen dieser der unmittelbar praktische ist. Bricht eine Etappe unrettbar zusammen, lässt sich auf den Zweig der Vorgängeretappe zurückspringen und damit auf einen Stand, der nachweislich baut und bootet. Die beiden übrigen Zwecke betreffen die Auswertung.

Ein Rebase spielt die eigenen Commits einzeln gegen den neuen Upstream-Stand ab, sodass jede Konfliktauflösung genau einer Upstream-Änderung zuordenbar bleibt. Ein Merge fasst denselben Vorgang zu einem Knoten zusammen. Danach ist nicht mehr feststellbar, welcher Eingriff zu welcher Compiler-Generation gehörte. Genau diese Zuordnung ist Voraussetzung für die Frage nach dem Anpassungsaufwand je Generation, mit der die erste Leitfrage in Kapitel 4 messbar gemacht wird. Abschnitt 4.5 beantwortet sie. Zusätzlich bleibt der Stand als Patch-Serie exportierbar. Der Preis dafür besteht darin, dass bei jedem Rebase jeder Commit neu geschrieben wird. Die Hashes ändern sich bei jeder Etappe, das Autorenfeld bleibt jedoch erhalten.
Damit die Kette lückenlos bleibt, wird vor jedem Rebase eine einzige Kennzahl geprüft. Der gemeinsame Vorfahre zwischen dem neuen Zielcommit und dem Kopf der Vorgängeretappe muss exakt deren Endbasis ergeben. Vor dem Rebase auf Etappe C, noch auf dem Branch `upgrade-2025-10-20`, stellt sich das wie folgt dar:

```
git merge-base 8d670b93d40737e1b320fd892c6f169ffa35e49e HEAD
f04e3dfc87d7e2b6ad53e7a52253812cd62eba50
```

Das ist die Basis, auf der Etappe B geendet hat. Ermittelt wird dieser Wert, indem der Anker mit `git fetch --shallow-since` und einem Datum kurz vor dem Ausgangsstand geholt wird, wie in Abschnitt 4.2 beschrieben. Diese Angabe bringt die Vorfahren bis zu jenem Datum mit. Erst dadurch verfügen Anker und Etappenkopf über einen gemeinsamen Abschnitt der Historie, in dem Git einen gemeinsamen Vorfahren bestimmen kann.
Aus dem Rebase folgt außerdem, dass die beiden Repositories verschiedene Topologien aufweisen. `ba-rust` ist rebase-basiert, die Etappen-Zweige laufen auseinander und der Baseline-Zweig `d3os` ist kein Vorfahre des Endprodukts. Derselbe Eingriff existiert dort zweimal, als `4909bb3e` auf `upgrade-2025-10-20` und als `f8ca6e7c` auf `upgrade-2026-01-01`, mit gleicher Autorenzeit, aber verschiedenem Hash. `ba-d3os-std` ist dagegen linear aufgebaut. `stdlib` ist direkter Vorfahre jedes Etappen-Kopfs und die Hashes bleiben stabil. Daraus folgt eine praktische Konsequenz für die Nachvollziehbarkeit. Alle `ba-rust`-Hashes, die in Kapitel 4 aufgeführt sind, beziehen sich auf `upgrade-2026-01-01` (in GitLab als `master`). Im Zweig der jeweiligen Etappe sind sie nicht auffindbar.

3.1.2 Von der Vorlage zum Backend

Dieser Abschnitt beantwortet die zweite Leitfrage in methodischer Hinsicht, nämlich in welchem Umfang sich bestehende Plattform-Backends als Vorlage übernehmen lassen und an welchen Stellen eine eigenständige Implementierung erforderlich ist.
Die Oberfläche eines neuen Backends ist nicht frei wählbar. Die `unsupported.rs` des jeweiligen Subsystems legt fest, welche Typen und Funktionen vorhanden sein müssen. Welche Vorlage je Modul gewählt wurde und welches Kriterium dabei den Ausschlag gab, weist Tabelle 2.1 aus. Tabelle 3.2 führt am Beispiel des Dateibackends auf, welche Frage aus welcher Quelle beantwortet wird, und gibt zugleich die Reihenfolge der Bearbeitung wieder.
Anschließend wird die Vorlage Methode für Methode durchgegangen, wobei jeweils dieselbe Frage zu beantworten ist. Steht die betreffende Fähigkeit in D3OS überhaupt zur Verfügung? Ist das nicht der Fall, verbleibt die Methode bei `unsupported()`. Im Dateibackend betrifft dies die Funktionen `unlink`, `rename`, `rmdir`, `stat` und `symlink`, da der Na-

Welche Typen und Funktionen sind Pflicht?	`unsupported.rs`
Woher stammen Grundgerüst, Signaturen und Stubs?	`solid.rs`
Welcher Syscall, welche Argumentreihenfolge?	`naming/lib.rs`
Die Hauptvorlage passt in der Schreibweise nicht, welche Alternative ist geeigneter?	`motor.rs`
Wie wird verfahren, wenn D3OS grundlegend abweicht?	eigener Code
Wie lässt sich die Wiederholung der Stub-Definitionen vermeiden?	`vexos.rs`

Tabelle 3.2: Zuordnung der Entwurfsfragen zu ihren Quellen am Beispiel des Dateibackends

ming Service keinen passenden Aufruf bereitstellt. Bei den Threads sind es `set_name` und `available_parallelism`, da D3OS weder Thread-Namen noch einen eigenen Syscall dafür kennt. Steht die Fähigkeit zur Verfügung, wird der Aufruf der Vorlage durch den D3OS-Syscall ersetzt und die Form der Methode unverändert belassen. Aus der Fremdaufruf-Zeile der Vorlage wird damit ein `syscall(SystemCall::..., &[...])`, dessen Argumentreihenfolge `naming/lib.rs` zu entnehmen ist. Um den Diff klein zu halten, wird der nicht implementierte Teil per `#[path]`-Attribut aus `unsupported.rs` eingebunden, anstatt ihn zu duplizieren. Dieses Verfahren stammt aus `vexos.rs` und ist in Abschnitt 2.3 angekündigt.
In einzelnen Fällen ist eine Vorlage inhaltlich passend, folgt jedoch einer ungeeigneten Schreibweise. `solid.rs` führt POSIX-Eigenheiten fort, etwa die Verwendung von `-1` als Fehlerwert oder das nachträgliche Abfragen der Dateiposition. In solchen Fällen wird ein schlankeres Geschwister-Backend herangezogen, anstatt eine eigene Lösung zu entwerfen. `motor.rs` scheidet als Bauplan aus, liefert jedoch drei verwendbare Einzelheiten, nämlich den Pfad über `path.to_str()` anstelle einer eigenen Zeichenkettenform, die Position über `seek(Current(0))` und die Standardhilfe `read_buf` für gepufferte Zugriffe. Die Übernahme dreier Zeilen aus einer dritten Vorlage bedeutet gegenüber einer Eigenentwicklung den geringeren Aufwand.
Der verbleibende Teil ist der Umbau, dessen Aufwand regelmäßig unterschätzt wird. Die Portierungsschicht erhält von oben eine andere Darstellung, als der Syscall unten erwartet, und genau diese Differenz hat sie auszugleichen. Bei den Threads reicht `std` einen `Box<ThreadInit>` herunter, während `ThreadCreate` eine Einsprungadresse und ein einzelnes Datenwort erwartet. Die Lösung besteht nicht in einem Nachbau, sondern in einer Übersetzung. Der Zeiger wird in eine Zahl überführt, und als Einsprungadresse dient ein eigenes Trampolin, das den Zeiger im neuen Thread wiederherstellt. Damit ist allerdings noch nicht entschieden, an welcher Stelle eine erforderliche Korrektur vorzunehmen ist.

3.1.3 Einordnung und Behandlung von Abweichungen

Die Frage, an welcher Stelle eine Korrektur vorzunehmen ist, lässt sich an der Standardeingabe erläutern. Der Terminal-Emulator lieferte dort jede Zeile ohne das abschließende `\n`. Ein Lösungsansatz, der das Zeichen in der Portierungsschicht wieder anhängt, wurde verworfen. Maßgeblich ist, dass der Emulator die einzige Stelle im System ist, die den Abschluss einer Zeile kennt. Dort stellt das Anhängen keine Erzeugung von Daten dar. Eine

Korrektur in der Portierungsschicht hätte demgegenüber jeden weiteren Leser gezwungen, dieselbe Abweichung erneut zu behandeln. Abschnitt 5.2 stellt den Fall im Einzelnen dar. Aus ihm und den drei übrigen Modulen ergibt sich die Einteilung in Tabelle 3.3. Sie ist kein allgemeines Ordnungsschema für beliebige Abweichungen, sondern ordnet die in dieser Arbeit konkret aufgetretenen Fälle.

Klasse	**Behandlung**	**Fälle, zum Beispiel**
Defekt	an der Quelle beheben	Die Öffnungs-Flags ohne Bitmaskensemantik. Das fehlende `\n`.
Andere Repräsentation	die Portierungsschicht überbrückt ohne Eingriff an der Quelle	Der Dateideskriptor als `usize`. Eine Closure mit Zustand anstelle eines einfachen Funktionszeigers.
Fehlende Fähigkeit	die Portierungsschicht meldet einen Fehler, statt die Fähigkeit vorzutäuschen	Negativer Seek. `truncate` und `append`.
Bewusste Design-Wahl	Rücksprache halten, nicht eigenmächtig ändern	`create` immer exklusiv, festgehalten im Kommentar „avoid creating a file twice“.

Tabelle 3.3: Vier Klassen von Abweichungen und der jeweilige Ort ihrer Behandlung

Eine Entscheidung erfordert dabei nur die erste Zeile. Ob eine Abweichung als Defekt gilt, bemisst sich nicht daran, ob sie die Portierung erschwert, sondern daran, ob die Quelle ihre eigenen Zusagen verfehlt. Die Öffnungs-Flags des Naming Service sind über `bitflags!` deklariert, und dieses Makro setzt für jede Konstante ein eigenes Bit voraus. Von eins bis sechs durchnummerierte Werte erfüllen diese Voraussetzung nicht, da `contains` dann Treffer für Bits meldet, die nie gesetzt wurden. Dies ist ein Defekt, unabhängig davon, ob er zuvor bemerkt wurde, und wurde daher an der Quelle behoben.
Schwieriger liegt der Fall, wenn die Absicht dokumentiert ist. Der Kommentar „avoid creating a file twice“ belegt, dass das ausschließliche Anlegen beabsichtigt war. Damit handelte es sich zunächst nicht um einen Defekt, sondern um eine Designentscheidung, die zur Entscheidung vorgelegt und nicht eigenmächtig geändert wurde. Erst nachdem sich gezeigt hatte, dass die parallele Weiterentwicklung des Naming Service dieselbe Stelle bereits in dieselbe Richtung korrigiert hatte, wurde daraus ein Eingriff. Ein Kommentar, der eine Absicht festhält, verschiebt eine Abweichung somit von der ersten in die vierte Zeile der Tabelle. Erst ein externer Beleg führt sie zurück. Liegt weder das eine noch das andere vor, verbleibt die Abweichung an ihrem Ort und wird von der Portierungsschicht getragen. Wie diese Schicht dafür aufgebaut ist, beschreibt Abschnitt 3.2.

3.2 Architektur

Abschnitt 3.1 beschreibt das Vorgehen sowie die Kriterien dafür, an welcher Stelle eine Korrektur vorzunehmen ist. Der vorliegende Abschnitt zeigt, woran diese Eingriffe ansetzen. Dargestellt werden zunächst die schmale Schicht aus Abschnitt 2.1, anschließend die Schnittstellen, die D3OS ihr anbietet, und abschließend die Veränderungen, die aus dieser Arbeit hervorgegangen sind.
Diese Schicht liegt im Verzeichnis `library/std/src/sys/` und ist auf zwei verschiedene Arten organisiert. Ein Ort ist die Portierungsschicht unter `library/std/src/sys/pal/d3os/`.

Es gibt ein Verzeichnis pro Plattform, in dem alles zusammenliegt, was diese Plattform betrifft. Für D3OS sind dies die Dateien `os.rs`, `time.rs` und `common.rs` sowie die beiden eigenen Ergänzungen `start.rs` und `alloc.rs`. Der andere Ort sind die thematischen Unterverzeichnisse daneben. Sie sind umgekehrt aufgebaut, also ein Verzeichnis pro Subsystem, in dem alle Plattformen nebeneinander liegen. D3OS steht dort jeweils als einzelne Datei `d3os.rs`, und zwar in den Verzeichnissen `sys/fs/`, `sys/stdio/`, `sys/thread/`, `sys/random/` und `sys/thread_local/key/`. Die PAL ist der ältere Bauplan. Upstream zieht seit einiger Zeit Abstraktionen aus ihr in die Subsystem-Verzeichnisse heraus, weshalb sie kleiner wird. Für Plattformen, die eine bestimmte Fähigkeit nicht unterstützen, hält Upstream unter beiden Bauplänen eine `unsupported`-Variante bereit, deren Funktionen einen Fehler zurückgeben oder die Ausführung abbrechen. Die PAL des übernommenen Ports ist genau daraus entstanden [3]. Die vier Module, um die der Port erweitert wurde, verteilen sich nach demselben Schema. `fs` und `thread` sind als neue `d3os.rs` in den Subsystem-Verzeichnissen abgelegt. Die Standardeingabe erweitert die vorhandene `sys/stdio/d3os.rs` und der Allokator verbleibt in der PAL.

Ein Makro in der `mod.rs` des jeweiligen Verzeichnisses entscheidet zur Übersetzungszeit, welche dieser Dateien für ein Ziel gilt. Es arbeitet eine Liste von Bedingungen ab und bindet den ersten passenden Zweig ein [12]. Für D3OS steht dort `target_os = "d3os" => { mod d3os; ... }`. Gibt es diesen Zweig, wird `d3os.rs` mitübersetzt, und `std` benutzt die Implementierung darin. Gibt es diesen Zweig nicht, greift der `_`-Zweig am Ende, der auf `unsupported` verweist. Die Funktionen lassen sich weiterhin aufrufen, liefern aber nur eine Fehlermeldung. An dieser einen Zeile entscheidet sich somit, ob ein Subsystem für D3OS aktiv oder lediglich vorhanden ist.

Das Makro heißt heute `cfg_select!` und ist in `std` eingebaut. An derselben Stelle stand vorher `cfg_if!` aus einer externen Crate. `cfg_select!` wurde erst mit Rust 1.95 [13] stabil und damit erst nach dem Versionsstand dieser Arbeit. Innerhalb der Standardbibliothek war es jedoch schon vorher in Gebrauch. Der Wechsel fällt in den Zeitraum der Etappen. Seine Auswirkungen auf den Port beschreibt Kapitel 4.

Eine Ausnahme bildet der Verteiler `sys/alloc/mod.rs`, der keinen `_`-Zweig besitzt. Trifft dort keine Bedingung zu, erzeugt das Makro einen `compile_error` und die Übersetzung bricht ab [12]. Der naheliegende Grund dafür ist, dass ein Rückfall auf `unsupported` für einen Allokator nicht geeignet ist. Bei Threads oder Pipes ist ein Stub, der einen Fehler meldet, eine brauchbare Vorgabe, denn das Programm startet und scheitert erst, wenn es die Fähigkeit tatsächlich benötigt. Ohne Allokator kommt ein Programm dagegen gar nicht erst zur Ausführung, weil `std` bereits beim Start Speicher anfordert. Upstream verlangt deshalb von jedem Ziel eine ausdrückliche Angabe. Der Allokator für D3OS liegt jedoch nicht, wie es die Konvention vorsieht, als eigenes Modul in `sys/alloc/`, da der übernommene Port ihn in der Portierungsschicht angelegt hat. Deshalb steht in `sys/alloc/mod.rs` ein leerer Zweig `target_os = "d3os" => {}`. Er wählt kein Modul aus, hält aber ausdrücklich fest, dass hier nichts auszuwählen ist. Woher der Wert `"d3os"` stammt, auf den alle diese Bedingungen prüfen, ist in Abschnitt 2.2 beschrieben. Wie sich die Schichten von oben nach unten zusammensetzen, zeigt Abbildung 3.1.

Jedes der vier Module stellt einen senkrechten Weg von der öffentlichen Schnittstelle bis zu einem Modul des Kerns dar. Die vier Wege berühren sich unterwegs nicht. Genau darauf beruht die Aufteilung der Testanwendung `stdlibtest` in Abschnitt 6.2, denn kein Test benötigt ein Modul, das von einem anderen Test benutzt wird. Grau hinterlegt ist die einzige Schicht, die in dieser Arbeit entstanden ist. Darüber liegt die Standardbibliothek,

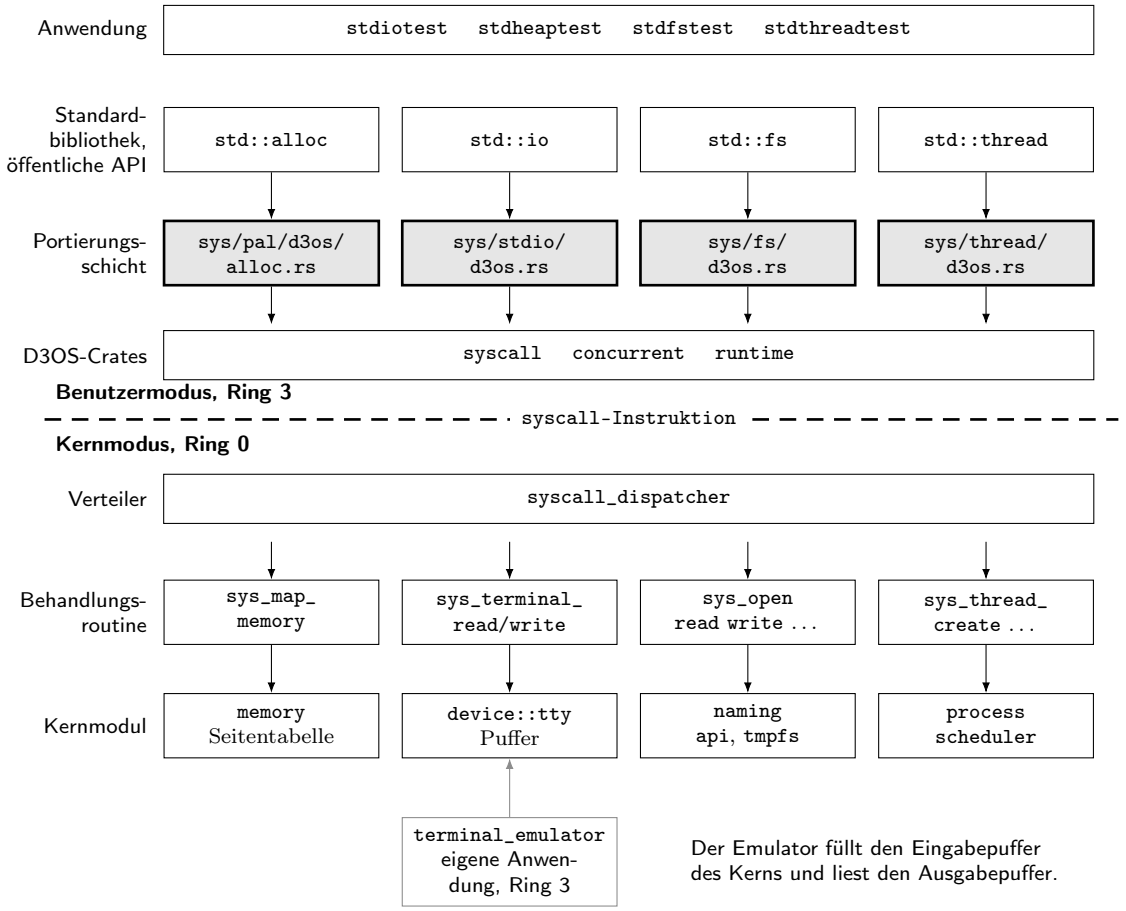


Abbildung 3.1: Gesamtsicht auf die vier Module. Grau hinterlegt und dick umrandet ist die Portierungsschicht, also der Teil, den diese Arbeit geschrieben hat. Darüber und darunter liegt jeweils fremder Code.

die unverändert bleibt. Darunter liegt der Kern, der die Aufrufe bedient. Der Port besteht somit aus vier Dateien, die jeweils eine fremde Schnittstelle auf eine andere fremde Schnittstelle abbilden.
Eine Asymmetrie in der Abbildung erklärt eine Entscheidung aus Abschnitt 3.1.3. Der Terminal-Emulator ist selbst eine Anwendung im Benutzermodus und kein Teil des Kerns. Eine Zeile, die eine Anwendung über `std::io::stdin` liest, stammt also nicht aus dem Kern, sondern hat den Kern nur als Puffer durchlaufen, nachdem der Emulator sie über einen eigenen Systemaufruf dort abgelegt hat. Deshalb ließ sich das fehlende Zeilenende nur im Emulator beheben und nirgendwo darunter. Was die untere Hälfte der Abbildung anbietet, beschreiben die nächsten Absätze.
Alles, was die Standardbibliothek vom Betriebssystem benötigt, läuft über einen einzigen Einstiegspunkt. Die Crate `syscall` aus `os/library/` stellt ihn bereit: `syscall(call: SystemCall, args: &[usize]) -> Result<usize, Errno>`. Die Funktion nimmt die Nummer aus einem Enum mit 47 Einträgen und höchstens sechs Argumente entgegen, legt die Nummer in `rax` und die Argumente in `rdi`, `rsi`, `rdx`, `r10`, `r8` und `r9` ab. Anschließend setzt sie den `syscall`-Befehl ab. Das ist die Kernel-Aufrufkonvention der AMD64-ABI, die sich gerade im vierten Register von der gewöhnlichen Funktionskonvention unterscheidet [14]. Daraus folgt, dass an keiner Stelle eigener Assembler-Code erforderlich ist, obwohl aus der Standardbibliothek heraus in den Kernel gerufen wird. Verwendet werden die folgen-

den Nummern: `MapMemory` für den Heap, `TerminalReadInput` und `TerminalWriteOutput` für die Konsole, `Open`, `Read`, `Write`, `Seek` und `Close` für Dateien sowie `ThreadCreate`, `ThreadJoin`, `ThreadSwitch`, `ThreadSleep` und `ThreadExit` für Threads.

Eine Eigenschaft dieses Rückgabewegs ist für die späteren Kapitel von solcher Bedeutung, dass sie bereits hier festzuhalten ist. Sechzehn Fehlercodes sind dem Kernel bekannt, von `EUNKN` über `ENOENT` und `EEXIST` bis `EAGAIN`. Für jeden davon schreibt der Kernel eine negative Zahl nach `rax`. Im Userspace kommt genau einer davon an. Die Funktion `convert_ret_code_to_syscall_result` in `syscall/src/return_vals.rs` liefert für jeden negativen Wert `Err(Errno::EUNKN)` zurück. Die Zeile, die den tatsächlichen Code zurückübersetzen würde, ist daneben auskommentiert. Der Fehler entsteht also im Kernel, geht aber eine Ebene später verloren.

D3OS erreicht Dateien über seinen Naming Service. Neben den Funktionen `Open`, `Read`, `Write`, `Seek` und `Close` stellt er auch die Funktionen `MkDir`, `Touch`, `Readdir`, `Cwd`, `Cd` und `Mkfifo` bereit. Was `Open` zurückgibt, ist kein Zeiger, sondern ein Index in eine kernelseitige Handle-Tabelle, die in `open_objects.rs` geführt wird. Darunter liegt ein tmpfs, das vollständig im Arbeitsspeicher gehalten und bei jedem Systemstart neu angelegt wird. Dateien überdauern folglich keinen Neustart, was für die Nachweise in Kapitel 6 von Bedeutung ist.

Threads werden ebenfalls vom Kernel verwaltet. Die Crate `concurrent` liefert dafür ein Handle `Thread { id: usize }`, und die Funktionen `join`, `exit`, `switch` und `sleep` sind jeweils ein einzelner Syscall. Auch der Stack stammt vom Kernel. Da die Funktion `ThreadCreate` keine Größenangabe entgegennimmt, erhält jeder Thread einen gleich großen, bedarfsweise eingelagerten Bereich. Damit folgen Dateien und Threads demselben Muster. Beides sind Objekte, die der Kernel verwaltet und die der Userspace nur über eine Zahl anspricht. Für die Portierungsschicht bedeutet dies, dass sie nichts nachzubauen hat, was das Betriebssystem bereits übernimmt.

Threadlokale Daten werden über das Thread Environment verwaltet. Jeder Thread erhält beim Start einen eigenen Block auf dem Heap, der mit der Funktion `init_thread_environment()` angelegt wird. Die Adresse dieses Blocks trägt `wrfsbase` als FS-Basis in das FS-Register ein. Wird der Block später benötigt, lässt sie sich mit `rdfsbase` zurücklesen. Ein Thread findet seine eigenen Daten also über ein Register wieder, ohne eine Kennung mitführen zu müssen. In dem Block liegt unter anderem ein Feld für threadlokale Daten, das Baranowski für die Standardbibliothek ergänzt hat [3]. Der Aufruf steht sowohl im Startpfad nativer D3OS-Threads als auch in `sys/pal/d3os/start.rs`, bevor `main` anläuft.

Damit die Standardbibliothek diese Crates überhaupt benutzen darf, muss in `library/std/Cargo.toml` ein eigener Abschnitt `[target.'cfg(target_os = "d3os")'.dependencies]` stehen. Dieser zieht `runtime`, `concurrent` und `syscall` als Git-Abhängigkeiten vom Branch `stdlib`, jeweils mit dem Feature `rustc-dep-of-std`. Dieses Feature ist die eigentliche Bedingung, und es ist auch der Weg, den die OSDev-Anleitung für eine eigene Laufzeit-Crate vorsieht [7]. Ohne dieses Feature bringt eine Crate ihr eigenes `core` und `alloc` mit, sodass im Build der Standardbibliothek eine zweite Instanz davon läge. Die anderen Ziele verfahren ebenso. `hermit-abi`, `moto-rt` und `vex-sdk` tragen dasselbe Feature. Entscheidend ist allerdings, was in diesem Abschnitt *nicht* steht, nämlich `naming` und `terminal`. Die beiden Bibliotheken, die im D3OS-Userspace genau die Aufgaben erledigen, um die es in Kapitel 5 geht, sind aus der Standardbibliothek heraus nicht erreichbar. Sie konnten lediglich gelesen und die von ihnen abgesetzten Syscalls eigenständig nach-

gebildet werden. Der im Folgenden verwendete Begriff der Referenzbibliothek bezeichnet entsprechend eine D3OS-eigene Bibliothek, die als Muster dient, jedoch nicht als Abhängigkeit eingebunden wird.
Was diese Arbeit an dieser Struktur verändert hat, zeigt Abbildung 3.2. Sie stellt je Komponente eine Zeile dar. Links ist der Zustand zu Beginn der Arbeit abgebildet, rechts der Zustand am Ende.

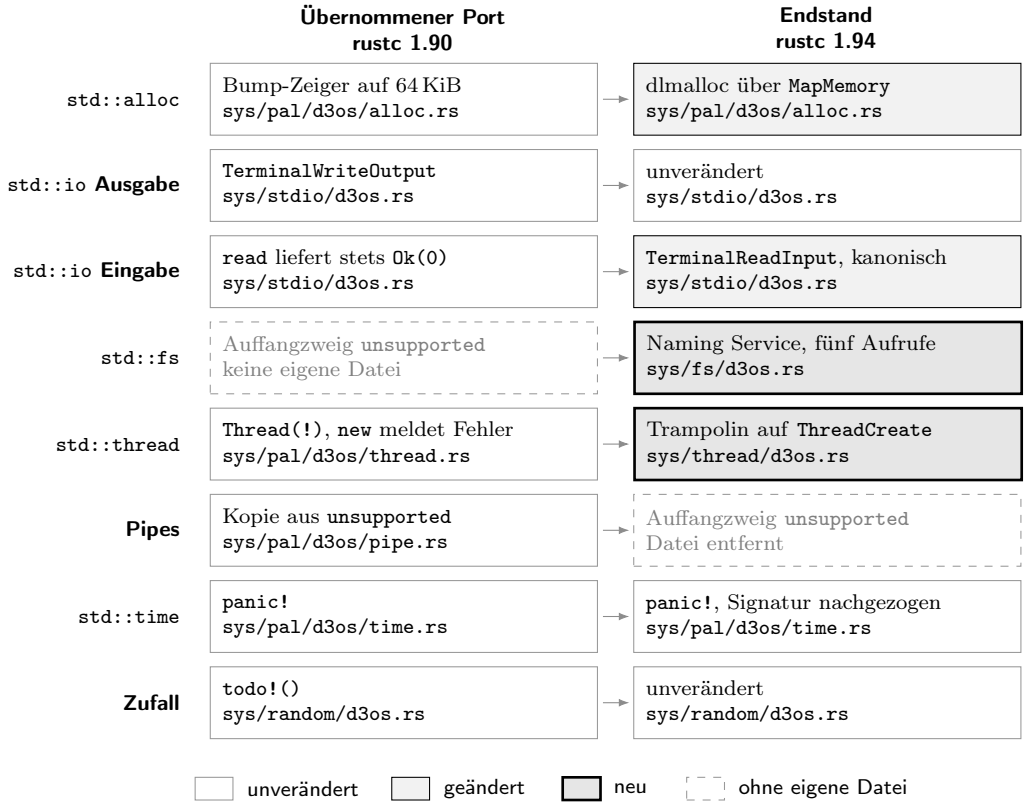


Abbildung 3.2: Die Komponenten der Standardbibliothek für D3OS vor und nach dieser Arbeit.

Zwei Komponenten sind neu, nämlich die Datei-Ein- und -Ausgabe sowie die Threads. Zwei wurden geändert, der Heap und die Standardeingabe. Eine ist ersatzlos entfallen. Drei sind unberührt geblieben, wobei nur die Ausgabe nutzbare Ergebnisse liefert. Zeit und Zufall brechen weiterhin ab, sobald sie aufgerufen werden. Diese beiden Zeilen bilden somit die Grenze der Arbeit, und Abschnitt 7.2 legt dar, was daraus folgt.
Im Zuge dessen hat sich die Portierungsschicht von acht auf sechs Dateien reduziert, während die Zahl der `d3os`-Dateien in den Subsystem-Verzeichnissen gestiegen ist. Das ist keine Entscheidung des Ports, sondern die oben beschriebene Bewegung von Upstream. Im Port ist sie zweimal aufgetreten, bei `thread` und bei `pipe`, wie die Abschnitte 4.3 und 4.4 im Einzelnen darlegen. Kapitel 4 weist im Anschluss nach, welchen Anpassungsaufwand die Kette aus Abschnitt 3.1.1 über vier Minor-Versionen verursacht hat, und liefert damit die Grundlage für die Beantwortung der ersten Leitfrage.

Kapitel 4

Toolchain-Upgrade: rustc 1.90 → 1.94

4.1 Ausgangszustand und Impact-Analyse

Dieses Kapitel legt die Grundlage für die Beantwortung der ersten Leitfrage, also für die Frage, woran die Aktualisierung einer betriebssystemspezifischen Standardbibliothek auf eine neuere Compiler-Version scheitert. Die Antwort ergibt sich aus drei Etappen, die die Abschnitte 4.2 bis 4.4 einzeln darstellen und die Abschnitt 4.5 miteinander vergleicht. Der vorliegende Abschnitt bestimmt zuvor den Ausgangszustand und schätzt ab, welche Stellen ein Versionswechsel treffen kann.

Zwei Annahmen standen am Anfang dieser Arbeit. Beide waren falsch. Die erste bestand darin, einen Versionswechsel für einen Zeilenwechsel in der `rust-toolchain.toml` zu halten. Abschnitt 2.2 hält jedoch fest, dass die Kopplung über den Namen der Toolchain und nicht über eine Versionsangabe erfolgt, sodass genau diese Datei unverändert bleibt. Der Wechsel verteilt sich stattdessen auf vier Teilschritte, von denen jeder eigenständig scheitern kann. Erstens ist der Zielcommit im Rust-Fork zu bestimmen. Zweitens ist der Patch-Stack darauf zu rebasen und sind die Konflikte aufzulösen. Drittens ist der Fork zu bauen und die Toolchain unter demselben Namen neu zu verlinken. Viertens ist D3OS damit zu übersetzen und in QEMU zu booten. Nur der zweite Teilschritt betrifft den Port selbst. Die übrigen drei betreffen die Infrastruktur und haben in Etappe A mehr Zeit beansprucht als der Rebase.

Abschnitt 2.3 beschreibt den Funktionsumfang des übernommenen Ports. Maßgeblich für den Rebase ist jedoch nicht dieser Umfang, sondern die Frage, an welchen Stellen der Port dieselben Dateien bearbeitet wie Upstream. `git diff 28f1c80..HEAD --stat` über `compiler/`, `library/`, `src/bootstrap/` und `tests/` liefert 25 Dateien mit 696 hinzugefügten Zeilen: vier Dateien für die Targetspezifikation aus Abschnitt 2.2, zwei Zeilen im Bootstrap, acht Dateien Portierungsschicht, sieben Dateien in den Subsystem-Verzeichnissen, drei neue `d3os.rs` und vier `mod.rs` mit ergänztem `cfg`-Zweig, dazu vier weitere Dateien, darunter `library/std/Cargo.toml` und der generierte `library/Cargo.lock`. Dreizehn dieser Dateien legt der Port neu an, zwölf bearbeitet er. Nur diese zwölf können in Konflikt geraten, denn eine Datei, die es bei Upstream nicht gibt, hat keine Gegenseite. Sie bilden damit die erwartete Konfliktfläche auf der Seite des Forks. Die zweite falsche Annahme knüpft unmittelbar daran an, nämlich dass der Aufwand eines Versionswechsels ungefähr dem Umfang dieses Stacks entspricht. Abschnitt 4.5 weist nach, dass diese Schätzung die Quelle des Aufwands verfehlt.

Damit ist die Berührungsfläche im Fork erfasst. Für die D3OS-Seite beantwortet dieselbe

Frage eine Zählung dessen, was ein Compiler-Wechsel dort treffen kann. Es bestehen acht `#![feature(...)]`-Gates in drei Crates, davon sechs allein in `os/kernel/src/lib.rs` in den Zeilen 10 bis 15. Ein solches Gate ist die Voraussetzung dafür, eine instabile Schnittstelle von Rust überhaupt verwenden zu können [15]. Hinzu kommen zwölf Assembly-Stellen in fünf Dateien, davon sieben `asm!` und fünf `naked_asm!` [16], sowie 29 Stellen mit `extern "C"`. Von diesen 29 sind fünf Importblöcke der Form `extern "C" { ... }`, in denen fremde Funktionen deklariert werden [17], und 24 Funktionsdefinitionen der Form `extern "C" fn`, also eigene Funktionen, die nach der C-Aufrufkonvention aufgerufen werden [18]. Die vierte Gruppe bilden die beiden inhaltlich identischen JSON-Zielbeschreibungen aus Abschnitt 2.2. Diese vier Zahlen bilden die Nenner, gegen die in den Etappen abgerechnet wird.
Jede Gruppe wird vorab danach eingestuft, wie wahrscheinlich sie bricht. In die Kategorie „hoch" fallen die acht Feature-Gates, denn Upstream darf eine instabile Schnittstelle jederzeit umbenennen, in ein anderes Gate verschieben oder stabilisieren, und jede dieser drei Bewegungen macht das Gate ungültig oder überflüssig. Derselben Kategorie sind die beiden JSON-Dateien zugeordnet, da die JSON-Zielbeschreibung keine stabile Schnittstelle ist und sich ihr Schema zwischen zwei Nightlies ändern kann. Die Kategorie „mittel" erhalten die Assembly-Stellen und die fünf Importblöcke, da beide an einer Blocksyntax hängen und eine Änderung dieser Syntax jede dieser Stellen zugleich treffen würde. Die 24 Funktionsdefinitionen mit `extern "C"` besitzen keine Blockform und fallen deshalb in die Kategorie „niedrig". Zählung und Einstufung aller Gruppen zusammen bilden die Impact-Analyse. Abschnitt 4.5 rechnet sie ab, die erste Prüfung folgt in der nächsten Etappe.

4.2 Etappe A: 1.90 → 1.91

Jede Etappe beginnt mit derselben Folge, wie in Abschnitt 3.1 beschrieben. Zielcommit ist hier die Nightly vom 18. August:

```
git fetch upstream 425a9c0a0e365c0b8c6cfd00c2ded83a73bed9a0 \
--shallow-since=2025-06-20
git merge-base 425a9c0a0e365c0b8c6cfd00c2ded83a73bed9a0 HEAD
git rebase 425a9c0a0e365c0b8c6cfd00c2ded83a73bed9a0
```

`git merge-base` liefert `28f1c807911c63f08d98e7b468cfcf15a441e34b` und bestätigt damit den Ausgangsstand. Der Rebase hat danach zehn Commits abzuspielen, erreicht jedoch nicht den ersten, da der Checkout abbricht:

```
error: The following untracked working tree files would be overwritten by checkout:
library/stdarch/.cirrus.yml
[... rund 80 weitere Dateien ...]
```

`library/stdarch` steht bei `28f1c80` als `160000 commit` im Tree und ist somit ein Submodul. Bei `425a9c0a` steht dort `040000 tree`, also ein gewöhnliches Verzeichnis. Upstream hat stdarch zwischen Juni und August in den Hauptbaum übernommen. Damit lagen die alten Arbeitsdateien dort, wo der neue Stand eigene getrackte Dateien erwartet. `git`

`submodule deinit -f library/stdarch` räumt den Pfad frei, danach läuft der Rebase durch. Ein Compiler-Wechsel bringt somit nicht nur geänderten Code mit, sondern auch eine geänderte Repository-Struktur.
Der Rebase selbst hinterließ zwei Konflikte. `library/Cargo.lock` wurde per `git checkout --theirs` übernommen, da ein von Hand zusammengeführtes Lockfile mit hoher Wahrscheinlichkeit inkonsistent ist und Cargo es ohnehin neu abgleicht. Der zweite Konflikt lag in `src/bootstrap/src/core/sanity.rs` in der Liste `STAGE0_MISSING_TARGETS`. Auf der Upstream-Seite stand der neue Eintrag `aarch64_be-unknown-none-softfloat`, auf der Seite des übernommenen Ports die beiden `loongarch32`-Targets und `x86_64-unknown-d3os`. Zunächst blieben beide Seiten stehen, gestützt auf die Annahme, dass überzählige Einträge in einer Liste fehlender Targets folgenlos bleiben. Der erste `./x build` widerlegte diese Annahme nach 57 Sekunden:

```
Following targets supported from the stage0 compiler, please remove them from
STAGE0_MISSING_TARGETS list.
loongarch32-unknown-none-softfloat
loongarch32-unknown-none
Build completed unsuccessfully in 0:00:57
```

Die Liste ist keine nachsichtige Whitelist, sondern eine Behauptung über den heruntergeladenen Stage-0-Compiler, und Bootstrap prüft sie in beide Richtungen. Beide Seiten zu behalten hieße zu behaupten, der Stage 0 der 1.91-Generation kenne die `loongarch32`-Targets nicht. Er kennt sie längst, denn jedes Upstream-Target durchläuft denselben Lebenszyklus. Es wird eingeführt, in die Liste aufgenommen und beim nächsten Stufenwechsel vom Stage 0 gekannt, woraufhin es wieder entfernt wird. `loongarch32-unknown-none` ist bei Version 1.91 herausgewachsen und `aarch64_be-unknown-none-softfloat` steht bei Version 1.92 an, während `x86_64-unknown-d3os` nie entfernt wird, da kein veröffentlichter Stage 0 es je enthalten wird. Der Port hält damit eine eingefrorene Kopie einer schrumpfenden Liste. Der Abbruch bezeichnet somit keinen neuen Defekt, sondern die Folge des eigenen Eingriffs, der textuell die sichere und inhaltlich die falsche Wahl war. Beide Einträge sind entfernt, `x86_64-unknown-d3os` bleibt bestehen, und derselbe Commit `10aa9c1e` ändert die `change-id` in `bootstrap.toml` von `142379` auf `145131`. Daraus folgt eine Regel für den gesamten Stack. Zunächst ist die Upstream-Seite vollständig zu übernehmen, danach sind die eigenen Änderungen gezielt einzufügen. Ein Union-Merge scheidet aus. Ob die Vorhersage für `aarch64_be` eintrifft, prüft die nächste Etappe.
Drei Hindernisse der Bauumgebung verzögerten den weiteren Verlauf, ohne den Code zu betreffen. Das erste betrifft den Speicherplatz. Das `deinit` von stdarch war nach dem Rebase mit `git submodule update --init --recursive` auszugleichen, wodurch der gesamte Satz zurückgeholt wurde, darunter `src/llvm-project` mit 2,24 GiB und `src/gcc` mit 2,70 GiB. Mitten im stage0-Download brach `curl` ab:

```
curl: (23) Failure writing output to destination
Warning: Problem (retrying all errors). Will retry in 1 seconds. 3 retries left.
```

`df -h` meldete für die Wurzelpartition eine Größe von 122 GB, davon 116 GB belegt und null GB verfügbar, während Baranowski in seinem Anhang A.1 mindestens 50 GB freien Speicherplatz voraussetzt [3]. Die Befehle `apt clean`, `journalctl --vacuum-time=7d`

und das Löschen des angefangenen `build/`-Verzeichnisses gaben 27 GB frei. Das zweite Hindernis betrifft die vorgebauten LLVM-Artefakte. `download-ci-llvm = true` hätte LLVM fertig geholt und den Durchlauf aus Abschnitt 2.2 verkürzt, doch der Server antwortete für den Zielcommit viermal mit HTTP 404, da Upstream alte CI-Builds löscht. Das dritte Hindernis betrifft den Hauptspeicher. Bootstrap baut LLVM deshalb lokal und wählt 32 parallele Jobs nach der Kernzahl. Bei rund 1,5 GB je Übersetzungseinheit entspricht das etwa 48 GB gegenüber verfügbaren 16 GB. Der Bau brach bei Einheit 2509 von 3795 mit `c++: fatal error: Killed signal terminated program cc1plus` ab. Seitdem steht `jobs = 4` in `bootstrap.toml` (Commit `b9044aac`).

Anschließend brach der Bau der Standardbibliothek für `x86_64-unknown-d3os` mit sechs Fehlern ab, die alle auf zwei Dateien zurückgingen, nämlich `thread.rs` und `time.rs`. Beide lagen zu diesem Zeitpunkt noch in der Portierungsschicht, und die Fehler betrafen deren Schnittstelle zum plattformunabhängigen Teil. Diese Schnittstelle ist std-intern und gehört nicht zur öffentlichen API, sodass Upstream sie ohne Ankündigung ändern darf. Dreimal fehlte dem Stub ein Element, das `std` inzwischen aufruft, dreimal war ein vorhandenes Element als `const fn` verlangt. Da es sich bei beiden Dateien um reine Stubs handelt, wurden sie vollständig aus `sys/pal/unsupported/` übernommen, anstatt die Signaturen von Hand nachzuziehen (Commit `f167f678`).

Die einzige Änderung auf der D3OS-Seite während der gesamten Etappe betraf die beiden JSON-Zielbeschreibungen. Der erste `cargo make`-Lauf brach mit der folgenden Meldung ab:

```
target-c-int-width: invalid type: string "32", expected u16 at line 9 column 30
```

Daraus wurde geschlossen, dass alle numerisch aussehenden Felder nunmehr JSON-Zahlen erwarten, und beide Felder wurden entsprechend umgestellt (Commit `6eb8913`). Der nächste Lauf widerlegte diesen Schluss:

```
target-pointer-width: invalid type: integer 64, expected a string at line 8 column
  30
```

Erst Commit `26f29b9` trennt die beiden Felder. `target-pointer-width` bleibt String, `target-c-int-width` wird Zahl. Beide Commits sind bewusst nicht zusammengefasst, da die verworfene Hypothese Teil des Befunds ist. Baranowski [3] musste in seinem Anhang B beide Felder in die andere Richtung umstellen, von Zahl auf String. Dort galt für beide Felder dieselbe Regel, hier für jedes eine eigene. Welchen Typ ein Feld erwartet, geht ausschließlich aus der Fehlermeldung hervor.

Am Ende der Etappe stehen acht Anpassungsklassen und zwei Merge-Konflikte, und der Patch-Stack ist von zehn auf dreizehn Commits gewachsen. Beide Konflikte lagen in den zwölf bearbeiteten Dateien aus Abschnitt 4.1, der stdarch-Abbruch dagegen in keiner davon. Den erreichten Stand belegt der erfolgreiche Boot in QEMU nach dem in Abschnitt 2.2 genannten `cargo make clean`. Das Banner meldet `rustc 1.91.0-dev` statt `1.90.0-dev`, dahinter laufen wie zuvor drei Prozesse und drei Threads, und `stdlibtest` gibt dieselben vier Zeilen aus. Ob acht Anpassungsklassen einen typischen Wert darstellen oder den Wert eines ersten Versuchs, klärt die nächste Etappe.

4.3 Etappe B: 1.91 → 1.92

Ziel ist die Nightly vom 20. Oktober 2025 mit dem Commit `f04e3dfc`. Der Rebase spielt dieselben dreizehn Commits ab wie in Etappe A, diesmal jedoch mit acht Dateikonflikten in vier Commits statt mit zwei Konflikten in einem Commit. Der erste Konflikt bestätigt die Vorhersage aus Abschnitt 4.2:

src/bootstrap/src/core/sanity.rs — Rust

```
const STAGE0_MISSING_TARGETS: &[&str] = &[
// just a dummy comment so the list doesn't get onelined
<<<<<<< HEAD
"x86_64-unknown-motor",
=======
"aarch64_be-unknown-none-softfloat",
"loongarch32-unknown-none",
"loongarch32-unknown-none-softfloat",
"x86_64-unknown-d3os",
>>>>>>> 24c22116fbe (Initial commit with first implementations)
```

`aarch64_be-unknown-none-softfloat` ist wie angekündigt herausgewachsen. Die beiden `loongarch32`-Einträge stehen erneut da, obwohl sie in Etappe A entfernt wurden. Die untere Seite ist nicht der aktuelle Stand, sondern der eingefrorene Commit `24c22116fbe`, denn die Entfernung erfolgte erst in einem späteren Commit derselben Reihe. Der Konflikt wird diesmal ohne vorherigen Build behoben. Die Upstream-Liste bleibt vollständig, `x86_64-unknown-d3os` wird hinzugefügt und die drei stabilisierten Targets werden entfernt. Die Anpassungsklasse, die in Etappe A erst nach 57 Sekunden Bauzeit auffiel, ist damit in die Rebase-Phase gewandert.
Der zweite Konflikt in demselben Commit verweist auf den in Abschnitt 3.2 beschriebenen Befund. In `library/std/src/sys/pal/mod.rs` steht auf der einen Seite die neue und auf der anderen die alte Grammatik:

library/std/src/sys/pal/mod.rs — Rust

```
        <<<<<<< HEAD
}
_ => {
        =======
} else if #[cfg(target_os = "d3os")] {
        mod d3os;
        pub use self::d3os::*;
} else {
        >>>>>>> 24c22116fbe (Initial commit with first implementations)
```

Der d3os-Arm lässt sich nicht übernehmen, sondern ist zu übersetzen. Aus `} else if #[cfg(target_os = "d3os")] {` werden zwei Zeilen, eine schließende Klammer und darunter `target_os = "d3os" => {`. Dasselbe gilt für `sys/stdio/mod.rs` und `sys/random/mod.rs`. Dort überlagert sich der Syntaxkonflikt mit einer inhaltlichen Änderung, denn Upstream hat im selben Codebereich den Arm für `motor` eingefügt, also genau jenes `x86_64-unknown-motor` aus `sanity.rs`. Zu beachten ist dabei die Re-Export-Konvention, die je Datei verschieden ausfällt: `pub use self::d3os::*;` in der Portierungsschicht, `pub use d3os::*;` in `stdio` und `pub use d3os::fill_bytes;` in `random`.
Die übrigen Konflikte verlangen geringere Eingriffe. In `library/std/Cargo.toml` ent-

steht der Konflikt, weil Upstream die Tabelle `[target.'cfg(target_os = "vexos")'.dependencies]` eingeführt hat, die unmittelbar an den D3OS-Block grenzt. Beide Tabellen bleiben stehen, da sie einander nie widersprechen. Derselbe Konflikt tritt in Commit 8 ein zweites Mal auf, während Commit 10 dieselbe Datei und denselben Block bearbeitet und dennoch durchläuft, weil seine Änderung im Inneren des Blocks liegt. Über den Konflikt entscheidet somit nicht die Datei, sondern die berührte Region.
Der D3OS-Block bleibt dabei unaufgeräumt, obwohl er in Commit 1 noch Baranowskis lokale Absolutpfade nach `/home/david-baranowski/ba-dabar113/` enthält. Genau diese Umstellung auf Git-Abhängigkeiten leistet Commit 10. Ein Vorgriff hätte diesen Commit desynchronisiert. Beim Rebase ist der historische Zustand zu rekonstruieren und nicht der gewünschte Endzustand.
Zwei Konflikte verbleiben. `library/Cargo.lock` wurde wie in Etappe A per `git checkout --theirs` übernommen. Commit 11 aus Etappe A entfernt die stabilisierten `loongarch32`-Targets und hebt die `change-id` an. Sein `sanity.rs`-Anteil ist hinfällig, da dieselbe Korrektur bereits in Commit 1 enthalten ist. Sein Diff schrumpft beim Auflösen auf nichts zusammen, und die darin enthaltene `change-id` ist ebenfalls veraltet. Der neue Wert `147157` stammt aus `src/bootstrap/src/utils/change_tracker.rs` und wurde vorab dort entnommen, anstatt ihn später aus der Build-Warnung abzulesen. Anpassungen sammeln sich somit über die Etappen an, und einzelne werden dabei überflüssig.
Der erste `./x build` nach dem Rebase brach ab:

```
error: There are multiple `compiler_builtins` packages in your project, and the
specification `compiler_builtins` is ambiguous.
Please re-run this command with one of the following specifications:
compiler_builtins@0.1.0
compiler_builtins@0.1.160
```

Die Diagnose verlief in drei Schritten und über zwei verworfene Erklärungen. Die erste Erklärung lautete, das `Cargo.lock` sei veraltet, da es das Ergebnis der letzten erfolgreichen Auflösung festhält [19] und dieses Ergebnis aus dem Rebase stammt. Das Zurückholen des Upstream-Stands per `git checkout f04e3dfc -- library/Cargo.lock` beseitigte den Fehler nicht. Auch `rm library/Cargo.lock`, das Cargo zu einer Auflösung von null zwingt, blieb wirkungslos. Genau dieser zweite Fehlschlag ist der entscheidende Hinweis, denn wenn bereits eine frische Auflösung zwei Kopien erzeugt, ist das Lockfile nicht veraltet, sondern der Dependency-Graph verlangt beide.
Der zweite Schritt bestätigt dies an der Datei selbst. In Zeile 60 ist `compiler_builtins` in Version `0.1.0` mit `source = "registry+..."` aufgeführt, in Zeile 66 dieselbe Crate in Version `0.1.160` ohne `source`. Das fehlende Feld ist das Erkennungszeichen, denn Pakete ohne `source` liegen als Pfad im Baum. Da Cargo ein Paket über Name, Version und Quelle identifiziert [20], handelt es sich um zwei echte Pakete und nicht um eine Doppelbuchung. Als Verursacher weist `Cargo.lock` `runtime`, `concurrent` und `syscall` aus, also die drei D3OS-Crates aus Abschnitt 3.2.
Damit bot sich eine zweite Erklärung an, nämlich dass eine dieser Crates genau Version 0.1.0 fordert. Die Kopie im Baum trägt Version 0.1.160 und hätte eine solche Forderung nicht erfüllen können, sodass eine zweite Kopie unvermeidlich gewesen wäre. Ein Eingriff hätte in diesem Fall zwei Repositories berührt, da die drei Crates nicht im Fork, sondern in `hhu-bsinfo/D3OS` liegen. Ihre Manifeste fordern jedoch Version 0.1, was in Cargo

eine Version zwischen 0.1.0 und 0.2.0 bezeichnet, und die Kopie im Baum erfüllt diese Anforderung. Auch die zweite Erklärung scheidet damit aus.
Erst der dritte Schritt liefert die Ursache, und zwar im Vergleich mit dem Ausgangsstand. Der Block `[patch.crates-io]` in `library/Cargo.toml` endet in 1.92 bei `rustc-std-workspace-std` in Zeile 61. Im Stand von 1.90 stand darunter noch eine Zeile, die genau diese Umleitung setzt:

library/Cargo.toml TOML

```toml
compiler_builtins = { path = "compiler-builtins/compiler-builtins" }
```

Upstream hat sie entfernt. Für den Upstream-Build bleibt das folgenlos, da dort niemand die Umleitung benötigt. Für den Port hingegen ist sie zwingend, denn ohne diese Zeile beziehen die drei D3OS-Crates `compiler_builtins` aus der Registry statt aus dem Baum [21].
Das Wiedereinfügen der Zeile allein blieb wirkungslos, der Bau brach nach 0:00:00 mit derselben Meldung ab. Ein `[patch]`-Eintrag wirkt erst, wenn Cargo den Graphen neu auflöst [21], und das `Cargo.lock` lag noch vor. Das `rm` aus dem ersten Diagnoseschritt war damit nachträglich betrachtet der richtige Eingriff, der lediglich zu früh erfolgte, weil ihm der `[patch]`-Eintrag noch fehlte. Wirksam ist erst die Kombination beider Teile in dieser Reihenfolge, also der Eintrag und ein anschließendes `rm library/Cargo.lock`.
Der nächste Lauf übersetzte 39 Sekunden lang tatsächlich `std` und meldete anschließend 37 Fehler. Die Zahl ist irreführend, denn dahinter stehen zwei Wurzelursachen und eine Kaskade.
In den Zeilen 55 bis 63 von `sys/stdio/mod.rs` standen unaufgelöste Konfliktmarker. Die Auflösung aus Commit 1 wurde nie gespeichert, und `git rebase --continue` prüft dies nicht. Die Marker sind so unbemerkt durch alle dreizehn Commits gereist.
In `sys/thread_local/mod.rs` lag der umgekehrte Fall vor. Git hat Baranowskis alte Zeile 189 und den Umbau von Upstream ohne Konfliktmeldung zusammengeführt, da beide Änderungen textuell nicht überlappen. Der Rumpf des Arms ist in beiden Grammatiken gültig, allein die Übergangszeile blieb alt. Die Korrektur umfasst eine einzige Zeile, die in zwei zerfällt.
Beide Defekte sind Parse-Fehler, und ein Modul mit Parse-Fehler exportiert nichts mehr. Die übrigen 35 Meldungen betrafen Namen aus `stdio` und `thread_local::key`, darunter `Stdin`, `is_ebadf` und `LazyKey`. Die Dateien `rt.rs`, `panicking.rs` und `io/stdio.rs` blieben unverändert, die Meldungen verschwanden mit den zwei Syntaxkorrekturen. Seitdem folgt auf jeden Rebase ein `git grep -nE '^(<<<<<<<|>>>>>>>)'` über den gesamten Baum. Der Scan liefert allerdings systematische Falschtreffer, da die rustc-Testsuite absichtlich Konfliktmarker enthält, beispielsweise unter `tests/ui/parser/diff-markers/`.
Beide Defekte wurden nicht durch den Merge, sondern durch den Compiler aufgedeckt.
Der darauffolgende Lauf brach mit einer Meldung ab, die keinen Syntaxfehler bezeichnet:

```
error: none of the predicates in this cfg_select evaluated to true
```

Die beiden Makros unterscheiden sich hier nicht in der Schreibweise, sondern im Verhalten. `cfg_if!` expandiert bei fehlendem Treffer still zu nichts, `cfg_select!` bricht stattdessen ab. `sys/alloc/mod.rs` hat Arme für elf andere Plattformen, von `unix` bis `zkvm`, jedoch weder einen `_`-Arm noch einen für D3OS. Der übernommene Port hat sich stillschweigend

auf das Nicht-Expandieren verlassen. An dieser Stelle ist der leere Arm entstanden, den Abschnitt 3.2 beschreibt. Eine Makro-Migration bricht somit nicht nur die Syntax, sondern auch Annahmen darüber, was bei fehlendem Treffer geschieht.

Fünf Fehler verblieben, alle in `sys/pal/d3os/thread.rs` und alle von derselben Art. Für `Thread`, für alle Methoden, für `DEFAULT_MIN_STACK_SIZE`, `current_os_id` und `available_parallelism` meldet der Compiler jeweils `never used`, und `-D warnings` eskaliert diese Meldungen zu harten Fehlern. Das deutet auf eine verwaiste Datei hin. Zwei Befunde bestätigen dies. `library/std/src/sys/thread/` existiert mit `unsupported.rs`, `unix.rs` und `motor.rs`, jedoch ohne `d3os.rs`, und in `sys/pal/unsupported/mod.rs` ist kein `pub mod thread;` deklariert, in `sys/pal/d3os/mod.rs` dagegen schon. Damit liegt der erste der beiden Fälle vor, die Abschnitt 3.2 ankündigt. Die Thread-Abstraktion ist aus der Portierungsschicht in ein eigenes Subsystem-Verzeichnis gezogen.

Der Eingriff besteht aus zwei Schritten, dem Löschen von `thread.rs` und dem Entfernen von `pub mod thread;` aus der Modulliste. Ein dritter Schritt, mit dem zu rechnen war, entfiel. Erwartet wurde, dass wie beim Allokator ein eigener Auffangzweig zu ergänzen ist. Am Ende von `sys/thread/mod.rs` befindet sich jedoch bereits ein `_`-Arm, der auf `unsupported` routet und alle Items re-exportiert. Zwei Subsysteme durchlaufen damit denselben Umzug bei unterschiedlichem Aufwand. Beim Allokator war ein Arm zu ergänzen, bei den Threads lediglich eine Datei zu entfernen. Der Unterschied liegt nicht am Port, sondern daran, ob Upstream für das jeweilige Subsystem einen sinnvollen Rückfall vorsieht. Für einen Allokator besteht keiner, für einen Thread-Stub hingegen schon.

Hinzu kamen fünf Fehler in `sys/pal/d3os/time.rs` von gleicher Form. `Sub`, `Try` und `FromResidual` seien noch nicht als `const trait` stabil, da 1.92 die const-Auswertung von `-` und `?` strenger absichert. Die Kopierregel aus Etappe A greift hier zum zweiten Mal, denn `time.rs` ist ein reiner Stub. Die Kopie aus `sys/pal/unsupported/` fügt fünf Zeilen hinzu und entfernt dreizehn.

Damit baute der Compiler durch. Auf der D3OS-Seite trat anschließend die Klasse erneut auf, die bereits in Etappe A betroffen war:

```
target-pointer-width: invalid type: string "64", expected u16 at line 8 column 32
```

Am Ende von Etappe A stand `target-pointer-width` als String und `target-c-int-width` als Zahl. 1.92 verlangt für beide eine Zahl. Der erwartete Typ hat sich somit nicht in eine Richtung entwickelt, sondern hin und zurück. Ein solches Schema, das ausschließlich über Fehlermeldungen zugänglich ist, lässt sich nicht vorhersagen, sondern nur abfragen. Der Eingriff war zweimal erforderlich, da `d3os_kernel.json` und `d3os_application.json` unabhängige Dateien sind und rustc jede für sich liest. Nach der Korrektur in `d3os_kernel.json` baute der Kernel durch, und `cargo make` ging weiter zum Userspace. Dieser liest `d3os_application.json`, wo der Wert noch als String stand, sodass der Build ein zweites Mal mit derselben Meldung abbrach. Ein anschließendes `grep` über alle JSON-Dateien im Repository bestätigte, dass keine weitere Spezifikation den alten String enthält.

Damit baute der Fork durch. Die Toolchain meldet `rustc 1.92.0-dev` und `cargo 1.92.0-dev (367fd9f21 2025-10-15)`. Nach einem `cargo clean` auf der D3OS-Seite bootet das Abbild in QEMU, und `stdlibtest` gibt dieselben vier Zeilen aus wie am Ausgangsstand. Der Eingriff im Fork umfasst neun Dateien mit 86 hinzugefügten und 174 entfernten Zeilen (Commit `f8ca6e7c`), auf der D3OS-Seite zwei Dateien mit je einer Zeile. Damit stehen

am Ende der Etappe zwölf Anpassungsklassen. Ob der Aufwand bei zwei Minor-Versionen weiter steigt, prüft die nächste Etappe.

4.4 Etappe C: 1.92 → 1.94

Ziel ist die Nightly vom 1. Januar 2026 mit dem Commit `8d670b93`. Zwischen ihr und dem Anker aus Etappe B liegen zwei Minor-Versionen. Der Rebase spielt vierzehn Commits ab und hält nur zweimal an, beide Male an bekannter Stelle: in `sanity.rs`, wo diesmal `x86_64-unknown-motor` herausgewachsen ist und `riscv64im-unknown-none-elf` neu hinzukommt, sowie in `library/Cargo.lock`.
Nach Etappe B ist das Vorgehen ein anderes. Anstatt unmittelbar zu bauen, gehen dem Build drei Prüfungen voraus, die sich aus den Befunden der Vorgängeretappe ergeben. Der Befehl `git grep` aus Abschnitt 4.3 durchsucht den gesamten Baum nach Zeilen, die mit `<<<<<<<` oder `>>>>>>>` beginnen, also nach Konfliktmarkern, die beim Rebase mitcommittet wurden. Er meldete allein die bekannten Falschtreffer aus der Testsuite. Die zweite Prüfung ging die `cfg_select`-Dispatcher einzeln durch, da Git nicht überlappende Änderungen ohne Warnung zusammenführt und ein falsch gemischter Arm erst beim Bauen auffällt. Alle d3os-Arme standen unverändert da. Die Dateien, die in Etappe B fünf Eingriffe erfordert hatten, brauchten keinen einzigen.
Die dritte Prüfung lieferte einen Befund. Der Abgleich von `sys/pal/d3os/` gegen `sys/pal/unsupported/` fand `pipe.rs` im Port, jedoch nicht mehr in der Vorlage. Upstream hat die Pipe-Abstraktion nach `sys/pipe/` gehoben, und zwar nach demselben Muster wie zuvor die Threads in Etappe B. Der dortige Dispatcher besitzt einen `_`-Arm. `pub mod pipe;` ist aus der Modulliste entfernt und die Datei gelöscht. Dass eine zweite Wanderung dieser Art folgen würde, war nach Etappe B absehbar. Neu ist, dass sie vor dem Build aufgefallen ist.
Der erste `./x build` brach anschließend an einer bislang unbeteiligten Stelle ab, nämlich im Compiler selbst und damit noch vor der Standardbibliothek.

```
error[E0277]: the trait bound `Os: From<&str>` is not satisfied
--> compiler/rustc_target/src/spec/base/d3os.rs:6:20
error[E0277]: the trait bound `spec::Arch: From<&str>` is not satisfied
--> compiler/rustc_target/src/spec/targets/x86_64_unknown_d3os.rs:16:24
```

In `spec/base/d3os.rs` stand in Zeile 6 `os: "d3os".into()` und in `spec/targets/x86_64_unknown_d3os.rs` in Zeile 16 `arch: "x86_64".into()`. Beide Felder waren bis dahin freie Zeichenketten, und `.into()` erzeugte daraus ein `Cow<str>`, ohne eine Prüfung vorzunehmen. Ein Tippfehler wie `os: "lnux"` wäre klaglos durchgelaufen. In Version 1.94 handelt es sich um typisierte Enums mit einer benannten Variante je Upstream-bekanntem Wert und einer Auffangvariante `Other` für alles Übrige.
Die Definition von `Os` gibt die Antwort. Da Upstream D3OS nicht kennt [22], gehört der Wert in die Auffangvariante, und aus Zeile 6 wird `os: Os::Other("d3os".into())`. Eine eigene Variante `Os::D3os` lag zunächst nahe, scheidet jedoch aus, da das Enum in erschöpfenden `match`-Ausdrücken im gesamten Compiler verwendet wird und jede zusätzliche Variante dazu zwingen würde, alle diese Ausdrücke zu erweitern. Für die Auswertung von `cfg(target_os = "d3os")` macht `Other` keinen Unterschied, da die Variante die Zeichenkette weiterträgt. Beim Feld `arch` gilt das Gegenteil. Upstream kennt `x86_64` [23],

und `Arch::Other("x86_64")` lieferte zwar das richtige `cfg`, ließe jedoch jeden Variantenvergleich im Compiler fehlschlagen. Aus Zeile 16 wird daher `arch: Arch::X86_64`. Hinzu kommt in beiden Dateien der Import des jeweiligen Enums. Ein bekannter Wert gehört somit auf die benannte Variante, ein eigener auf `Other`.
Beim Bau der Standardbibliothek trat anschließend eine bereits bekannte Klasse erneut auf:

```
error[E0599]: no associated item named `MAX` found for
`pal::d3os::time::SystemTime`
```

Die Kopierregel aus Etappe A greift zum dritten Mal, da `time.rs` im Port weiterhin ein reiner Stub ist. Danach baute der Fork durch, und die Toolchain meldet `rustc 1.94.0-dev`.
Auf der D3OS-Seite liefen die beiden JSON-Zielbeschreibungen erstmals ohne Eingriff durch, da die Enum-Umstellung die Rust-Spezifikationen und nicht die JSON-Dateien betrifft. Betroffen war stattdessen der Kernel-Code selbst:

```
error[E0599]: no function or associated item named `new_const` found
for struct `Arguments<'a>`
--> os/kernel/src/lib.rs:95:26
```

In Zeile 90 legt der Panic-Handler die Panic-Nachricht in einem Array mit genau einem Element ab und erzeugt in Zeile 95 mithilfe von `Arguments::new_const(&args)` ein speicherbares `Arguments` für den Log-Eintrag. Dieser Konstruktor ist in Version 1.94 nicht mehr verfügbar, und der Compiler schlägt stattdessen `from_str` vor. Für ein einzelnes Element ist beides gleichwertig, da sowohl `new_const(&[s])` als auch `from_str(s)` ein `Arguments` ohne Formatargumente liefern. Damit stehen zwei Fassungen zur Wahl. Entweder entfallen in Zeile 90 die eckigen Klammern, sodass der Aufruf `from_str(args)` lautet, oder das Array bleibt stehen und wird in Zeile 95 indiziert, sodass der Aufruf `from_str(args[0])` lautet. Gewählt ist die zweite Fassung, da sie nur eine einzige Zeile ändert. Das Ein-Element-Array bleibt dadurch überflüssig bestehen, verursacht zur Laufzeit jedoch keine Kosten.
Der folgende Lauf widerlegte die Annahme, dass die neue Methode wie die alte unter dem bereits aktiven `fmt_internals` liegt:

```
error[E0658]: use of unstable library feature `fmt_arguments_from_str`
```

Upstream hat sie in ein eigenes Gate mit eigenem Tracking-Issue herausgelöst, was dem üblichen Vorgehen vor der Stabilisierung einer einzelnen Schnittstelle entspricht. Ein neuntes `#![feature(...)]` in `os/kernel/src/lib.rs` behebt das Problem, wobei `fmt_internals` daneben bestehen bleibt, da Zeile 90 weiterhin davon abhängt. Eine umbenannte und neu verriegelte instabile Schnittstelle äußert sich somit in zwei aufeinanderfolgenden Meldungen, zuerst `not found` und danach `feature not enabled`.
Damit übersetzt D3OS und bootet in QEMU. Der Kernel-Header meldet `Compiler: rustc 1.94.0-dev`, und `stdlibtest` gibt dieselben Zeilen aus wie am Ausgangsstand. Der Eingriff umfasst im Fork fünf geänderte und eine gelöschte Datei (Commit `1fffe612`),

auf der D3OS-Seite eine Datei mit zwei Zeilen (Commit `9b52aa93`), und am Ende der Etappe stehen sieben Anpassungsklassen. Was die drei Etappen zur Vorhersage aus Abschnitt 4.1 beitragen, ist Gegenstand des folgenden Abschnitts.

4.5 Bilanz der drei Etappen

Abschnitt 4.1 legt vier Zahlen als Nenner fest: acht Feature-Gates, zwölf Assembly-Stellen, 29 Stellen mit `extern "C"` und zwei JSON-Zielbeschreibungen. Tabelle 4.1 rechnet sie nach drei Etappen und vier Minor-Versionen ab.

Gruppe	**Anzahl**	**Kategorie**	**betroffen**
Feature-Gates	8	hoch	0
JSON-Zielbeschreibungen	2	hoch	2
Assembly-Stellen	12	mittel	0
Importblöcke `extern "C"`	5	mittel	0
Funktionsdefinitionen `extern "C"`	24	niedrig	0
Summe	51		2

Tabelle 4.1: Die Impact-Analyse aus Abschnitt 4.1 und ihre Abrechnung nach drei Etappen. Die letzte Spalte nennt die Stellen, an denen tatsächlich ein Eingriff erforderlich war. Das in Etappe C ergänzte neunte Feature-Gate zählt nicht dazu, da es keine der erfassten Stellen betrifft.

Was sich in Etappe A bereits abzeichnete, bestätigte sich in den Etappen B und C. Betroffen war allein die kleinste Gruppe, nämlich die beiden JSON-Dateien in den Etappen A und B, sodass sich von 51 erfassten Stellen genau zwei verändert haben.
Die Einstufung trifft dabei in zweifacher Hinsicht nicht zu. Die acht Gates und die beiden JSON-Dateien standen gemeinsam in der Kategorie „hoch". Der Eingriff traf jedoch nur den kleineren Teil, und die Begründung für den größeren Teil war zusätzlich falsch. Erwartet wurde eine Stabilisierung, also der Wegfall eines Gates. In Etappe C kam der Eingriff aus der entgegengesetzten Richtung. Upstream löste eine Schnittstelle in ein neues Gate heraus, sodass der Kernel ein neuntes Gate benötigte anstatt eines weniger.
Zwei der drei Kategorien trafen hingegen zu. Für „mittel" standen die Assembly-Stellen und die fünf Importblöcke, deren Blocksyntax sie hätte brechen können. Über vier Minor-Versionen hinweg ist das nicht eingetreten. Für „niedrig" standen die 24 C-ABI-Definitionen, die ebenfalls unverändert blieben. Zugetroffen hat die Einstufung somit in zwei von drei Kategorien, nicht jedoch in der obersten.
Der Grund liegt in der Fragestellung selbst. Die Inventur zählt, was D3OS an Code enthält. Die Anpassungen stammen jedoch fast ausschließlich aus dem Fork, nämlich aus der Bootstrap-Zielliste, aus den Dispatchern der Standardbibliothek, aus wandernden Subsystemen und zuletzt aus der Targetspezifikation des Compilers. Über alle drei Etappen zusammen stehen 27 Anpassungsklassen, auf der D3OS-Seite waren dabei genau drei Dateien betroffen.
Damit sind alle drei Schätzungen widerlegt, die dieser Arbeit vorausgingen. Ein Versionswechsel ist nicht der Zeilenwechsel, für den er anfangs gehalten wurde. Sein Aufwand entspricht nicht dem Umfang des Patch-Stacks, wie in Abschnitt 4.1 angenommen. Und die Impact-Analyse hat genau die Gruppe verfehlt, der sie das größte Risiko zugeschrieben hatte. Die drei Fehleinschätzungen weisen in dieselbe Richtung, denn sie messen alle den

eigenen Code, während der Aufwand aus dem fremden Projekt stammt. Wer den Aufwand eines solchen Wechsels vorab schätzen will, zählt somit auf der falschen Seite.
Tragfähig waren stattdessen die Vorhersagen aus den Etappen selbst. Das Herauswachsen von `aarch64_be` ist in Abschnitt 4.2 namentlich benannt und in Etappe B eingetreten. Die zweite Subsystem-Wanderung war nach Etappe B zu erwarten und wurde in Etappe C bei `pipe` gefunden. Beide ergaben sich daraus, dass die Vorgängeretappe ein Muster offengelegt hatte.
Woran der Aufwand tatsächlich hängt, zeigen die drei Etappen selbst. Tabelle 4.2 stellt sie nebeneinander.

Etappe	**Sprung**	**Minor**	**Klassen**	**Konflikte**
A	1.90 → 1.91	1	8	2
B	1.91 → 1.92	1	12	8 in 4 Commits
C	1.92 → 1.94	2	7	2

Tabelle 4.2: Minor-Versionen, Anpassungsklassen und Konflikte je Etappe

Der größte Sprung hat die wenigsten Konflikte verursacht. Etappe C überspringt zwei Minor-Versionen und kommt auf dieselben zwei Konflikte wie die kleinste Etappe, während Etappe B acht Konflikte bei einer einzigen Version erzeugt. Ein Aufwandsmodell nach Versionsdistanz ist damit widerlegt. Maßgeblich sind stattdessen die Dispatcher. In Etappe B waren fünf manuelle Eingriffe erforderlich, in Etappe C liefen dieselben Dateien unverändert durch, und von den sieben Anpassungsklassen der Etappe C waren fünf bereits aus den Vorgängeretappen bekannt. Der Aufwand folgt somit der Restrukturierungstiefe in genau denjenigen Subsystemen, die der Port berührt.
Etappe C fügt eine zweite Größe hinzu, nämlich die Lage. Bis dahin lag die Last ausschließlich bei der PAL und den Dispatchern. In Etappe C waren erstmals die Compiler-Zielbeschreibung und der D3OS-eigene Quellcode betroffen, während PAL und Dispatcher mit zwei Klassen auskamen, die sich auf das Löschen und das Kopieren einer Datei beschränkten. Verallgemeinert bedeutet dies: Aufwand entsteht dort, wo der Port eigene Artefakte hält, also eine eigene Spec-Datei, eigene PAL-Module oder eine eigene Feature-Nutzung. Wo er den Auffangzweig von Upstream mitbenutzt, entsteht keiner.
Einschränkend bilden drei Messpunkte keinen Trend, und die Konfliktzahl selbst ist vorsichtig zu interpretieren. Sie unterschätzt die Anpassungsfläche, denn in Etappe B kamen zwei Dateien hinzu, die Git nie gemeldet hat. `thread_local/mod.rs` verschmolz still, da beide Textänderungen disjunkt waren, und `stdio/mod.rs` gelangte mit Konfliktmarkern in den Baum, ohne dass eine Prüfung dies aufgedeckt hätte.
Damit liegt die Grundlage für die Beantwortung der ersten Leitfrage vor, die Kapitel 7 abschließend zusammenführt. Am Ende der Toolchain-Arbeit bootete das System fehlerfrei und stand für die vier Erweiterungen von `std` bereit, die Kapitel 5 darstellt.

Kapitel 5

Die Standardbibliothek erweitern

Dieses Kapitel beantwortet die zweite Leitfrage, also die Frage, in welchem Umfang sich bestehende Plattform-Backends der Rust-Standardbibliothek als Vorlage übernehmen lassen und an welchen Stellen eine eigenständige Implementierung erforderlich ist. Das Verfahren dazu legt Abschnitt 3.1.2 fest, die je Modul gewählte Vorlage weist Tabelle 2.1 aus. Die vier Module folgen in der Reihenfolge ihrer Entstehung, und Abschnitt 5.5 rechnet die Herkunft des entstandenen Codes gegen diese Vorgabe ab.

5.1 Dynamische Speicherverwaltung

Den Ausgangszustand dieses Moduls beschreibt Abschnitt 2.3 am alten Allokator. Er erfüllt eine vollständige Signatur und weist dahinter vier Defekte auf. Die erste Frage lautete daher nicht, wie ein Allokator zu schreiben ist, sondern welcher zu verwenden ist. Da Freilisten, Splitting und Coalescing bereits gelöste Probleme sind, wäre eine Eigenentwicklung die schlechteste der verfügbaren Lösungen gewesen.
Zwei Kandidaten standen zur Wahl. Der erste lag am nächsten, denn D3OS setzt `linked_list_allocator` 0.10.5 bereits im `KernelAllocator` und in der `runtime`-Crate ein. Er scheitert nicht an seiner Funktion, sondern am Build-Graphen. Diese Version kennt weder das Feature `rustc-dep-of-std` noch die `rustc-std-workspace`-Shims. Somit entstünde genau die zweite Instanz von `core`, die Abschnitt 3.2 beschreibt. Nutzbar wäre er nur über einen Fork. Übrig blieb `dlmalloc` 0.2.10, das ohnehin im Quellbaum liegt, weil std es für wasm, xous, vexos und sgx benutzt, und das zusätzlich `rustc-dep-of-std` mitbringt. Die Betreuung hat die Wahl bestätigt und auf ein anderes Betriebssystem verwiesen, in dem diese Anbindung bereits existiert. Dabei handelt es sich um xous, das Tabelle 2.1 als Vorlage für dieses Modul führt.
Die resultierende Architektur gliedert sich in drei übereinanderliegende Schichten. Diese Aufteilung entscheidet darüber, wie groß der eigene Implementierungsanteil ausfällt. Abbildung 5.1 zeigt die einzelnen Schichten. Wichtiger als der Umfang ist jedoch die Frage, wie oft die Schichten laufen. Schicht 3 und Schicht 2 arbeiten bei jeder Allokation und bei jeder Freigabe, Schicht 1 dagegen nur, wenn die Engine nichts Wiederverwendbares mehr hat. Eine Freigabe erreicht das Betriebssystem nie.
Die Schichten 3 und 2 sind nach `sys/alloc/xous.rs` aufgebaut [24], während Schicht 1 auf der Datei `src/xous.rs` aus `dlmalloc-rs` basiert [25]. Der eigentliche Befund ist, dass die Vorlage auf zwei Repositories verteilt liegt. Xous konstruiert seine Engine mit `Dlmalloc::new()` und erhält so das Backend, das die Crate über `cfg(target_os)` für

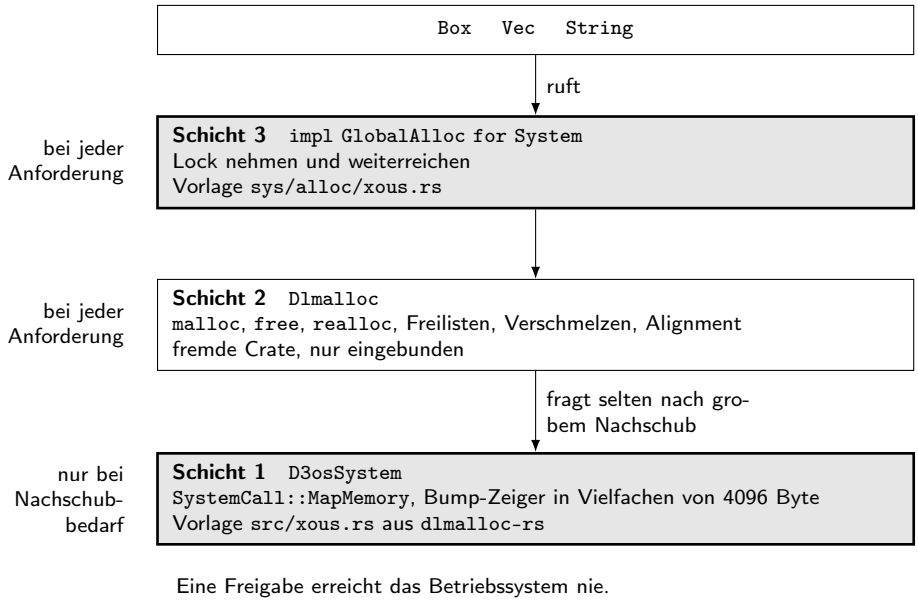


Abbildung 5.1: Die drei Schichten des Allokators. Die Schichten 3 und 2 laufen bei jeder Anforderung, Schicht 1 nur bei Nachschubbedarf. Grau hinterlegt und dick umrandet ist der Teil, den diese Arbeit geschrieben hat.

ihr Ziel mitbringt. Für D3OS gibt es dort nichts. Deshalb wird `new()` zu `new_with_allocator(D3osSystem)` [26], und an dieser einen Zeile hängt die gesamte Schicht 1.

Schicht 1 reserviert den Heap in einem einzigen `SystemCall::MapMemory` und vergibt daraus über einen Bump-Zeiger in Vielfachen von 4096 Byte. Die Werte `HEAP_START` bei 63 TiB und `HEAP_SIZE` von 1 TiB sind aus der Datei `runtime/src/env.rs` übernommen, damit std genau die Region benutzt, die D3OS für Userspace-Heaps vorsieht. Xous fordert stattdessen bei jedem Nachschubbedarf nach. Der Entscheidung gegen dieses Verfahren ging eine Prüfung der Kernel-Seite voraus. `sys_map_memory` in `os/kernel/src/syscall/sys_vmem.rs` legt ausschließlich eine VMA an und vermerkt das im Quelltext mit „This just sets up the VMA, no page tables are created yet“. Physische Frames folgen erst beim Page-Fault. Ein Terabyte zu reservieren kostet somit nichts, während inkrementelles Anfordern mit jedem Nachschub eine weitere VMA in der `BTreeMap` des Adressraums hinterließe, die der Page-Fault-Handler bei jedem Fehler abfragt.

Eine Rückgabe an das Betriebssystem findet nicht statt. `remap` liefert einen Nullzeiger, die übrigen Rückgabemethoden liefern `false` und der Bump-Zeiger wächst nur. Ein Speicherleck entsteht dadurch nicht, da die Wiederverwendung vollständig in Schicht 2 stattfindet. Beim Lock in Schicht 3 gibt der Wartende die CPU per `SystemCall::ThreadSwitch` ab, den der Kernel als `sys_thread_switch` bereitstellt.

Einen Wert der Vorlage übernimmt das Backend bewusst nicht. Xous meldet `allocates_zeros() == true` und teilt der Engine damit mit, dass der neu beschaffte Speicher bereits genullt ist. Für D3OS steht dort `false`, und zwar aus zwei Gründen. Erstens soll sich das Backend nicht darauf verlassen, dass der Kernel die Frames beim Einlagern nullt. Träte das nicht ein, würde `alloc_zeroed` unbrauchbare Daten liefern, ohne dass es zu einem Absturz oder einer Meldung käme. Zweitens bleibt `false` auch dann richtig, wenn sich D3OS darunter ändert, während `true` nach jedem Eingriff in die Speicherverwaltung neu zu prüfen wäre. Genullt wird ohnehin, nur eben von `dlmalloc` und nicht vom Betriebssystem.

An einer Stelle ließ sich die Vorlage nicht übernehmen. Xous rechnet die vom Kernel gelieferte Adresse mit einem einfachen `as`-Cast in einen Zeiger um. Genau das lehnt der Bau ab, denn in Zeile 248 von `library/std/src/lib.rs` ist der Lint `fuzzy_provenance_casts` auf `deny` gesetzt. Die Vorlage darf den Cast jedoch benutzen, weil sie in einer externen Crate liegt. Das Backend dieser Arbeit liegt dagegen im Quellbaum von std und unterliegt dessen internen Lints. Dort steht jetzt `ptr::with_exposed_provenance_mut::<u8>(addr)`, was zur Sache ohnehin besser passt, denn die Adresse stammt aus einem Kernel-Mapping und nicht aus einem Zeiger, von dem sie abgeleitet wäre. Danach baute die Toolchain durch (Commit `069a1e3`). Damit ist das erste der vier Module abgeschlossen. Das zweite scheiterte nicht an einem fehlenden Systemaufruf, sondern an einem fehlenden Byte.

5.2 Standardeingabe

Was `Stdin` bis dahin tat, beschreibt Abschnitt 2.3. `read` liefert bedingungslos `Ok(0)` zurück, und ebenso verhalten sich alle übrigen Methoden des Traits. Tabelle 2.1 führt dieses Modul als einziges ohne Vorlage, da das Backend `sys/stdio/d3os.rs` vorhanden und in der Schreibrichtung funktionsfähig war. Nur der Lesepfad hat gefehlt.

Die Anbindung selbst ist klein. Der Trait `Read` verlangt genau eine Methode ohne Vorgabeimplementierung, nämlich `fn read(&mut self, buf: &mut [u8])`, während alle übrigen darauf zurückgeführt sind [10]. Wichtiger als der neue Code war deshalb, was wegfiel. Die geerbten Stubs waren nicht nur überflüssig, sondern schädlich, denn ein `read_to_end`, das selbst `Ok(0)` zurückgibt, hätte die neue Implementierung nie aufgerufen. Die Aufrufkonvention ist nicht geraten, sondern der Referenzbibliothek `terminal` aus Abschnitt 3.2 entnommen und gegen die Signatur des Kernel-Handlers `sys_terminal_read_input( address: *mut u8, length: usize, mode: usize)` gehalten. Der Modus steht als 0 im Aufruf, weil std nicht von der `terminal`-Crate abhängen darf und `TerminalMode::Canonical` dort die erste Variante des Enums ist. Der ganze Lesepfad umfasst neun Zeilen Code (Commit `ec31af0`). Der Aufwand dieses Moduls liegt jedoch nicht in dieser Anbindung, sondern in den daran anschließenden Prüfungen.

Der rohe `read` lief unmittelbar. Damit ist allerdings noch wenig erreicht, denn kein Rust-Programm liest die Standardeingabe so, sondern über `read_line` und die übrigen Komfortfunktionen von std. Eine Prüfung auch dieser Funktionen ergab, dass jede Eingabe zweimal Enter statt einmal benötigt.

Die Ursache liegt auf beiden Seiten der Schnittstelle. `Canonical::submit()` in der D3OS-Anwendung `terminal_emulator` liefert die fertige Zeile ohne abschließenden Umbruch aus, da dieser nur an das Display geschickt wird. Im kanonischen Modus wird eine Zeile aber durch das Newline begrenzt [27]. Die Grenze zu setzen ist also Aufgabe des Terminals und nicht des Lesers. `BufRead::read_line` verlässt sich darauf und ist nichts anderes als `read_until(b'\n')`. Ein Trennzeichen, das nie ankommt, wird nie gefunden. Erst beim zweiten Enter kommen null Bytes zurück, und die liest std als Ende der Daten.

Der erste Eingriff lag in der Portierungsschicht. Wenn der Puffer noch Platz hat, hängt `read` das `\n` an, also im Zweig `Ok(n) if n < buf.len()`. Dieser Eingriff hat funktioniert und wurde der Betreuung als Leitlinie vorgeschlagen, weil er D3OS unangetastet lässt und die Abweichung an der Grenze der Standardbibliothek kapselt. Die Antwort war eine Gegenentscheidung. Die Korrektur gehöre nach D3OS selbst. Aus dieser Antwort ist die Regel in Abschnitt 3.1.3 entstanden.

Damit war die Frage jedoch nicht beantwortet, denn zwischen der Tastatur und der Standardbibliothek liegen mehrere Schichten, und in jeder ließe sich das fehlende Newline nachtragen. Tabelle 5.1 fasst die vier geprüften Stellen zusammen. Ausschlaggebend war, welche davon überhaupt weiß, worum es geht. Der Eingriff gehört an die Stelle, die die semantische Information besitzt. Das ist `Canonical::submit()`, denn nur dort ist bekannt, dass eine Zeile mit Enter abgeschlossen wurde. Dort steht jetzt ein `buffer.push('\n')` (Commit `acf9c20`). Damit wird kein Byte erfunden, sondern ein tatsächlich getipptes weitergereicht. Dass dieser eine Eingriff jeden Leser erreicht, lässt sich nachprüfen. Alle Leser holen ihre Bytes aus derselben Warteschlange des Kernels, und hineinlegen kann dort nur, wer den Systemaufruf `TerminalWriteInput` absetzt. Ein `grep` danach liefert außerhalb des Kernels genau einen Aufrufer, den Emulator.

Ort	Befund
`tty.rs::write` im Kernel	Der Schreibpfad reicht die Bytes lediglich durch und sieht nie hinein. Zwar prüft `write` den Modus und lehnt einen unpassenden ab, doch dass eine Zeile zu Ende ist, weiß der Kernel nicht, da der Emulator die Eingabe Enter bereits in Bytes übersetzt hat. Verworfen.
`terminal::read`	Erreicht std nicht, da diese den Systemaufruf selbst absetzt. Dazu der Randfall einer Zeile, die den Puffer exakt füllt und keinen Platz mehr lässt. Verworfen.
Portierungsschicht	Kapselt die Abweichung an der Grenze, lässt D3OS selbst aber falsch. Von der Betreuung abgelehnt.
`Canonical::submit()`	Einzige Stelle, an der bekannt ist, dass die Zeile mit Enter abgeschlossen wurde. Nur im kanonischen Pfad aktiv, versorgt alle Leser aus einer Quelle. Gewählt.

Tabelle 5.1: Die vier geprüften Orte für den fehlenden Zeilenumbruch und der jeweilige Grund für das Ausscheiden.

Vor der Änderung waren die bestehenden Leser zu ermitteln. Genau drei Programme rufen `read` im kanonischen Modus auf, nämlich `nc`, die `legacy_shell` und `keytest`. Die Anwendung `nc` enthielt in `nc.rs` eine eigene Abhilfe in Form eines angehängten `message.push('\n')`. Das hätte nun einen doppelten Umbruch zur Folge gehabt. Die Zeile ist entfallen (Commit `e4f64fd`). Die `legacy_shell` zerlegt ihre Eingabe mit `split_whitespace`, und ein Umbruch ist ein Whitespace. Somit war eine Änderung nicht nötig. Unberücksichtigt blieb zunächst `keytest`. Das Programm vergleicht die gelesene Zeile direkt mit `"exit"`, und dieser Zweig griff seitdem nie mehr, sodass der kanonische Testmodus nicht mehr zu verlassen war. Der Vergleich schneidet den Umbruch jetzt ab (Commit `95feda3`). Nicht jeder Leser verträgt also das zusätzliche Byte, und `keytest` ist nicht die einzige Folge. Ein leeres Enter liefert jetzt ein Byte statt null Bytes, was zur kanonischen Semantik passt, dem System aber sein bisheriges Ende-Signal nimmt. Da der Emulator kein Gegenstück zu Strg+D kennt, also zu der Taste, die an einem Unix-Terminal das Ende der Eingabe meldet, ist dieses Ende seitdem gar nicht mehr zu signalisieren und `read_to_string` blockiert unbegrenzt. Das ist keine Eigenheit von D3OS, sondern die Folge einer fehlenden Taste.
Beim Prüfen des Lesepfads trat ein Defekt zutage, der mit dem Zeilenumbruch nichts zu tun hat. Die Eingabe von `ä` erzeugte weder ein Echo noch eine Fehlermeldung, und anschließend nahm das Terminal überhaupt keine Eingaben mehr an. `Canonical::cursor_pos`

wird pro Zeichen hochgezählt, aber als Byte-Index in einen UTF-8-String benutzt. Unter ASCII sind beide Werte gleich, ein `ä` belegt jedoch zwei Byte, sodass schon das Echo des ersten ä mitten in das Zeichen schneidet. `redraw_canonical_content` bricht daraufhin mit einem Panic ab, der die Meldung „byte index is not a char boundary“ trägt und den niemand sieht, weil der Panic-Handler der no_std-Laufzeitumgebung den Thread stumm beendet.
Ein Debugger half dabei nicht weiter. Der gdb-Stub von QEMU debuggt die Maschine und nicht das Betriebssystem, er kennt weder die D3OS-Prozesse noch deren Ladeadressen, und der Absturz fand ohnehin in einem anderen Prozess statt als dem beobachteten. Vor und hinter jeden Schritt des Editors trat deshalb eine Zwischenausgabe, die direkt über `self.terminal.write_str` in den Bildspeicher schreibt, also an der blockierten TTY-Kette vorbei. Die Ausgabe `Hex:E4,Bytes:2` belegte, dass Tastatur, Layout und Transportweg das Zeichen korrekt bis in den Editor übermitteln. Die letzte Zwischenausgabe stand vor, aber nicht hinter dem Redraw-Aufruf. Wenn kein Debugger die Prozessgrenze kennt, muss die Messung somit einen Ausgabeweg nehmen, den die Störung nicht blockiert.
Behoben ist das Problem damit nicht. Nach Rücksprache mit der Betreuung liegt die Fehlerbehebung außerhalb des Arbeitsumfangs, da Mehrbyte-Zeichen im D3OS-Terminal generell fehlerhaft sind und auch die Shell abstürzt, sobald der Cursor über eines davon läuft.
Die Fehlerdifferenzierung bleibt offen. Da der Rückgabeweg aus Abschnitt 3.2 jeden Fehlercode auf `EUNKN` zusammenfaltet, bildet `read` pauschal auf `ErrorKind::Other` ab. Für die Standardeingabe genügt das. Bei Dateien ist es nicht ausreichend, da die Fehler `NotFound`, `AlreadyExists` und `PermissionDenied` dort unterscheidbar sein müssen. Damit ist der Gegenstand des folgenden Abschnitts benannt.

5.3 Datei-Ein- und -Ausgabe

Für Dateien gab es im übernommenen Port nichts. In `sys/fs/mod.rs` fiel D3OS in den `_`-Zweig, sodass jede Operation einen Fehler lieferte, obwohl der Naming Service aus Abschnitt 3.2 die passenden Systemaufrufe längst bereitstellt.
Für die Vorlage sind sechs Backends verglichen und fünf davon ausgeschlossen worden. Die Vorbereitung dieser Arbeit nannte `wasi.rs` als den naheliegenden Kandidaten. Die Datei existiert jedoch nicht mehr, da WASI inzwischen in den `unix`-Zweig fällt. Sowohl `hermit` als auch `motor` bauen auf dem `sys/fd`-Gerüst auf. `motor` setzt zusätzlich eine fertige Dateisystem-Runtime des Zielsystems voraus, deren D3OS-Gegenstück ausgerechnet `naming` wäre. `vexos` wäre auch eine gute Wahl, arbeitet aber über C-`FILE*` statt über rohe Deskriptoren. Und `uefi` scheidet sofort aus, denn dort ist `File` als `File(!)` deklariert und hat gar keinen Deskriptor.
Aus drei Gründen blieb schließlich `solid.rs` übrig. Erstens liegt sein Deskriptor in derselben Datei, sodass kein zusätzliches `sys/fd/d3os.rs` benötigt wird. Zweitens entspricht sein Fehlermodell durchgehend der D3OS-Konvention aus Abschnitt 3.2, also „Rückgabe kleiner null heißt Fehler“. Drittens kommt jede Methode mit einem einzigen Aufruf an das Zielsystem aus, was dem Muster des bereits verifizierten stdio-Backends entspricht.
Die Vorlage trägt jedoch nicht überall. An vier Stellen sitzt die POSIX-Annahme nicht im ersetzten Aufruf, sondern im Code darum herum. Erstens hält `solid` den Deskriptor als `c_int`, in dem der Wert `-1` als Fehlerkennung reserviert ist. D3OS liefert `Result<usize,`

`Errno>` und benötigt keinen solchen Wächterwert. Zweitens verlangt `solid::cstr` einen Pfad, der mit einem Backslash beginnt, und lehnt relative Pfade ab. Drittens gibt `SOLID_FS_Lseek` die neue Position nicht zurück, weshalb `solid` anschließend `tell` aufruft. Viertens sind die Offsets dort vorzeichenbehaftet.

Für drei dieser Stellen gab es eine schlankere Alternative. `motor.rs` nimmt den Pfad mit `path.to_str()`, definiert `tell` als `seek(SeekFrom::Current(0))` und führt die vektorisierten Varianten auf die byteweise Fassung zurück. Genau das steht jetzt im Backend, obwohl `motor` als Vorlage ausgeschieden war. Nur für die vierte Stelle gab es kein Vorbild, dort steht `non_negative_offset`. Daraus wurde eine Regel. Erst im nächstliegenden Geschwister nachsehen, dann selbst schreiben.

Das in Abschnitt 3.1.2 beschriebene Verfahren aus `vexos` stellt hier dreiundzwanzig Typen und Funktionen als Stub bereit. Der Gewinn gilt nur für die Hälfte von ihnen, denn die Stubs zerfallen in zwei Gruppen. `readdir` und `mkdir` könnte D3OS durchaus, denn die entsprechenden Systemaufrufe existieren. Sie liegen jedoch außerhalb des Aufgabenbereichs dieser Arbeit. Für `unlink`, `rename`, `rmdir`, `stat`, `chmod`, `utimes`, `link` und `symlink` gibt es im `SystemCall`-Enum dagegen überhaupt kein Gegenstück. Diese Hälfte bleibt `unsupported`, solange D3OS sie nicht anbietet.

Die Anbindung folgt dem stdio-Backend. `naming` steht nicht in den Abhängigkeiten aus Abschnitt 3.2, also setzt das Backend die Systemaufrufe `Open`, `Read`, `Write`, `Seek` und `Close` selbst ab, und zwar mit Argumentlisten aus `naming/src/lib.rs`. Der Deskriptor ist der rohe Wert aus der Handle-Tabelle des Kernels, den `Drop` wieder schließt. Der Pfad erfordert eine eigene Behandlung. Ein `&str` in Rust ist ein Zeiger und eine Länge. Über die Grenze geht aber nur der Zeiger, da `sys_open` neben den Flags nichts weiter entgegennimmt. Wo die Zeichenkette endet, muss der Kernel deshalb selbst herausfinden. `ptr_to_string` tut das auf C-Art, indem es Bytes zählt, bis eine Null kommt. Also baut die Portierungsschicht einen `CString`, denn genau dieser Typ garantiert die abschließende Null und verbietet eine Null mittendrin. Ohne ihn würde der Kernel über das Ende hinaus lesen.

Dann folgten zwei Eingriffe an der Quelle. Der erste betrifft `naming/src/shared_types.rs`. Die Datei benutzt das Makro `bitflags!`, das für jede Konstante ein eigenes Bit erwartet, vergab die Werte aber fortlaufend von 1 bis 6. Was daraus folgt, zeigt `create_new`. Das ist `CREATE` zusammen mit `EXCLUSIVE`, also 3 und 4, was 7 ergibt. Die Abfrage `contains(DIRECTORY)` rechnet daraufhin 7 und 5 zu 5 zusammen und ist damit wahr. Eine gewöhnliche Datei wäre als Verzeichnis angefordert worden. Hinzu kommt, dass `sys_open` die Bits mit `from_bits(...).unwrap()` einliest. Die Maske aus 1 bis 6 ist 7, jeder Wert darüber liefert `None` und der Kernel bricht ab. Geändert wurde erst nach einem Blick auf alle elf Verwendungsstellen im Betriebssystem. Sieben davon konstruieren Flags und benutzen ausnahmslos die benannten Konstanten, während die vier Lesestellen bitweise arbeiten. Deshalb blieb es bei einer einzigen geänderten Datei, in der die Werte jetzt 1, 2, 4, 8, 16 und 32 heißen (Commit `322e47f`). Die Alternative wäre gewesen, aus der Portierungsschicht nur Kombinationen zu senden, die der Kernel unterscheiden kann. Das funktioniert, verlegt aber einen Defekt des Betriebssystems in die Übersetzungsschicht. Im Hauptzweig von D3OS stehen dieselben Werte schon seit dem 2. Januar 2026 (Commit `be4aa53`). In den Zweig, gegen den der Port gebaut wird, ist die Änderung nie angekommen, weshalb derselbe Defekt sieben Monate später ein zweites Mal gefunden wurde.

Der zweite Eingriff ergab sich aus dem Testlauf. Der erste Lauf nach dem Booten war erfolgreich, ein zweiter im selben Boot scheiterte jedoch beim Öffnen. Die Datei `/stdlibtest.txt`

überlebt im tmpfs innerhalb eines Boots, und `api::open` behandelte `CREATE` immer exklusiv und gab also `EEXIST` zurück. D3OS-`CREATE` entsprach damit dem POSIX-`O_CREAT | O_EXCL` [28]. std kennt drei Anlege-Modi, nämlich nur öffnen, öffnen oder anlegen und nur neu anlegen. D3OS konnte den mittleren nicht. Der Kommentar „avoid creating a file twice“ im Kernel zeigte allerdings, dass es sich dabei nicht um eine unbeabsichtigte Fehlfunktion handelte, sondern um eine bewusste Entscheidung. Daher erfolgte eine Rückfrage, anstatt den Kernel stillschweigend anzufassen. Die Antwort lag bereits vor. Tom Gehrmanns parallele Weiterentwicklung des Naming Service behandelt `open` genauso wie vorgeschlagen, nämlich `EEXIST` nur bei `CREATE` zusammen mit `EXCLUSIVE` und sonst öffnen oder anlegen. Beide Eingriffe enden gleich. Was an der Quelle erforderlich war, gab es schon, einmal im Hauptzweig und einmal in einem parallelen Zweig, nur nicht in dem Zweig, in dem diese Arbeit entstanden ist.

Übernommen ist genau diese eine Funktion (Commit `7c5964a`). Der Aufruf `resolve_canonical_path` bleibt auskommentiert, weil Pfadnormalisierung erst zusammen mit `getcwd` und `chdir` Sinn ergibt. Seine überarbeiteten `tmpfs.rs`, `open_objects.rs`, `traits.rs` und `shared_types.rs` sind aus drei Gründen nicht übernommen. Erstens liegen sie außerhalb des Aufgabenbereichs, zweitens vergrößern sie die Änderungsfläche ausgerechnet in Kernelbereichen, die nicht getestet werden, und drittens hätten sie die Grenze zwischen seinem und dem eigenen Beitrag verwischt. Was dabei liegen bleibt, sind keine Fehler des Datei-I/O, sondern Verfeinerungen, nämlich die Durchsetzung des Zugriffsmodus.

Gemäß derselben Regel ist die Fehlerdifferenzierung liegen geblieben, mit der der vorherige Abschnitt endet. Zwei Wege standen zur Wahl. Der erste bildet alles auf `ErrorKind::Other` ab, wie es bei der Standardeingabe der Fall ist. Der zweite aktiviert die auskommentierte Variante in `return_vals.rs` wieder und hätte den Fehlertyp für alle Nutzer der `syscall`-Crate und somit für das ganze Betriebssystem geändert. Genau diesen Weg ist der Hauptzweig von D3OS inzwischen gegangen. Dort steht nun `Err(Errno::from(ret_code))` statt `Err(Errno::EUNKN)`. Aus der Portierungsschicht heraus wäre das derselbe Griff in fremde Bereiche gewesen, der schon bei den Kerneldateien abgelehnt wurde. Gewählt ist der erste Weg. Die Kosten dieser Wahl zeigt der Test. Der abgelehnte negative Seek meldet `InvalidInput`, da dieser Fehler im eigenen Code entsteht. Ein `create_new` auf eine vorhandene Datei meldet dagegen nur `Other`, obwohl der Kernel sehr wohl `EEXIST` schreibt.

Nicht jede Abweichung ist ein Defekt. `SeekFrom::End` und `SeekFrom::Current` tragen ein `i64`, der D3OS-Offset ist ein `usize` und der Kernel addiert ihn lediglich. Ein Cast hätte aus `-1` beinahe `usize::MAX` gemacht und wäre unbemerkt übergelaufen. Deshalb lehnt die Portierungsschicht negative Offsets mit `InvalidInput` ab. Zwei Öffnungs-Flags von std haben in D3OS gar keine Entsprechung und werden deshalb ignoriert. `truncate` setzt die Länge einer vorhandenen Datei beim Öffnen auf null, sodass ihr bisheriger Inhalt verschwindet, und `append` legt jeden Schreibzugriff an das Dateiende, unabhängig von der aktuellen Position. Spürbar wird das bei `File::create`, das intern `truncate(true)` setzt. Wer damit kürzere Daten in eine bestehende Datei schreibt, überschreibt nur deren Anfang und behält den Rest der alten Datei. Der Zugriffsmodus, die beiden fehlenden Flags und die Fehlerdifferenzierung liegen alle unter der Portierungsschicht und nicht in ihr. Nur bei einem davon hat der Hauptzweig inzwischen entschieden, wie er aussehen soll.

Der letzte Fund kam nach dem erfolgreichen Test. `sys_read` und `sys_write` lehnen einen Puffer der Länge 0 mit `EINVAL` ab, während der Vertrag von `io::Read` dafür `Ok(0)` ver-

langt [10]. Der Schutz steht jetzt als erste Zeile in `read` und `write` (Commit `b86264d`). Bei der Standardeingabe lautete die Frage, was der Trait an den Rändern verlangt, und genau dort ist ein solcher Schutz eingebaut. Bei den Dateien lautete die Leitfrage, wie D3OS seine eigene Schnittstelle benutzt. Diese führt zum Datenpfad, aber nicht zum Vertrag. Beim letzten Modul ist deshalb von der Seite von std ausgegangen worden.

5.4 Threads

In dem übernommenen Port waren Threads eine Signatur ohne Inhalt. Der erste Hinweis auf die Vorlage stand im Verzeichnis-Listing. `unix.rs` hat 33 232 Byte und `solid.rs` 14 582, `hermit.rs` hingegen nur 2479 und `motor.rs` 1780. Dieser Unterschied hat seinen Grund. Solid und unix bauen eigene Thread-Tabellen und eigene Stapelverwaltung sowie ein `join` über Mutex und Condvar, da ihre Unterlage das nicht mitbringt. D3OS im Vergleich liefert `join`, `exit`, `switch` und `sleep` als je einen Systemaufruf. Erforderlich war somit eine Vorlage, die ebenfalls nur weiterreicht. Gewählt ist `hermit.rs`, ein dünner Wrapper über `hermit_abi`. Diese Crate spielt für hermit dieselbe Rolle, die `concurrent` und `syscall` für diese Arbeit einnehmen. Der zweitnächste Kandidat war `xous.rs` mit 5427 Byte, ebenfalls ein Mikrokernel über Systemaufrufen. Allerdings verwaltet xous `join` und die Thread-ID selbst im Benutzerraum. Das ist überflüssig, denn bei D3OS erledigt der Kernel diese Aufgabe.
Mit der Vorlage ist die zentrale Frage dieses Moduls noch nicht beantwortet, nämlich ob ein Zeiger auf eine Closure über `ThreadCreate` in den neuen Thread gelangt, ohne D3OS anzufassen. std übergibt beim Start einem Backend einen `Box<ThreadInit>`. D3OS erwartet dagegen ein `extern "sysv64" fn()` ohne Argument. Die Antwort steht in keiner Dokumentation, sondern im Datenpfad. Dieser ist Datei für Datei verfolgt worden.
Er beginnt in `syscall/src/lib.rs`, wo das erste Argument nach `rdi` und das zweite nach `rsi` geht. In `sys_concurrent.rs` nimmt `sys_thread_create` genau diese beiden entgegen, und `Thread::new_user_thread` legt sie als `user_kickoff` und `entry` im Thread ab. Beim Wechsel in den Benutzermodus setzt `process/thread.rs` das erste Argument als Sprungziel und reicht das zweite an `thread_user_start` weiter. Dort befindet sich die entscheidende Zeile, und der Kommentar daneben erklärt, worum es geht:

os/kernel/src/process/thread.rs — Rust

```
1 "mov rsp, rdi", // Load 'old_rsp' (first parameter)
2 "mov rdi, rsi", // Second parameter becomes first parameter for
    'kickoff_user_thread()'
3 "iretq"         // Switch to user-mode
```

Nach der Aufrufkonvention System V AMD64 liegt das erste Zeigerargument einer Funktion in `rdi` [14]. Der Kernel legt das zweite Argument also dorthin, wo das Trampolin das erste erwartet. Dabei achtet er nicht darauf, was er weiterreicht. Genau darauf verlässt sich `kickoff_user_thread` schon heute. Abbildung 5.2 zeigt den Weg im Zusammenhang. Damit ist zugleich die Einordnung nach Abschnitt 3.1.3 entschieden. Hier liegt weder ein Defekt noch eine bewusste Design-Wahl vor, sondern lediglich eine andere Repräsentation. D3OS legt an dieser Stelle einen Funktionszeiger ab und std einen Datenzeiger, und der Mechanismus trägt beide. Der Kernel hält diese Annahme selbst fest. Über `thread_user_start` steht `#[allow(improper_ctypes_definitions)]` mit der Begründung, dass `entry` weder Argumente noch einen Rückgabewert hat. Die Portierungsschicht

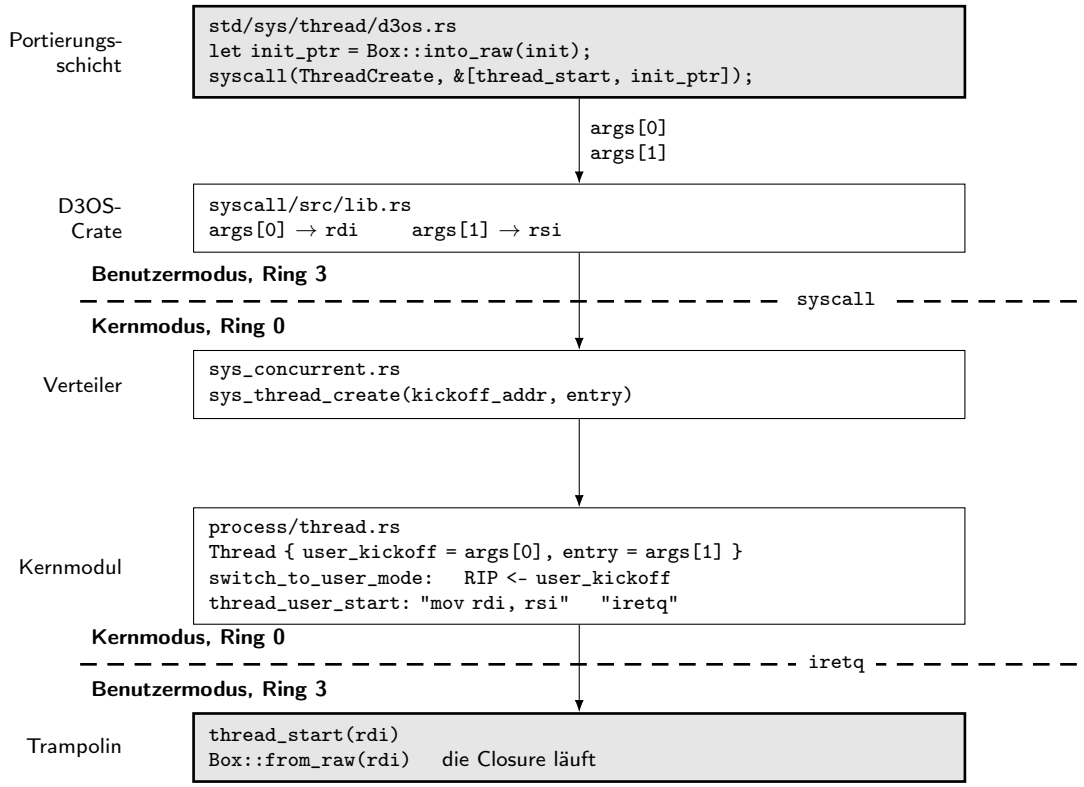


Abbildung 5.2: Der Weg des Datenzeigers von der Portierungsschicht bis in den neuen Thread. Nur `args[0]` und `args[1]` überqueren die Systemaufruf-Grenze, und `mov rdi, rsi` macht das zweite Argument zum ersten Parameter des Trampolins.

überbrückt das still, ohne dass Änderungen am Kernel oder an `concurrent` erforderlich sind. Das ist der Unterschied zu den Flags aus Abschnitt 5.3, bei denen die Quelle tatsächlich defekt war.

Die resultierende Implementierung stammt aus drei Quellen. Aus `unsupported.rs` kommen die Pflichtsignaturen und mit ihnen `DEFAULT_MIN_STACK_SIZE`, da `sys/thread/mod.rs` die Konstante exportiert. Ohne den Wert übersetzt std nicht. Benutzt wird sie trotzdem nie, denn die Funktion `sys_thread_create` nimmt keine Stapelgröße entgegen und die Funktion `new_user_thread` legt immer `MAX_USER_STACK_SIZE` von 1 GiB an, von denen anfangs eine einzige Seite mit physischem Speicher hinterlegt ist. Das Muster für Spawn und Trampolin stammt aus `hermit`, nämlich `Box::into_raw` mit `expose_provenance` auf der Hinseite und `Box::from_raw` mit `with_exposed_provenance_mut` auf der Rückseite. Aus `concurrent::thread` stammen die Rümpfe, `join` sogar zeichengleich. Das Trampolin richtet zuerst die Thread-Umgebung ein, wie `kickoff_user_thread` es für native Threads tut und wie `start.rs` es bereits für den Hauptthread macht. Es endet mit `thread::exit()`, weil es keinen Aufrufer gibt, zu dem es zurückkehren könnte. Am Ende sind es 86 Zeilen (Commit `ff3c364`).

Der erste Baulauf scheiterte trotzdem, und zwar an einer Warnung:

```
warning: direct cast of function item into an integer
--> library/std/src/sys/thread/d3os.rs:26:64
help: first cast to a pointer `as *const ()`
error: warnings are denied by `build.warnings` configuration
```

Der Lint `function_casts_as_integer` ist in Version 1.94 neu und standardmäßig auf Warnstufe eingestellt. Erst durch die letzte Zeile wird der Abbruch ausgelöst, da der Bootstrap die Standardbibliothek mit verbotenen Warnungen übersetzt. Der Vorschlag des Compilers ist zugleich die Lösung. Seitdem steht dort `thread_start as *const () as usize`. Die Adresse bleibt dieselbe, nur die Schreibweise ändert sich. Bemerkenswert ist die andere Seite. `concurrent::thread::create` verwendet weiterhin die direkte Form `kickoff_user_thread as usize` und bleibt warnungsfrei, nicht wegen eines Unterschieds im Code, sondern weil Cargo die Lints von Abhängigkeiten unterbindet. Die Datei des Ports liegt dagegen im Quellbaum von std und unterliegt der vollen Stufe. In D3OS ist dieselbe Zeile zulässig, in std hingegen führt sie zu einem Abbruch. Die Portierung in die Standardbibliothek erzwingt somit strengere Lint-Hygiene als der umgebende Betriebssystemcode.
Das Ergebnis ist in `stdlibtest` geprüft (Commit `89f0a8b`). Eine Lücke bleibt dabei offen. Eine thread-lokale Variable kann einen Destruktor tragen, der beim Ende des Threads auszuführen ist und die darin gehaltenen Ressourcen freigibt. Das Trampolin von hermit ruft diese Destruktoren vor dem Aussteigen auf, das hier entstandene nicht. Was ein beendeter Thread in thread-lokalen Variablen hält, bleibt deshalb liegen. Damit sind alle vier Module beschrieben. Was sie leisten und wo ihre Grenzen liegen, weist Kapitel 6 nach.

5.5 Herkunft des Quellcodes

Das in Abschnitt 3.1.2 beschriebene Verfahren nennt für jede Frage eine Quelle, aus der sich vier Herkunftsklassen ableiten lassen. Die erste Klasse ist die Pflicht. In der Datei `unsupported.rs` ist festgelegt, welche Typen und Funktionen überhaupt vorhanden sein müssen. Aus ihr stammen die dreiundzwanzig Stubs des fs-Backends und bei den Threads `set_name` und `available_parallelism`. Zweitens gibt es die Vorlage, also das nächstliegende Geschwister-Backend. Dies ist `solid.rs` für Dateien und `hermit.rs` für Threads. Von dem stammt auch der Wert `DEFAULT_MIN_STACK_SIZE`. Drittens gibt es die Referenzbibliothek auf der D3OS-Seite. Aus `naming/src/lib.rs` sind die Argumentlisten der Dateiaufrufe und aus `terminal` die Aufrufkonvention der Standardeingabe übernommen. Und viertens gibt es den eigenen Code, wenn kein Vorbild passt. Dazu zählen `non_negative_offset` und `to_flags` bei Dateien sowie das Backend `D3osSystem` unter dlmalloc.
Für das größte der vier Module ist jede Zeile einer dieser Klassen zugeordnet. Tabelle 5.2 zeigt das Ergebnis für `sys/fs/d3os.rs`. Die Zahlen gehören zu Commit `c98f0fa`, mit dem das Backend entstanden ist. Der spätere Commit `b86264d` hat die Kommentare überarbeitet und den Schutz gegen den leeren Puffer ergänzt, wodurch die Datei auf 257 Zeilen gewachsen ist. Die vollständige farbig markierte Fassung findet sich in Anhang B. Das Gerüst besteht somit zu rund zwei Dritteln aus einer geteilten Schablone. Die Herausforderung besteht nicht darin, den Rest dazuzuschreiben, sondern darin, die Zeilen der Vorlage zu identifizieren, die nicht übernommen werden dürfen, da sie stillschweigende

Herkunft	Zeilen	Anteil
`solid.rs`	124	64 %
eigener Code	31	16 %
`stdio/d3os.rs`	12	6 %
`vexos.rs`	10	5 %
`motor.rs`	9	5 %
`naming/src/lib.rs`	7	4 %
Summe	193	100 %

Tabelle 5.2: Herkunft der 193 zuordenbaren Zeilen in `sys/fs/d3os.rs` zum Stand von Commit `c98f0fa`. Die übrigen Zeilen der Datei sind leer

POSIX-Annahmen enthalten. Der Deskriptor aus Abschnitt 5.3 ist ein Beispiel hierfür, und im Backend steht deshalb `File(usize)` statt eines `c_int` mit reserviertem Fehlerwert. Über die vier Module hinweg sind die Klassen ungleich verteilt, was an sich schon ein Befund ist. Die vier Backends umfassen zusammen 617 Zeilen, nämlich 176 Zeilen für den Allokator, 98 Zeilen für die Standardeingabe, 257 Zeilen für Dateien und 86 Zeilen für Threads. Bei der Standardeingabe fehlt die Klasse Vorlage komplett, denn Tabelle 2.1 führt dieses Modul als einziges ohne eine solche auf. Beim Allokator liegt sie auf zwei Repositories verteilt, weil Xous seine Engine in `rust-lang/rust` anbindet und das Backend dazu aus `dlmalloc-rs` bezieht. Bei den Threads fällt die Klasse Referenzbibliothek fast aus, da `concurrent::thread::create` den Zeiger nicht tragen kann und der Aufruf deshalb roh abgesetzt wird. Wo eine Klasse fehlt, wächst eine andere. Ob das Ergebnis trägt, entscheidet der Nachweis im folgenden Kapitel.

Kapitel 6

Evaluation

6.1 Wahl des Nachweisverfahrens

Die naheliegende Antwort auf die Frage, ob ein Modul funktioniert, lautet `cargo test`. Für dieses Ziel gibt es sie nicht. Ein Testrunner meldet sein Ergebnis über den Rückgabewert des Prozesses und über dessen abgefangene Ausgabe [29], und beides erreicht unter D3OS den Host nicht.

Als Instrument bleiben deshalb Anwendungen, die im laufenden System etwas behaupten und das Ergebnis ausdrucken. Sie verwenden ausschließlich die öffentliche Schnittstelle von std, etwa `std::fs::write` oder `std::thread::spawn`, und nicht das darunterliegende Backend. Ein Test, der das Backend direkt aufruft, würde genau den Teil überspringen, um den es in dieser Arbeit geht, nämlich die Anbindung.

Eine einzige Zeile zeigt, wie viele Schritte zwischen Aufruf und Systemaufruf liegen: `std::fs::write` geht über `File::create`, das intern die Methoden `write`, `create` und `truncate` aufruft, weiter zu `OpenOptions::to_flags`, das daraus die D3OS-Bits erzeugt. Von dort geht es über `File::open` in den Systemaufruf, im Kernel durch `sys_open` und `ptr_to_string` und schließlich nach `api::open`. Insgesamt sind es sieben Stationen, von denen vier in dieser Arbeit entstanden sind: die beiden in `sys/fs/d3os.rs`, die Flag-Werte, die `sys_open` auspackt, und `api::open` selbst. Die anderen drei sind unverändertes std und unveränderter Kernel-Code. Ein Aufruf von oben läuft durch alle sieben Stationen und damit über jeden Übergang zwischen ihnen.

Was einen erfolgreichen Lauf ausmacht, ist dabei von Anwendung zu Anwendung verschieden. Alle vier führen ihre Schritte nacheinander aus und drucken jedes Ergebnis aus, und weil `x86_64_unknown_d3os.rs` das Ziel auf `PanicStrategy::Abort` festlegt, beendet ein Abbruch unterwegs den gesamten Prozess. Bei `stdheaptest` und `stdthreadtest` genügt das als Kriterium, denn dort beendet jeder fehlgeschlagene Schritt den Lauf. Erreichen sie ihre Schlusszeile `HEAP TEST PASSED` beziehungsweise `THREAD TEST PASSED`, ist unterwegs nichts fehlgeschlagen. `stdiotest` und `stdfstest` fangen dagegen jeden Fehler in einem `match` ab und drucken ihn aus. Sie laufen bis zum Ende durch, auch wenn unterwegs etwas schiefgeht, und ihr Ergebnis muss gelesen werden. Beide Bauformen haben dieselbe Grenze, die den Rest des Kapitels bestimmt. Was ein erfolgreicher Lauf belegt, hängt allein davon ab, ob benennbar ist, wann er fehlgeschlagen wäre.

6.2 Aufteilung der Testanwendung

Zunächst bestand der Nachweis aus einer einzigen Anwendung. `stdlibtest` umfasste 236 Zeilen und prüfte die vier Module nacheinander in einem Prozess, in der Reihenfolge Ein- und Ausgabe, Heap, Dateien und Threads. Möglich ist eine Aufteilung überhaupt nur deshalb, weil sich die vier Wege der Module nach Abschnitt 3.2 unterwegs nicht berühren, sodass kein Test ein Modul benötigt, das ein anderer Test benutzt. Erforderlich wird sie aus drei Gründen.

Der wichtigste Grund ist, dass die Module weiterwachsen werden. Das Dateibackend ist dafür nur das nächstliegende Beispiel. Die Methoden `mkdir` und `readdir` sind offene Arbeit und im Kernel sind die passenden Systemaufrufe bereits vorhanden. Dasselbe gilt überall sonst, denn jeder Systemaufruf, den D3OS künftig dazubekommt, ermöglicht auf der std-Seite eine weitere Funktion. Wer eine solche Funktion nachträgt, soll genau eine Testanwendung anpassen. In einer gemeinsamen Anwendung läge jede solche Ergänzung in derselben Datei wie Heap, Threads und Eingabe, sodass jeder Eingriff die anderen drei Module mitberührte.

Hinzu kommen zwei Beobachtungen aus dem Betrieb. Der Heap-Teil geht bewusst an die Speichergrenze der Maschine, denn das ist sein Zweck. Stürzt er dabei ab, werden die Teile für Dateien und Threads nicht mehr ausgeführt. Ein Defekt in einem der vorderen Teile macht alles Dahinterliegende unsichtbar. Die Eingabe stand an erster Stelle und wartet auf einen Tastendruck, da es genau darum in Abschnitt 5.2 geht. Solange niemand die Enter-Taste drückt, läuft nichts anderes.

Der Schnitt verläuft entlang der Backend-Dateien und nicht entlang der std-Schnittstelle. Der Grund liegt in der Eindeutigkeit. Ein Schnitt entlang von `std::io` hätte Ein- und Ausgabe zusammengelegt und damit zwei Backends berührt. Bei einem fehlgeschlagenen Lauf wäre nicht mehr klar gewesen, welches der beiden gemeint ist. Aufwand verursacht der Schnitt dennoch, da jede Anwendung ein eigenes `Cargo.toml`, ein eigenes `Makefile.toml` und einen Eintrag im Arbeitsbereich benötigt. Dafür gehört jetzt jede Anwendung zu genau einer Datei unter `sys`, und ein Fehlschlag zeigt ohne Zwischenschritt auf sie.

6.3 Ergebnisse und ihre Grenzen

Der Heap-Test zeigt am deutlichsten, woran ein Nachweis hängt. In der ersten Fassung lief er tausendmal durch einen Block von 64 KiB, was zusammen 62,5 MiB ergibt. Die virtuelle Maschine bekommt in `Makefile.toml` an den Zeilen 26, 61 und 98 jeweils die Angabe

Makefile.toml — TOML

```
1 "-m", "512M,slots=2,maxmem=1G",
```

also 512 MiB. Der Lauf wäre demnach auch dann erfolgreich gewesen, wenn `dealloc` überhaupt nichts zurückgewinnt, denn 62,5 MiB passen achtmal in den verfügbaren Speicher. Er hätte gar nichts belegt.

Erst mit zehntausend Zyklen ist er als Beleg gültig (Commit `c4f3251`). Das sind 625 MiB gegen 512 MiB, und die Kette dahinter ist geschlossen. Nach Abschnitt 5.1 wächst der Bump-Zeiger in Schicht 1 nur, `free` meldet dort unverändert `false` und eine Freigabe erreicht das Betriebssystem nie. Der Ausdruck `vec![0; 64 * 1024]` geht außerdem über

`alloc_zeroed` in die Methode `calloc` des Allokators `HEAP`. Dort nullt dlmalloc jeden Block selbst, denn `allocates_zeros()` liefert für dieses Backend `false`. Jeder Block wird dadurch tatsächlich beschrieben, sodass der Kernel die zugehörigen Frames auch einlagern muss. Würde `dealloc` den Speicher nicht zurückgeben, würde die Maschine unweigerlich gegen ihre Grenze laufen.
Daraus folgt das Kriterium für alles Weitere. Ein Nachweis zählt nur, wenn benennbar ist, unter welcher Bedingung er fehlgeschlagen wäre. Tabelle 6.1 stellt für jede der vier Anwendungen nebeneinander dar, was geprüft wird und woran die Anwendung am alten Stand gescheitert wäre.

Anwendung	**Was sie prüft**	**Warum sie am alten Stand fehlgeschlagen wäre**
`stdiotest`	`print!`, `println!` und ein direktes `write` auf `stdout`, dazu `read_line` und eine Leseschleife über `stdin`, die bei einer leeren Zeile endet.	Vor `ec31af0` gab `Stdin::read` bedingungslos `Ok(0)` zurück, `read_line` hätte also sofort null Bytes gemeldet statt zu blockieren. Und vor `acf9c20` kam die Zeile ohne `\n` zurück, sodass `read_line` erst nach dem zweiten Enter zurückkehrte.
`stdheaptest`	1 MiB am Stück, 10 000 Zyklen à 64 KiB, eine Anforderung mit Ausrichtung 4096, dazu 1000 gleichzeitig lebende Blöcke mit `Vec`-Wachstum und Teilfreigabe.	Der alte Stub hielt 65 536 Byte statisch vor, wies mit `if layout.align() > 8` jede stärkere Ausrichtung durch einen Nullzeiger ab und sein `dealloc` war leer.
`stdfstest`	Anlegen, zweimal schreiben, `Start`, `End`, `Current` und `stream_position`, ein Kurzlesen ab Position 7, `create_new` zweimal auf denselben Pfad, ein Nur-Lese-Öffnen, dazu `std::fs::write` und `read_to_string`.	Vor `c98f0fa` gab es kein Dateibackend, jeder Aufruf fiel auf `unsupported.rs` durch. Ohne `non_negative_offset` ginge `Current(-1)` als $2^{64} - 1$ in den Kernel. Und vor `7c5964a` wies `api::open` schon einfaches `create` auf eine vorhandene Datei mit `EEXIST` ab.
`stdthreadtest`	`spawn` mit gefangenem Wert und Rückgabe über `join`, vier Threads auf einem gemeinsamen Zähler, dazu `yield_now`, `sleep` und `available_parallelism`.	Vor `ff3c364` gab es kein Thread-Backend. Ein `join`, das nicht wartet, ergäbe weniger als 4000, und ein `spawn`, das den gefangenen Wert verliert, ergäbe nicht 123.

Tabelle 6.1: Was die vier Testanwendungen prüfen und unter welcher Bedingung sie fehlgeschlagen wären

Eine Voraussetzung dieser Nachweise liegt außerhalb der Anwendungen. Das tmpfs aus Abschnitt 3.2 wird bei jedem Systemstart neu angelegt, und darauf beruht die Wiederholbarkeit von `stdfstest`. Das zweimalige `create_new` auf denselben Pfad setzt voraus, dass dieser Pfad am Anfang des Laufs frei ist. Überdauerte die Datei den Neustart, schlüge bereits der erste der beiden Aufrufe fehl.
Nicht jede Eigenschaft ist über eine Zusicherung erreichbar. Bei der Standardeingabe liegt der kritische Teil nicht im Rückgabewert, sondern im Editierpfad des Terminals, den eine Anwendung über `std::io::stdin` nur mittelbar ansteuert. Geprüft wurde dieser

Teil deshalb nach Äquivalenzklassen, aufgebaut entlang der Dimensionen Inhalt, Länge, Editieren und Zeitpunkt der Eingabe (Commit `6de8583`). Die Grenzwerte für die Länge folgen aus `STDIN_BUF_SIZE = 128`, also 127, 128 und 140 Zeichen. Tabelle 6.2 stellt jeder Klasse die Vorhersage und die Beobachtung gegenüber.

#	**Eingabe**	**Vorhersage**	**Beobachtung**	**trifft zu**
1	`hallo`	6 Byte, `hallo\n`	6 Byte, `hallo\n`	ja
2	`abc` mit einem Leerzeichen	kein Trimmen	5 Byte, `abc \n`, das Leerzeichen bleibt erhalten	ja
3	dreimal Esc, dreimal Tab	Esc wird verschluckt	4 Byte, `\t\t\t\n`, alle drei Escapes verworfen, alle drei Tabs durchgereicht	ja
4	127 Zeichen	der Umbruch passt noch in die 128	128 Byte, die 127 Zeichen und der abschließende `\n`	ja
5	128 Zeichen	Puffer voll, der Umbruch fällt heraus	128 Byte, die 128 Zeichen ohne abschließendes `\n`	ja
6	140 Zeichen	der Kernel liefert 128, der Rest bleibt in der Deque	128 Byte, die ersten 128 Zeichen, die Zeichen 129 bis 140 folgen beim nächsten Lesen	ja
7	nur Enter	1 Byte, `\n`	1 Byte, `\n`, die Leseschleife bricht ab	ja
8	`Umlaute z.B. ä`	Echo und Rückgabe wie bei einem ASCII-Zeichen	kein Echo, danach nimmt das Terminal keine Eingabe mehr an	nein

Tabelle 6.2: Die geprüften Äquivalenzklassen der Standardeingabe mit Vorhersage und Beobachtung

Sieben der acht Fälle verliefen wie vorhergesagt, der achte nicht. Genau dadurch ist der Defekt aufgefallen, und Abschnitt 5.2 führt die Ursache in `Canonical::cursor_pos` sowie den Weg zu ihr aus. Für die Bewertung des Nachweises sind daran zwei Punkte wesentlich. Gefunden hat den Defekt keine Zusicherung einer Testanwendung, sondern die Klasse der Mehrbyte-Zeichen, die vorab im Entwurf der Prüfung stand. Und behoben ist er nach Abschnitt 5.2 nicht, sodass die Dimension Inhalt nur für ASCII-Eingaben abgedeckt ist.
Über diese benannten Grenzen hinaus belegt das Kapitel vier Dinge nicht, und zwar aus jeweils anderen Gründen.
So ist der Yield-Zweig des Allokator-Locks beispielsweise nie ausgeführt worden. Unmöglich zu treffen ist er nicht, da `LOCKED` in `sys/pal/d3os/alloc.rs` ein prozessweiter `static` ist. Die vier Threads aus `stdthreadtest` teilen sich also dasselbe Lock, und der Scheduler verdrängt über den APIC-Timer. Nur entsteht der Wettstreit in keinem der Tests. Der Heap-Test ist einfädig und die Kindthreads zählen lediglich einen `AtomicUsize` hoch, was keine Speicheranforderung auslöst. Auch ein Test, dessen Threads alle anfordern, würde den Zweig nicht zuverlässig treffen, da der QEMU-Aufruf kein `-smp` enthält. Die Maschine hat einen Kern und zwei Threads fordern nie im selben Augenblick an. Der Zweig greift also nur, wenn ein Timer-Interrupt genau zwischen die beiden `swap`-Aufrufe fällt. Ein erfolgreicher Lauf verrät ohnehin nicht, ob das passiert ist. Das würde nur ein Zähler im Lock selbst zeigen, und dieser müsste dauerhaft im ausgelieferten Code stehen.
Ein Panic aus einem Kindthread kommt über `join` nicht zurück. Das ist keine Lücke im

Test, sondern eine Festlegung des Ziels. `x86_64_unknown_d3os.rs` setzt `panic_strategy = PanicStrategy::Abort`. Ein Panic wickelt den Stapel also nicht ab, sondern bricht über `core::intrinsics::abort` den gesamten Prozess ab. Der `Err`-Zweig von `join` ist auf diesem Ziel unerreichbar.
Der Fehlerzweig des Eingabe-Backends ist ebenfalls unerreichbar. Dies ist allerdings durch die Vorlage bedingt. `is_ebadf` in `sys/stdio/d3os.rs` liefert stets `true`, unverändert aus `unsupported.rs` übernommen, und `solid.rs` und `xous.rs` tragen dieselbe Zeile. `handle_ebadf` in `io/stdio.rs` macht daraus beim Lesen für jeden Fehler ein `Ok(0)`. Über `std::io::stdin()` sähe ein Lesefehler somit wie ein Dateiende aus. Dieser Zweig ließe sich nur prüfen, indem der Test das Backend direkt aufruft. Genau das schließt jedoch Abschnitt 6.1 aus.
Zudem ist `stdthreadtest` innerhalb einer Sitzung nicht beliebig wiederholbar. Ein zweiter Start im selben Boot bleibt manchmal in `handle_page_fault` hängen, denn der Handler läuft mit abgeschalteten Interrupts und nimmt über `map_partial_vma` den Schreib-Lock der Seitentabelle in `memory/pages.rs`. Beide Stellen liegen im Kernel. Das Backend dieser Arbeit setzt hier nur `ThreadCreate` ab und rührt keine Seitentabelle an. Für den Nachweis bedeutet dies, dass ein Lauf pro Boot das Belegbare ist.
Ein Nachweis ist damit nur so viel wert wie die Liste dessen, was er nicht abdeckt. Ein Boot kann nur zeigen, dass die Anwendung läuft, und nicht, dass sie korrekt ist. Damit ist die Frage aufgeworfen, woran der Aufwand einer solchen Portierung tatsächlich hängt. Der folgende Abschnitt beantwortet sie.

6.4 Diskussion

Die vier Backends aus Kapitel 5 umfassen zusammen 617 Zeilen, die Abschnitt 5.5 nach Herkunft aufschlüsselt. Auf den ersten Blick scheint das für vier Subsysteme eines eigenen Betriebssystem-Ziels wenig zu sein. Der Aufwand steckt jedoch nicht im Schreiben dieser Zeilen, sondern in zwei Entscheidungen davor: Erstens, welche Vorlage überhaupt passt. Zweitens, welche ihrer Zeilen übernommen werden und welche nicht.
Beides lässt sich am Dateibackend zeigen. Die Auswahl unter sechs Backends führt Abschnitt 5.3 vor, die zeilenweise Herkunft des Ergebnisses Abschnitt 5.5. Der Befund liegt dazwischen. Ein ausgeschiedenes Backend kann ein Verfahren beisteuern, und das gewählte liefert nicht jede Zeile. Aus `vexos.rs` stammt die Einbindung der unimplementierten Oberfläche, obwohl die Datei als Vorlage ausgeschieden ist. Aus `solid.rs` stammt das Gerüst, nicht jedoch sein Deskriptor, dessen reservierter Fehlerwert eine Konvention voraussetzt, die D3OS nicht hat. Über die Herkunft entscheidet somit nicht die Wahl der Vorlage, sondern jede Zeile für sich.
Das eigentlich Schwierige liegt eine Stufe tiefer. Wer eine Vorlage übernimmt, übernimmt auch ihre Lücken. Dass der Fehler bei Länge null aus Abschnitt 5.3 so lange unentdeckt blieb, liegt an der Vorlage selbst. Sowohl `solid.rs` als auch `motor.rs` vertragen einen leeren Puffer, weshalb dort keine Prüfung steht, die man hätte mitkopieren können. Bei `solid.rs` kommt hinzu, dass `read_buf` nativ implementiert ist und einen leeren Slice gar nicht erst weiterreicht. Die Zeile `if buffer.is_empty() { return Ok(0); }`, die heute am Anfang von `read` und `write` steht, hat in keiner der beiden Vorlagen ein Vorbild. Struktur übernommen bedeutet in diesem Fall also, eine Vorsichtsmaßnahme zu übernehmen, die es nie gab.
Ein strukturelles Risiko bleibt bestehen. Der Port hängt von `runtime`, `concurrent` und

`syscall` aus einem fremden Repository ab und alle drei fordern `compiler_builtins` als `version = "0.1"` an. Diese lose Angabe lässt jede Fassung von 0.1.0 bis unter 0.2.0 zu. Nur deshalb ließ sich der Fehler aus Etappe B mit einem `[patch.crates-io]`-Eintrag allein im Fork beheben. Stünde dort eine feste Angabe wie `= "0.1.160"`, würde die im Quellbaum liegende Fassung 0.1.0 diese Anforderung nicht erfüllen. Der Patch griffe nicht und die Korrektur wäre nur über beide Repositories möglich gewesen. Dass dieser Fall ausblieb, ist kein Ergebnis des Entwurfs.

Die vier Anwendungen liefen auf zwei Rechnern, dem des Verfassers und dem des Betreuers. Allein dieser eine zusätzliche Rechner hat etwas zum Vorschein gebracht. Der in Abschnitt 6.3 beschriebene Seitenfehler-Deadlock ist auf dem ersten Rechner in keinem Lauf aufgetreten. `stdthreadtest` ließ sich dort mehrfach hintereinander starten, ohne dass etwas hängen blieb. Aufgefallen ist er erst, als der Betreuer denselben Stand ausgeführt hat. Welche Befunde ein dritter Rechner liefern würde, ist offen.

Nicht alles davon fällt allerdings in den Aufgabenbereich dieser Arbeit. Wo ein Defekt an der Quelle lag, ist er nach dem Kriterium aus Abschnitt 3.1.3 dort behoben worden, etwa bei den Flag-Werten und dem fehlenden Zeilenumbruch. Der Seitenfehler-Deadlock liegt dagegen jenseits der Systemaufruf-Grenze im Seitenfehler-Handler des Kernels und die Portierungsschicht setzt dort lediglich `ThreadCreate` ab. Aufgabe dieser Arbeit war die Erweiterung der Standardbibliothek und nicht die Härtung von D3OS.

Kapitel 7

Fazit und Ausblick

7.1 Kritische Reflexion des Vorgehens

Der erste Fehler war ein falsches Modell. Am Anfang dieser Arbeit stand die Annahme, ein Versionswechsel sei ein Zeilenwechsel in `rust-toolchain.toml`. Genau diese Datei bleibt unverändert, weil die Kopplung über den Namen der Toolchain läuft und nicht über eine Version. Aus dieser Fehlannahme folgt Kapitel 4, nämlich drei Etappen und 27 Anpassungsklassen statt einer geänderten Zeile.

Drei weitere Fehler sind handwerklicher Art. Der Klon mit `--depth=1` statt `--shallow-since` ließ keinen gemeinsamen Vorfahren übrig und stellte den Rebase vor 203 832 Commits. In Etappe A blieben beide Seiten der Zielliste erhalten, in der Annahme, überzählige Einträge in einer Liste fehlender Ziele könnten nicht schaden, was der erste `./x build` nach 57 Sekunden widerlegt hat. Und in `sys/stdio/mod.rs` ist eine Konfliktauflösung nie gespeichert worden, sodass die Marker unbemerkt über dreizehn Commits hinweg bestehen blieben und in Etappe B 37 Fehler aus zwei Wurzelursachen erzeugt haben. Daraus ist der Marker-Scan entstanden.

Ein Fehler betrifft keinen Handgriff, sondern eine zu weit gefasste Regel. Beim verschluckten Zeilenende lag der Fix zuerst in der Portierungsschicht, weil fremder Code als unantastbar galt, solange seine Absicht nicht bekannt ist. Der Grundsatz stimmt, nur lag hier keine Absicht vor, und genau das ließ sich vorher feststellen. Wer nicht danach sucht, verlagert jeden Defekt eine Ebene nach unten. Abschnitt 5.2 stellt den Fall dar.

Der Fehler mit dem geringsten Aufwand hatte die größten Folgen. Abschnitt 2.2 hält fest, dass nach jedem `./x build` ein `cargo make clean` folgen muss, weil D3OS die neue Standardbibliothek sonst nicht bis in die Anwendung durchreicht. Der Hinweis stammt von Baranowski [3], steht in dieser Arbeit an derselben Stelle und ist in den ersten Wochen dennoch mehrfach übergangen worden. Die Folge ist keine Fehlermeldung, sondern ein Programm, das vollständig durchläuft und den alten Stand zeigt. Zweimal führte das zu einer Fehlersuche in einer Änderung, die längst richtig war. Ein Fehler, der wie ein Erfolg aussieht, kostet mehr als einer, der abbricht.

Bei `compiler_builtins` waren drei Hypothesen zu falsifizieren, ehe die Ursache feststand. Abschnitt 4.3 führt den Weg vor. Zwei Sätze bleiben davon übrig. Bleibt ein Fehler nach dem Verwerfen des abgeleiteten Artefakts bestehen, liegt die Ursache im erzeugenden. Und auffällige Versionsnummern können ablenken, denn die Meldung nannte 0.1.0 gegen 0.1.160 und legte eine Inkompatibilität nahe, obwohl das Problem die Anzahl war, nämlich zwei Pakete statt eines.

Drei Fehler liegen nicht im Code, sondern in der Arbeitsumgebung. Die Wurzelpartition lief voll, weil nach dem Rebase mit `git submodule update --init --recursive` der gesamte Submodulsatz zurückgeholt wurde, ohne vorher seinen Umfang zu prüfen. Der anschließende Versuch, den LLVM-Bau mit `download-ci-llvm = true` zu überspringen, funktioniert für einen Zielcommit von vor mehreren Monaten nicht mehr, weil Upstream alte CI-Artefakte löscht. Und der lokale Bau brach ab, weil Bootstrap seine Parallelität nach der Kernzahl wählt und 32 Übersetzungseinheiten zu je rund 1,5 GB nicht in 16 GB Hauptspeicher passen. Abschnitt 4.2 führt alle drei vor. Alle drei gehen auf dieselbe Ursache zurück. Die Vorgaben der Vorlage und die Voreinstellungen von Upstream setzen eine andere Maschine voraus als die tatsächlich verwendete. Vor der Übernahme einer Bauumgebung ist somit zu prüfen, für welche Maschine ihre Vorgaben gedacht waren.
Der größte Planungsfehler war die Impact-Analyse aus Abschnitt 4.1. Sie hat die falsche Seite gezählt, und die Abrechnung dazu steht in Abschnitt 4.5. Tragfähig waren am Ende Vorhersagen aus dem Verstehen eines Musters und nicht aus einer Zählung.
Drei Fehler liegen außerhalb des Programmierens. Der erste betrifft das Sprachliche. Für die sprachliche Überarbeitung kam DeepL Write zum Einsatz, und das Werkzeug formuliert nicht nur um, sondern greift in die Aussage ein. Mehrfach ist aus einem Satz, der eine Beobachtung festhielt, einer geworden, der etwas Stärkeres behauptete, und einmal wäre eine solche Umformulierung beinahe übernommen worden. Seitdem wird jeder Vorschlag gegen den Befund geprüft und nicht gegen das Sprachgefühl.
Der zweite Fehler betrifft den Umgang mit Vorschlägen. Claude Code lag an mehreren Stellen daneben, und der Fehler liegt nicht darin, dass ein Werkzeug irrt, sondern darin, dass zwei Vorschläge ungeprüft übernommen wurden. Fünf Beispiele stehen stellvertretend für weitere Fälle dieser Art:

- `read_line` sollte gar nicht geprüft werden, weil der rohe `read` auf Anhieb lief. Kein Rust-Programm liest so, und die Prüfung hat gezeigt, dass jede Eingabe zwei Mal Enter braucht (Abschnitt 5.2).
- Der erste Heap-Test konnte nicht fehlschlagen. Tausend Durchläufe zu 64 KiB sind 62,5 MiB, und die virtuelle Maschine hat 512 MiB (Abschnitt 6.3).
- `wasi.rs` galt als naheliegende Vorlage für das Dateibackend. Die Datei gibt es nicht mehr, weil WASI inzwischen in den `unix`-Zweig fällt (Abschnitt 2.3).
- `rustup toolchain link` sollte eine Kopie oder einen Symlink der frisch gebauten Stufe anlegen. Der Befehl schreibt nur einen Pfadeintrag und prüft nicht, ob dort überhaupt etwas liegt (Abschnitt 2.2).
- Die Bauanleitung dieses Projekts hat das Werkzeug nie übernommen. Vorgeschlagen wurde jedes Mal der Standardweg und nicht die in Abschnitt 2.2 beschriebene Kette.
- [. . .]

Alle fünf klingen plausibel und halten dem ersten Blick in den Quelltext oder in das laufende System nicht stand. Die beiden ungeprüft übernommenen Vorschläge haben je einen halben Tag gekostet.
Der dritte Fehler betrifft die eigene Dokumentation. Die verworfenen Ideen sind fast alle festgehalten worden oder ließen sich im Nachhinein rekonstruieren, der vollgelaufene

Speicher ist aus der Erinnerung rekonstruiert und die Zahlen dazu sind nachträglich aus der Historie nachgezogen worden. Beides hat funktioniert und beides hat Zeit gekostet. Eine von Anfang an geführte Aufzeichnung des Verworfenen und seiner Gründe hätte das Schreiben beschleunigt, denn eine verworfene Idee erklärt eine Entscheidung besser als die Entscheidung selbst.

7.2 Zusammenfassung und Ausblick

Diese Arbeit hat den übernommenen Rust-Port von rustc 1.90 auf 1.94 nachgeführt und vier Module ergänzt, nämlich die dynamische Speicherverwaltung, die Standardeingabe, die Datei-Ein- und -Ausgabe und die Threads. Am Ende laufen Anwendungen für D3OS gegen eine Standardbibliothek, die Speicher anfordert und wieder freigibt, eine Zeile von der Tastatur liest, Dateien anlegt, schreibt und liest sowie Threads startet und wieder einsammelt.

Damit sind beide Leitfragen zu beantworten. Die erste fragt, woran die Aktualisierung einer betriebssystemspezifischen Standardbibliothek auf eine neuere Compiler-Version scheitert. Sie scheitert nicht an der Versionsdistanz, sondern an der Restrukturierungstiefe in denjenigen Subsystemen, die der Port berührt. Abschnitt 4.5 zeigt das an drei Etappen, in denen ausgerechnet der größte Sprung über zwei Minor-Versionen die wenigsten Konflikte verursacht hat.

Die zweite Leitfrage betrifft den Umfang, in dem sich bestehende Plattform-Backends als Vorlage übernehmen lassen und an welchen Stellen eine eigenständige Implementierung erforderlich ist. Übernehmen lässt sich der größere Teil, denn rund zwei Drittel des umfangreichsten Backends stammen aus einer geteilten Schablone. Die Arbeit einer Portierung liegt deshalb weniger im Schreiben des fehlenden Codes als im Erkennen derjenigen Zeilen einer Vorlage, die man nicht übernehmen darf. Die Abschnitte 5.5 und 6.4 zeigen beides an derselben Datei.

Was diese Befunde tragen, hat Grenzen. Hinter ihnen stehen ein Bearbeiter, eine Maschine, ein Ziel-Betriebssystem und die drei Etappen aus Kapitel 4 als Messpunkte. Der Zusammenhang zwischen Restrukturierungstiefe und Aufwand ist damit an drei Etappen belegt und nicht statistisch gesichert.

Für eine nächste Etappe stehen drei prüfbare Vorhersagen bereit. Issue #116344 kündigt an, dass die Kombination aus `-sse` und `+soft-float` ein harter Fehler wird. Der Lint `function_casts_as_integer` trifft im D3OS-Quellcode drei Stellen, nämlich `process/thread.rs` in Zeile 298, `syscall/syscall_dispatcher.rs` in Zeile 88 und `concurrent/src/thread.rs` in Zeile 101. Und mit der Stabilisierung von `cfg_select` in 1.95 entfällt ein Feature-Gate, womit die Konflikte in `pal/mod.rs` ausbleiben sollten.

Offen bleibt zuerst die Zeit, denn `Instant::now()` und `SystemTime::now()` brechen weiterhin ab. Danach kommt der Zufall, denn `fill_bytes` in `sys/random/d3os.rs` ist ein `todo!()`. Das Trampolin führt am Ende keine TLS-Destruktoren aus. `append` zählt in `to_flags` nur als Schreibzugriff, sodass ein Anhängen still überschreibt. Und alle vom Kernel gemeldeten Fehler des Dateibackends fallen auf `ErrorKind::Other` zusammen, obwohl der Kernel längst differenziert.

Beim Dateibackend fehlen außerdem die Verzeichnisse. `MkDir`, `Touch` und `Readdir` gibt es als Systemaufruf, im Backend stehen `mkdir` und `readdir` aber weiterhin bei `unsupported()`. Das Netzwerk fehlt ganz. D3OS bietet mit `SockOpen`, `SockBind`, `SockAccept`, `SockConnect`,

`SockSend` und `SockReceive` sechs Aufrufe an, und weil `sys/net/connection/mod.rs` einen Auffangzweig hat, landet D3OS dort stillschweigend bei `unsupported`.
Zuletzt steht die Zusammenführung mit dem Zweig `development` aus. Der Port hängt an `nightly-2026-01-01` und D3OS hat seine eigene Toolchain seitdem dreimal weitergezogen, zuletzt auf `nightly-2026-07-09`. Die Zusammenführung ist damit kein Merge, sondern die nächste Anwendung derselben Kette aus Abschnitt 3.1.1.
Übernommen wurde ein Port, der Text ausgeben konnte. Weitergegeben wird einer, der Speicher verwaltet, eine Zeile von der Tastatur liest, Dateien führt und Threads startet. Der nächste Bearbeiter beginnt an derselben Stelle, nur vier Minor-Versionen weiter, und erbt mit der Prüfkette aus Kapitel 6 etwas, das in dieser Arbeit erst entstehen musste, nämlich die Möglichkeit, an jedem Punkt zu sehen, ob sein Stand noch trägt.

Anhang

A Commit-Chronologie

Die Tabelle führt alle eigenen Commits in zeitlicher Reihenfolge auf. Die Spalte Teil unterscheidet den Fork der Rust-Toolchain (F) von der D3OS-Seite (D). Die Hashes beziehen sich auf den Zweig `master` des Abgabe-Repositorys, in dem beide Historien zusammengeführt sind. Die Commit-Nachrichten sind unverändert übernommen und daher englisch.

Tabelle A.1: Die 25 eigenen Commits in zeitlicher Reihenfolge.

Commit	Datum	Teil	Nachricht
`10aa9c1`	2026-05-28	F	fix: remove stabilized loongarch32 targets, bump change-id to 145131
`f167f67`	2026-05-28	F	fix: re-sync d3os PAL thread.rs and time.rs with upstream unsupported
`b9044aa`	2026-05-28	F	fix: added jobs = 4
`6eb8913`	2026-05-28	D	fix: target spec fields target-pointer-width and target-c-int-width must be numbers, not strings (1.91 format)
`26f29b9`	2026-05-28	D	fix: target spec field types for 1.91 (target-c-int-width as number)
`f8ca6e7`	2026-06-01	F	upgrade toolchain 2025-08-18 → 2025-10-20
`b62c852`	2026-06-01	D	fix: target-pointer-width as u16 for rustc 1.92 target-spec schema
`1fffe61`	2026-06-04	F	upgrade toolchain 2025-08-18 → 2026-01-01
`9b52aa9`	2026-06-04	D	upgrade toolchain 2025-08-18 → 2026-01-01
`069a1e3`	2026-07-08	F	std/d3os: real heap via dlmalloc added
`d60c470`	2026-07-08	D	stdlibtest: verify std heap (large alloc, dealloc reuse, alignment)
`ec31af0`	2026-07-21	F	Implement std::io::Stdin:read for D3OS
`acf9c20`	2026-07-21	D	submit appends \n & doc comment updated in read.rs
`6de8583`	2026-07-21	D	add interactive stdin read / read_line test
`e4f64fd`	2026-07-21	D	drop the manually appended '\n' (would now be doubled)
`c98f0fa`	2026-08-02	F	std: add d3os fs backend (open/read/write/seek/close)
`322e47f`	2026-08-02	D	naming: make OpenOptions flags distinct powers of two

Fortsetzung auf der nächsten Seite

Tabelle A.1, Fortsetzung

Commit	Datum	Teil	Nachricht
`7c5964a`	2026-08-02	D	naming: open() create/create_new fix (O_EXCL semantics)
`a1b0700`	2026-08-02	D	stdlibtest: add std::fs test (round-trip, seek variants, create_new)
`ff3c364`	2026-08-12	F	std: add thread backend for d3os
`89f0a8b`	2026-08-12	D	stdlibtest: add std::thread test
`b86264d`	2026-08-17	F	revise comments in d3os backends (+ zero-length fs read/write guard)
`c4f3251`	2026-08-17	D	stdlibtest: raise alloc/free loop to 10000
`e71b9a9`	2026-08-17	D	stdlibtest: split into stdiotest, stdheaptest, stdfstest, stdthreadtest
`95feda3`	2026-08-29	D	keytest: strip trailing newline before comparing (canonical read now returns it)

B Code-Herkunft farblich markiert

Diese Anlage zeigt `library/std/src/sys/fs/d3os.rs` im Stand von Commit `b86264d` vollständig. Jede Zeile ist einer der vier Herkunftsklassen aus Abschnitt 5.5 zugeordnet. Die Zuordnung entsteht durch Vergleich in fester Reihenfolge. Zuerst gegen `unsupported.rs`, danach gegen die Vorlagen, danach gegen die Referenzbibliothek. Was übrig bleibt, ist eigener Code. Die Kommentare bleiben unmarkiert, da sie selbst verfasst sind und die Herkunft beschreiben, statt selbst eine zu haben.

	Klasse	Quelle	Zeilen	Anteil
P	Pflicht	`sys/fs/unsupported.rs`	52	37,7 %
V	Vorlage	`solid.rs`, `vexos.rs`, `motor.rs`	44	31,9 %
R	Referenzbibliothek	`naming/src/lib.rs`, `naming/src/shared_types.rs`	12	8,7 %
E	eigener Code	keine	30	21,7 %
		zuordenbar	**138**	100 %

Tabelle B.1: Herkunft der 138 zuordenbaren Zeilen in `sys/fs/d3os.rs`. Nicht markiert sind leere Zeilen, Zeilen mit nur einer schließenden Klammer und die Kommentare.

Die Reihenfolge des Vergleichs entscheidet über Zweifelsfälle. Eine Signatur, die sowohl in `unsupported.rs` als auch in `solid.rs` steht, zählt als Pflicht, denn ohne sie übersetzt die Standardbibliothek nicht. Als Vorlage zählt erst, was über die Pflicht hinausgeht. Als Referenzbibliothek zählen die Werte und die Argumentreihenfolgen, die aus `naming` abgelesen sind.

```
1  // The fd-based structure is modelled on solid.rs; the un-implemented surface
2  // is reused from unsupported.rs (same trick as vexos.rs).
3  // Implemented against the D3OS naming service: open, read, write, seek, close.
4
5  // Support for mkdir and readdir is left as future work. Most of the remaining stubs
6  // cannot be implemented at all yet: D3OS currently has no syscall for unlink, rename, rmdir,
7  // stat, chmod, utimes, link or symlink.
8
V9 use crate::ffi::CString;
```

```
P10 use crate::fmt;
P11 use crate::fs::TryLockError;
P12 use crate::io::{self, BorrowedCursor, IoSlice, IoSliceMut, SeekFrom};
P13 use crate::path::Path;
V14 use crate::sys::{unsupported, unsupported_err};
15
E16 use syscall::*; // syscall from D3OS
17
18  // Reuse the whole un-implemented surface (types + free functions) from
19  // unsupported.rs; we only implement File and OpenOptions ourselves.
E20 #[allow(dead_code)]
V21 #[path = "unsupported.rs"]
V22 mod unsupported_fs;
V23 pub use unsupported_fs::{
P24     DirBuilder, DirEntry, FileAttr, FilePermissions, FileType, FileTimes, ReadDir, canonicalize,
P25     copy, exists, link, lstat, readdir, readlink, remove_dir_all, rename, rmdir, set_perm,
P26     set_times, set_times_nofollow, stat, symlink, unlink,
27  };
28
29  // D3OS open flags, mirroring the OpenOptions bitflags in naming/shared_types.rs.
30  // Now real powers of two so OR-combining them is safe.
R31 const D3OS_READONLY: usize = 1;
R32 const D3OS_READWRITE: usize = 2;
R33 const D3OS_CREATE: usize = 4;
R34 const D3OS_EXCLUSIVE: usize = 8;
35
36  // Seek origins (SeekOrigin enum also in naming/shared_types.rs):
R37 const D3OS_SEEK_START:   usize = 1;
R38 const D3OS_SEEK_END:     usize = 2;
R39 const D3OS_SEEK_CURRENT: usize = 3;
40
41  // Unlike solid (which niches a -1 error sentinel), D3OS returns
42  // Result<usize, Errno>, so a plain usize is sufficient.
E43 pub struct File(usize);
44
P45 #[derive(Clone, Debug)]
P46 pub struct OpenOptions {
V47     read: bool,
V48     write: bool,
V49     append: bool,
V50     truncate: bool,
V51     create: bool,
V52     create_new: bool,
53  }
54  // Everything in the following impl is 1:1 from solid.rs except to_flags; solid's
55  // system-specific custom_flags field was dropped, since D3OS has no OpenOptionsExt.
56  // `truncate` and the append semantics have no D3OS equivalent yet (`append` only
57  // counts as write access).
P58 impl OpenOptions {
P59     pub fn new() -> OpenOptions {
V60         OpenOptions {
V61             read: false,
V62             write: false,
V63             append: false,
V64             truncate: false,
V65             create: false,
V66             create_new: false,
67          }
68      }
69
P70     pub fn read(&mut self, read: bool) {
V71         self.read = read;
72      }
P73     pub fn write(&mut self, write: bool) {
V74         self.write = write;
```

```
 75      }
P76      pub fn append(&mut self, append: bool) {
V77          self.append = append;
 78      }
P79      pub fn truncate(&mut self, truncate: bool) {
V80          self.truncate = truncate;
 81      }
P82      pub fn create(&mut self, create: bool) {
V83          self.create = create;
 84      }
P85      pub fn create_new(&mut self, create_new: bool) {
V86          self.create_new = create_new;
 87      }
 88      // solid.rs splits this into get_access_mode and get_creation_mode, mapping the two
 89      // POSIX flag groups and rejecting invalid flag combinations on the way. That validation
 90      // is dropped here (an empty OpenOptions silently becomes READONLY), and a single mapping
 91      // method is enough; what D3OS does check happens in the kernel instead (EINVAL for
 92      // DIRECTORY|CREATE, EEXIST for CREATE|EXCLUSIVE).
 93      // Access bits are passed for forward compatibility even though the kernel/tmpfs
 94      // does not enforce them yet (`_options` in read/write (tmpfs), and similar for open_objects).
E95      fn to_flags(&self) -> usize {
E96          let mut flags = if self.write || self.append { D3OS_READWRITE }
E97              else { D3OS_READONLY };
E98          if self.create || self.create_new { flags |= D3OS_CREATE; }
 99          // create_new means "must not exist yet", which is what EXCLUSIVE adds.
E100         if self.create_new { flags |= D3OS_EXCLUSIVE; }
E101         flags
102      }
103  }
104
P105 impl File {
P106     pub fn open(path: &Path, opts: &OpenOptions) -> io::Result<File> {
107          // D3OS expects a null-terminated utf-8 path; the kernel does from_utf8 on it.
V108         let path =
V109             path.to_str().ok_or_else(|| io::Error::new(io::ErrorKind::InvalidInput,
V110                 "path is not valid utf-8"))?;
E111         let c_path = CString::new(path)
E112             .map_err(|_| io::Error::new(io::ErrorKind::InvalidInput, "path contains a null byte"))?;
113
114          // Arguments match naming/src/lib.rs (path pointer, flags).
R115         match syscall(SystemCall::Open, &[c_path.as_ptr() as usize, opts.to_flags()]) {
E116             Ok(file_descriptor) => Ok(File(file_descriptor)),
E117             Err(_) => Err(io::Error::new(io::ErrorKind::Other, "open failed")),
118          }
119      }
120
121      // read and write mirror naming/src/lib.rs as well: (handle, buffer, length).
P122     pub fn read(&self, buffer: &mut [u8]) -> io::Result<usize> {
123          // D3OS rejects zero-length reads with EINVAL, but io::Read requires Ok(0).
E124         if buffer.is_empty() { return Ok(0); }
R125         match syscall(SystemCall::Read, &[self.0, buffer.as_mut_ptr() as usize, buffer.len()]) {
E126             Ok(bytes_read) => Ok(bytes_read),
E127             Err(_) => Err(io::Error::new(io::ErrorKind::Other, "read failed")),
128          }
129      }
130
P131     pub fn write(&self, buffer: &[u8]) -> io::Result<usize> {
132          // Same as in read: a zero-length write is not an error, it writes nothing.
E133         if buffer.is_empty() { return Ok(0); }
R134         match syscall(SystemCall::Write, &[self.0, buffer.as_ptr() as usize, buffer.len()]) {
E135             Ok(bytes_written) => Ok(bytes_written),
E136             Err(_) => Err(io::Error::new(io::ErrorKind::Other, "write failed")),
137          }
138      }
139
P140     pub fn seek(&self, pos: SeekFrom) -> io::Result<u64> {
```

```
141          // D3OS SeekOrigin: Start=1, End=2, Current=3; the offset is a usize,
142          // so negative offsets (End/Current) cannot be expressed yet. The kernel adds the offset.
V 143        let (origin, offset) = match pos {
E 144            SeekFrom::Start(offset)   => (D3OS_SEEK_START,   offset as usize),
E 145            SeekFrom::End(offset)     => (D3OS_SEEK_END,     non_negative_offset(offset)?),
E 146            SeekFrom::Current(offset) => (D3OS_SEEK_CURRENT, non_negative_offset(offset)?),
V 147        };
148          // Unlike SOLID_FS_Lseek, which forces solid.rs to call tell() afterwards,
149          // the D3OS syscall returns the new position directly.
R 150        match syscall(SystemCall::Seek, &[self.0, offset, origin]) {
E 151            Ok(new_position) => Ok(new_position as u64),
E 152            Err(_) => Err(io::Error::new(io::ErrorKind::Other, "seek failed")),
153          }
154      }
155
156      // Mirroring sys/fs/motor.rs: returns the current stream position via a zero-offset seek.
P 157    pub fn tell(&self) -> io::Result<u64> {
V 158        self.seek(SeekFrom::Current(0))
159      }
160
161      // Writes go straight to the kernel, so there is nothing buffered on the std
162      // side; flush and the two sync variants have nothing left to do.
P 163    pub fn flush(&self) -> io::Result<()> {
V 164        Ok(())
165      }
166
P 167    pub fn fsync(&self) -> io::Result<()> {
V 168        self.flush()
169      }
170
P 171    pub fn datasync(&self) -> io::Result<()> {
V 172        self.flush()
173      }
174
175      // D3OS has no scatter/gather syscalls, so is_read_vectored / is_write_vectored
176      // report false and the default helpers just forward the first non-empty buffer
177      // to the single-buffer path (read_buf as in motor.rs, the vectored pair as in solid.rs).
P 178    pub fn read_vectored(&self, bufs: &mut [IoSliceMut<'_>]) -> io::Result<usize> {
V 179        crate::io::default_read_vectored(|buffer| self.read(buffer), bufs)
180      }
181
P 182    pub fn is_read_vectored(&self) -> bool {
V 183        false
184      }
185
P 186    pub fn read_buf(&self, cursor: BorrowedCursor<'_>) -> io::Result<()> {
V 187        crate::io::default_read_buf(|buffer| self.read(buffer), cursor)
188      }
189
P 190    pub fn write_vectored(&self, bufs: &[IoSlice<'_>]) -> io::Result<usize> {
V 191        crate::io::default_write_vectored(|buffer| self.write(buffer), bufs)
192      }
193
P 194    pub fn is_write_vectored(&self) -> bool {
V 195        false
196      }
197
198      // Optimization hint for std. Returns None since file_attr is unsupported.
P 199    pub fn size(&self) -> Option<io::Result<u64>> {
V 200        None
201      }
202
203      // Not supported by the D3OS naming service.
P 204    pub fn file_attr(&self) -> io::Result<FileAttr> {
P 205        unsupported()
206      }
P 207    pub fn truncate(&self, _size: u64) -> io::Result<()> {
```

```
P 208           unsupported()
  209       }
P 210       pub fn duplicate(&self) -> io::Result<File> {
P 211           unsupported()
  212       }
P 213       pub fn set_permissions(&self, _perm: FilePermissions) -> io::Result<()> {
P 214           unsupported()
  215       }
P 216       pub fn set_times(&self, _times: FileTimes) -> io::Result<()> {
P 217           unsupported()
  218       }
P 219       pub fn lock(&self) -> io::Result<()> {
P 220           unsupported()
  221       }
P 222       pub fn lock_shared(&self) -> io::Result<()> {
P 223           unsupported()
  224       }
P 225       pub fn unlock(&self) -> io::Result<()> {
P 226           unsupported()
  227       }
P 228       pub fn try_lock(&self) -> Result<(), TryLockError> {
P 229           Err(TryLockError::Error(unsupported_err()))
  230       }
P 231       pub fn try_lock_shared(&self) -> Result<(), TryLockError> {
P 232           Err(TryLockError::Error(unsupported_err()))
  233       }
  234   }
  235
  236   // Seek syscall takes usize; reject negative offsets early to prevent wrap-around.
E 237   fn non_negative_offset(offset: i64) -> io::Result<usize> {
E 238       if offset < 0 {
E 239           Err(io::Error::new(io::ErrorKind::InvalidInput,
E 240               "D3OS seek does not support negative offsets"))
E 241       } else {
E 242           Ok(offset as usize)
  243       }
  244   }
  245
V 246   impl Drop for File {
V 247       fn drop(&mut self) {
  248           // Close the handle; nothing sensible to do with an error while dropping.
R 249           let _ = syscall(SystemCall::Close, &[self.0]);
  250       }
  251   }
  252
V 253   impl fmt::Debug for File {
V 254       fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
  255           // Mirrors solid.rs: using `&self.0` directly since D3OS holds a plain `usize`.
V 256           f.debug_struct("File").field("fd", &self.0).finish()
  257       }
  258   }
```

Literatur

[1] Lehrstuhl für Betriebssysteme, Heinrich-Heine-Universität Düsseldorf. "D3OS". (o. J.), Adresse: `https://github.com/hhu-bsinfo/D3OS` (besucht am 13. 09. 2026).

[2] Android Team, Google. "`no_std`, Comprehensive Rust". (o. J.), Adresse: `https://google.github.io/comprehensive-rust/bare-metal/no_std.html` (besucht am 13. 09. 2026).

[3] D. Baranowski, "Porting the Rust standard library to D3OS", Bachelorarbeit, Heinrich-Heine-Universität Düsseldorf, Operating Systems Group, Feb. 2026.

[4] The Rust Organization. "The Rust Standard Library". (o. J.), Adresse: `https://doc.rust-lang.org/stable/std/index.html` (besucht am 13. 09. 2026).

[5] The Rust Organization. "Unstable Features, The Cargo Book". Abschnitt `build-std`. (o. J.), Adresse: `https://doc.rust-lang.org/cargo/reference/unstable.html` (besucht am 13. 09. 2026).

[6] The Rust Organization. "The rustup Book". Abschnitte *Concepts*, *Toolchains* und *Overrides*. (o. J.), Adresse: `https://rust-lang.github.io/rustup/` (besucht am 13. 09. 2026).

[7] "Porting Rust standard library, OSDev Wiki". (o. J.), Adresse: `https://wiki.osdev.org/Porting_Rust_standard_library` (besucht am 13. 09. 2026).

[8] The Rust Organization. "Platform Support, The rustc Book". (o. J.), Adresse: `https://doc.rust-lang.org/rustc/platform-support.html` (besucht am 13. 09. 2026).

[9] The Rust compiler team. "What Bootstrapping does, Rust Compiler Development Guide". (o. J.), Adresse: `https://rustc-dev-guide.rust-lang.org/building/bootstrapping/what-bootstrapping-does.html` (besucht am 13. 09. 2026).

[10] The Rust Organization. "Trait `std::io::Read`, The Rust Standard Library". (o. J.), Adresse: `https://doc.rust-lang.org/std/io/trait.Read.html` (besucht am 13. 09. 2026).

[11] The Rust Organization. "The Rust Release Channel Layout, Rust Forge". (o. J.), Adresse: `https://forge.rust-lang.org/infra/channel-layout.html` (besucht am 13. 09. 2026).

[12] The Rust Organization. "Macro `std::cfg_select`, The Rust Standard Library". (o. J.), Adresse: `https://doc.rust-lang.org/std/macro.cfg_select.html` (besucht am 13. 09. 2026).

[13] The Rust Release Team. "Announcing Rust 1.95.0, Rust Blog". (16. Apr. 2026), Adresse: `https://blog.rust-lang.org/2026/04/16/Rust-1.95.0/` (besucht am 13. 09. 2026).

[14] H. J. Lu, M. Matz, M. Girkar, J. Hubička, A. Jaeger und M. Mitchell, "System V Application Binary Interface, AMD64 Architecture Processor Supplement (With LP64 and ILP32 Programming Models)", Version 1.0, 2025, Anhang *A.2.1 Calling Conventions*.

[15] The Rust Organization. "The Unstable Book". (o. J.), Adresse: `https://doc.rust-lang.org/unstable-book/` (besucht am 13. 09. 2026).

[16] The Rust Organization. "Inline assembly, The Rust Reference". (o. J.), Adresse: `https://doc.rust-lang.org/reference/inline-assembly.html` (besucht am 13. 09. 2026).

[17] The Rust Organization. "External blocks, The Rust Reference". Abschnitt *ABI*. (o. J.), Adresse: `https://doc.rust-lang.org/reference/items/external-blocks.html` (besucht am 13. 09. 2026).

[18] The Rust Organization. "Functions, The Rust Reference". Abschnitt *Extern function qualifier*. (o. J.), Adresse: `https://doc.rust-lang.org/reference/items/functions.html` (besucht am 13. 09. 2026).

[19] The Rust Organization. "Cargo.toml vs Cargo.lock, The Cargo Book". (o. J.), Adresse: `https://doc.rust-lang.org/cargo/guide/cargo-toml-vs-cargo-lock.html` (besucht am 13. 09. 2026).

[20] The Rust Organization. "Package ID Specifications, The Cargo Book". (o. J.), Adresse: `https://doc.rust-lang.org/cargo/reference/pkgid-spec.html` (besucht am 13. 09. 2026).

[21] The Rust Organization. "Overriding Dependencies, The Cargo Book". (o. J.), Adresse: `https://doc.rust-lang.org/cargo/reference/overriding-dependencies.html` (besucht am 13. 09. 2026).

[22] The Rust Organization. "Enum `rustc_target::spec::Os`, nightly-rustc API Documentation". (o. J.), Adresse: `https://doc.rust-lang.org/nightly/nightly-rustc/rustc_target/spec/enum.Os.html` (besucht am 13. 09. 2026).

[23] The Rust Organization. "Enum `rustc_target::spec::Arch`, nightly-rustc API Documentation". (o. J.), Adresse: `https://doc.rust-lang.org/nightly/nightly-rustc/rustc_target/spec/enum.Arch.html` (besucht am 13. 09. 2026).

[24] The Rust Organization. "Source code repository for Rust, Version 1.94.0". Datei `library/std/src/sys/alloc/xous.rs`. (2026), Adresse: `https://github.com/rust-lang/rust/blob/1.94.0/library/std/src/sys/alloc/xous.rs` (besucht am 13. 09. 2026).

[25] A. Crichton. "`src/xous.rs`, A port of dlmalloc to Rust". Version 0.2.10. (2025), Adresse: `https://github.com/alexcrichton/dlmalloc-rs/blob/0.2.10/src/xous.rs` (besucht am 13. 09. 2026).

[26] A. Crichton. "Struct `Dlmalloc`, A port of dlmalloc to Rust". Version 0.2.10. (2025), Adresse: `https://docs.rs/dlmalloc/0.2.10/dlmalloc/struct.Dlmalloc.html` (besucht am 13. 09. 2026).

[27] The Open Group. "General Terminal Interface, The Open Group Base Specifications Issue 8, IEEE Std 1003.1-2024". Abschnitt *11.1.6 Canonical Mode Input Processing*. (2024), Adresse: `https://pubs.opengroup.org/onlinepubs/9799919799/basedefs/V1_chap11.html` (besucht am 13. 09. 2026).

[28] The Open Group. “open, The Open Group Base Specifications Issue 8, IEEE Std 1003.1-2024”. (2024), Adresse: `https://pubs.opengroup.org/onlinepubs/9799919799/functions/open.html` (besucht am 13. 09. 2026).

[29] The Rust Organization. “`cargo test`, The Cargo Book”. (o. J.), Adresse: `https://doc.rust-lang.org/cargo/commands/cargo-test.html` (besucht am 13. 09. 2026).

Abbildungsverzeichnis

2.1 Die drei Ebenen der Rust-Standardbibliothek und die beiden Bauwege in D3OS. 4
2.2 Die Kopplung vom Eintrag in `rust-toolchain.toml` im D3OS-Repository bis zum bootenden Abbild. Gezeigt ist der Stand des übernommenen Ports mit `rustc 1.90`. 6

3.1 Gesamtsicht auf die vier Module. Grau hinterlegt und dick umrandet ist die Portierungsschicht, also der Teil, den diese Arbeit geschrieben hat. Darüber und darunter liegt jeweils fremder Code. 15
3.2 Die Komponenten der Standardbibliothek für D3OS vor und nach dieser Arbeit. 17

5.1 Die drei Schichten des Allokators. Die Schichten 3 und 2 laufen bei jeder Anforderung, Schicht 1 nur bei Nachschubbedarf. Grau hinterlegt und dick umrandet ist der Teil, den diese Arbeit geschrieben hat. 31
5.2 Der Weg des Datenzeigers von der Portierungsschicht bis in den neuen Thread. Nur `args[0]` und `args[1]` überqueren die Systemaufruf-Grenze, und `mov rdi, rsi` macht das zweite Argument zum ersten Parameter des Trampolins. 38

Tabellenverzeichnis

2.1 Gewählte Vorlagen je Modul und das jeweils ausschlaggebende Kriterium. . 9

3.1 Die vier Anker der Sprungkette, bestimmt aus den Release-Manifesten . . . 10
3.2 Zuordnung der Entwurfsfragen zu ihren Quellen am Beispiel des Dateibackends 12
3.3 Vier Klassen von Abweichungen und der jeweilige Ort ihrer Behandlung . . 13

4.1 Die Impact-Analyse aus Abschnitt 4.1 und ihre Abrechnung nach drei Etappen. Die letzte Spalte nennt die Stellen, an denen tatsächlich ein Eingriff erforderlich war. Das in Etappe C ergänzte neunte Feature-Gate zählt nicht dazu, da es keine der erfassten Stellen betrifft. 28
4.2 Minor-Versionen, Anpassungsklassen und Konflikte je Etappe 29

5.1 Die vier geprüften Orte für den fehlenden Zeilenumbruch und der jeweilige Grund für das Ausscheiden. 33
5.2 Herkunft der 193 zuordenbaren Zeilen in `sys/fs/d3os.rs` zum Stand von Commit `c98f0fa`. Die übrigen Zeilen der Datei sind leer 40

6.1 Was die vier Testanwendungen prüfen und unter welcher Bedingung sie fehlgeschlagen wären 43
6.2 Die geprüften Äquivalenzklassen der Standardeingabe mit Vorhersage und Beobachtung 44

A.1 Die 25 eigenen Commits in zeitlicher Reihenfolge. 51
B.1 Herkunft der 138 zuordenbaren Zeilen in `sys/fs/d3os.rs`. Nicht markiert sind leere Zeilen, Zeilen mit nur einer schließenden Klammer und die Kommentare. 52